Ingegneria del sw

Modulo 1: introduzioni, qualità del software e caratteristiche di Java

Ingegneria del Software: disciplina che mette in pratica un insieme di metodi, strumenti e principi per produrre software di qualità. Il termine qualità non si riferisce soltanto alla correttezza funzionale, ma soprattutto alla **manutenibilità**: un codice ben scritto è un codice che può essere modificato, esteso e corretto nel tempo senza sforzo sproporzionato.

L'**IA generativa**, ovviamente, non basta:

- Il codice generato dall'AI può contenere **difetti o problemi di sicurezza**: bisogna saper fare debug e diagnosticare il problema.
- L'AI riusa soluzioni esistenti, ma **non progetta l'intero sistema** e potrebbe non trovare il modo di risolvere problemi nuovi.
- L'AI potrebbe **non riuscire a inventare nuovi algoritmi** o trasformare idee originali in sistemi software.
- Conoscere l'ingegneria del software permette di **formulare prompt più precisi** e ottenere risultati migliori dall'AI.

Qualità del software

A differenza dell'hardware, il software non è costituito da atomi: è **intrinsecamente modificabile.** Un software di successo sopravvive all'hardware per cui era stato sviluppato, generando continuamente nuove esigenze di adattamento. Le richieste di estensione aumentano al crescere del successo del sistema.

I criteri per cui un sistema software si definisce di qualità sono i seguenti:

- **Correttezza:** il sistema aderisce allo scopo ed è conforme alle specifiche (requisiti). La verifica avviene tramite il testing. È importante che il sistema

software faccia quello che il cliente vuole.

- **Manutenibilità:** facilità a introdurre modifiche. È la qualità più importante in Ing. del Software. Un software di successo deve poter essere aggiornato, corretto ed esteso per anni.
- **Efficienza:** nessuno spreco di risorse (RAM, CPU). Manutenibilità ed efficienza vanno spesso in **direzione opposta**: un codice più efficiente tende ad essere meno leggibile. Si preferisce la manutenibilità, salvo in HPC (High Performance Computing), dove le prestazioni giustificano codice più complesso.
- **Dependability:** include **security** (protezione dei dati da accessi non autorizzati), **safety** (il sistema non mette in pericolo persone o cose) e **reliability** (probabilità di operare senza guasti, ovvero che il sistema esegua correttamente la sua funzione specifica per un determinato periodo di tempo). In generale, è la capacità di un sistema informatico di fornire un servizio su cui è possibile riporre, in modo giustificato, un alto livello di fiducia.
- **Usabilità:** capacità di fornire all'utente condizioni di uso efficaci. Cioè misura quanto il sistema è facile, efficace e piacevole da usare per i suoi utenti finali.

Processo di sviluppo del software

Le fasi principali sono:

1. **Analisi dei requisiti:** raccolta delle informazioni su ciò che il sistema deve fare. Un obiettivo singolo (user story) rappresenta una singola funzionalità che l'utente potrà usare. I requisiti cambiano nel tempo: bisogna coinvolgere il cliente il prima possibile. Se i requisiti non sono scritti bene, è impossibile verificare se il sistema li soddisfa.
2. **Progettazione:** si usano OOP, Design Pattern, stili architetturali e UML per definire l'architettura.
3. **Implementazione:** scrittura del codice.
4. **Testing:** esecuzione controllata per verificarne la correttezza.
5. **Manutenzione:** modifiche dopo la consegna: correzione di bug, refactoring, nuove feature, adeguamento a nuove piattaforme.

Esempi di processi: modello **a cascata** (sequenziale), modelli **agili** come XP (Extreme Programming, iterativo e incrementale).

- **Modello a cascata:** ogni fase scende nella successiva come l'acqua di una cascata, non si può tornare su. Quindi il flusso è come il seguente: Requisiti → Progettazione → Implementazione → Test → Manutenzione. Il vantaggio di questo approccio è che porta a processi molto documentati e scadenze chiare se il progetto è semplice. Il problema è che se ci si accorge di un errore dopo mesi, allora tornare indietro costa moltissimo: è **poco flessibile.**
- **Modelli agili (Extreme Programming - XP):** è un approccio **iterativo** (si ripete a cicli) e **incrementale** (aggiunge un pezzetto alla volta). Il lavora si articola in cicli brevi: alla fine di ogni ciclo, si consegna una parte di software funzionante. È una versione estrema che punta tutto sulla qualità tecnica. Include pratiche come la Pair Programming e test continui. Il cambiamento è il benvenuto, il cliente collabora costantemente e vede i progressi ogni settimana. Il contro è che richiede un team molto esperto e un cliente molto presente.

Caratteristiche del linguaggio Java

JDK e stack di esecuzione

L'**applicazione in Java** viene compilata una volta e questo produce il bytecode. Il **bytecode** verrà poi interpretato dalla **JVM** della specifica architettura che vuole eseguire il programma. Successivamente ci sono le varie operazioni di **I/O**: il layer di I/O consiste sia di interazioni con l'utente che interazioni con risorse esterne, come file su disco, connessioni di rete, socket, etc. Infine, le istruzioni native ottenute grazie alla traduzione del bytecode da parte della JVM vengono eseguite fisicamente dalla CPU.

Bytecode: formato intermedio in cui **javac** (**compilatore statico** Java) compila il codice Java. Non è codice macchina per una CPU specifica, ma non è nemmeno il sorgente .java leggibile. È un set di istruzioni pensate per una **CPU immaginaria** (JVM). Ogni istruzione è codificata come un byte, e il set di istruzioni è semplice e generico.

JDK (Java Development Kit): per fare tutte le operazioni sopra citate, necessitiamo chiaramente della JDK, ovvero dell'insieme completo degli strumenti per sviluppare in Java. Comprende il compilatore **javac**, i tool di debug e la **JRE.**

JRE (Jara Runtime Environment): sottoinsieme della JDK, comprende le **librerie** standard di Java e la **JVM**.

JVM (Java Virtual Machine): si occupa di tradurre il bytecode nelle istruzioni native della CPU dell'architettura di riferimento.

JIT (Just In Time): è un **compilatore dinamico** (in quanto compila durante l'esecuzione) che trasforma il bytecode in codice macchina nativo durante l'esecuzione. Si trova dentro la JVM.

Compilazione: per compilare scriviamo javac NomeClasse.java, che genera NomeClasse.class. Per l'esecuzione, invece, ci basta scrivere java NomeClasse. Questo a patto che la classe abbia public static void main(String[] args). Per file sorgente singoli posso anche scrivere direttamente java NomeClasse.class, ma usare librerie esterne è più difficile. In ogni caso, il file NomeClasse.class viene creato internamente perché il file viene compilato, solo che non viene salvato su disco.

Convenzioni

- **Nome di classe:** inizia con la **maiuscola**.
- **Nome di variabile/metodo: camelCase** minuscolo.
- I **metodi** hanno nomi che sono **verbi**. I **sostantivi** descrivono le classi.
- I **commenti** costituiscono circa il **30% della lunghezza del codice.**

Keyword e comportamenti di Java

- **static:** usata per dichiarare un campo o un metodo appartenente alla classe, e non all'istanza. È importante notare che il main di una classe è una funziona statica, per cui al suo interno potrà chiamare altri metodi statici. Ovviamente può anche chiamare metodi non statici, l'importante è che ci sia un'istanza a chiamare il metodo in questione. Quindi dal main non possiamo chiamare direttamente metodi non statici della stessa classe senza prima creare un'istanza.

- **Variabili e valori di default:** le variabili di istanza (ovvero dichiarate dentro la classe, fuori dai metodi) non inizializzate valgono: 0 per tipi numerici, false per boolean, null per riferimenti. Le variabili locali (dichiarate, invece, dentro un metodo) **devono** essere inizializzate prima dell'uso.
- **Import e package:** le classi nel package java.lang (es. String, System, Math) sono importate automaticamente. Questo perchè java.lang è il package fondamentale di Java, e viene importato automaticamente in ogni file Java. Per tutto il resto bisogna fare import manuali.
  Esempio: se scriviamo `import java.time.*`, allora questo non è uno spreco: il compilatore include nel bytecode solo le classi effettivamente usate: `.*` non causa spreco.
- **Enhanced for:** iterazione su tutti gli elementi, senza però dover gestire gli indici manualmente. Una sorta di ciclo for avanzato. Zucchero sintattico, in realtà. Il compilatore lo trasforma internamente in un while con hasNext e nel corpo del while usa next. Per capire meglio, vedi la parte successiva su **Iterable.** Qulasiasi classe implementi Iterable diventa compatibile con l'enhanced for automaticamente. Anche le mie classi personalizzate posso usarlo, basta che implementino Iterable.
  ```
  for (float v : valori) totale += v;
  ```
- **Collection:** un insieme di interfacce e classi della JRE che servono a gestire **gruppi di oggetti.** Stanno nel package java.util. Offrono strutture dati già pronte: non dobbiamo implementare un array dinamico, una lista concatenata o una mappa da zero. Le tre interfaccie principali sono:
  1. **List:** sequenza ordinata, ammette duplicati, accesso per indice.
  2. **Set:** insieme, non ammette duplicati, nessun ordine garantito.
  3. **Map:** coppie chiave-valore, le chiavi sono uniche.

  Tutte e tre sono delle interfacce, e ammettono implementazioni diverse (ArrayList vs LinkedList, HashSet vs TreeSet, HashMap vs TreeMap).

  List e Set estendono **Collection.** Collection raggruppa elementi singoli, mentre Map raggruppa delle coppie chiave valore. Per questo non condividono la stessa gerarchia. Map è a parte, non estende Collection.

Collection estende **Iterable.** È un'**interfaccia semplicissima:** definisce un solo **metodo iterator.** Significa "questo oggetto sa produrre un iteratore". Un iterator è un oggetto che sa scorrere una sequenza. Possiede hasNext() per sapere se ci sono altri elementi, e next() per prendere il prossimo.

Differenza tra interfacce e classi astratte

Le interfacce non contengono nessuna implementazione. Le classi astratte possono avere metodi parzialmente implementati (alcuni con corpo, alcuni no). Si usa `abstract` in una classe astratta per i metodi che si vuole **forzare** le sottoclassi a implementare. Le sottoclassi della classe astratta che non implementano tutti i metodi astratti devono a loro volta essere dichiarate `abstract`.

Nota (sottile): se una sottoclasse eredita dei metodi da una superclasse e fa override di uno di essi, nel diagramma UML il metodo ridefinito viene mostrato nella sottoclasse. Più si va in basso nella gerarchia, più i nomi diventano specifici.

Interfacce e implementazioni

Per capire meglio, partiamo con un esempio.

- **List e ArrayList:** List è un'interfaccia che definisce add(), get(), size(), remove() e contains(), che serve per scorrere la lista e controllare se è presente al suo interno l'elemento cercato (con equals). ArrayList la implementa come un array a ridimensionamento automatico. Usare i **Generics** (**<T>**) garantisce **type-safety** a compile time. Esistono molte **implementazioni** di List: **LinkedList, Vector.**

Un'**interfaccia** è un **contratto.** Definisce cosa un oggetto sa fare, senza dire come lo fa. Definisce un tipo senza implementarne il codice, elencando le signature dei metodi, senza fornirne il corpo. Contiene **solo firme** di metodi (il nome, i parametri, il tipo di ritorno), ma non contiene **nessuna implementazione**. Niente attributi non inizializzati, niente costruttori. Chi implementa l'interfaccia (una classe) si impegna a fornire il corpo di quei metodi. Il vantaggio è che l'interfaccia **nasconde i dettagli implementativi,** e il codice che usa List non sa se sotto c'è un ArrayList o una LinkedList, ma sa che può chiamare operazioni come add, get, size, etc.

Digressione sugli attributi: le interfacce possono avere attributi, ma devono essere **costanti** (public static final). Non possono avere attributi di istanza non inizializzati. Quando dichiariamo una variabile all'interno dell'interfaccia, il compilatore la tratta **sempre** come **public static final**, anche quando non lo specifichiamo.

Differenza tra interfacce e classi astratte

Cos'è una classe astratta? Una classe astratta è **parzialmente implementata**: alcuni metodi sono implementati, altri no (dichiarati **abstract**). Una classe astratta non può essere istanziata.
Un **metodo abstract** è utile perché altri metodi della stessa classe (già implementati) possono invocarlo. I client si aspettano di poterlo invocare. E, inoltre, un metodo abstract costringe le sottoclassi concrete a implementarlo.

Nota: il nome di una classe abstract, in UML, è scritto in **corsivo.** Il metodo abstract viene anch'esso scritto in corsivo (sia il nome che i parametri).

- **Differenza** tra **classe astratta** e **interfaccia:** una classe può implementare **più interfacce.** Una classe astratta è, innanzitutto, parzialmente implementata. Può contenere sia metodi astratti (senza corpo, come un'interfaccia), che concreti (con corpo). Una classe può estendere **una sola classe astratta.**
- **Regole di utilizzo:** usiamo un'interfaccia quando vogliamo definire capacità che **classi diverse e non correlate** possono condividere. Classi che non hanno nulla in comune tranne il fatto di rispettare un certo contratto. Usiamo una classe astratta quando abbiamo una vera relazione fra le classi, quando la classe figlia "è un" della classe madre. Es. un cane (classe figlia) è un animale (classe madre). Si usa quando le classi hanno dei comportamenti condivisi, come per esempio il comportamento "respira", che è proprio sia di Cane che di Gatto, ma hanno la differenza che implementano "verso" diversamente.
- Ricordiamo infatti che una classe può estendere solo una classe, e non due, per evitare il **diamond problem:** se ho due classi con il metodo respira che hanno un figlio in comune, quale metodo respira utilizza il figlio? Per questo Java **non permette di estendere due classi.**

Note sull'interfaccia: tramite l'interfaccia, i client sanno cosa possono invocare. Per istanziazioni di oggetto e invocazioni di metodo, si può usare una qualsiasi

delle implementazioni disponibili per l'interfaccia. Es. le implementazioni disponibili per List sono molte, come ArrayList, LinkedList, etc. Un client che usa un'interfaccia rimane immutato quando l'implementazione dell'interfaccia cambia.

Principio fondamentale: programmare per le interfacce, non per le implementazioni. Significa che quando dichiariamo una variabile, un parametro o un tipo di ritorno, dobbiamo usare il più astratto possibile (l'interfaccia), e non la classe concreta. Es. se il metodo ritorna un ArrayList di stringhe, devo in realtà scrivere che ritorna una List di stringhe. Questo riduce la **propagazione dei cambiamenti** e rende più facile il **testing**.

1. **Propagazione dei cambiamenti:** supponiamo di avere un metodo che prende in input un ArrayList di stringhe. Scopro più avanti che in realtà mi serve una LinkedList perché ho molte inserzioni in mezzo. Devo cambiare la firma del metodo e tutti i posti nel codice che lo chiamano. Il cambiamento si propaga, ed è molto soggetto ad errori. Se invece scrivo direttamente che il metodo prende in input una List di stringhe, posso passare ArrayList, LinkedList, etc. Quindi devo dichiarare il tipo come List e istanziare come ArrayList. Il codice che usa un'interfaccia rimane **immutato** quando l'implementazione cambia. Quindi quando cambiano l'implementazione sottostante non dobbiamo modificare il **client** (ovvero il codice che usa l'interfaccia). Se lo facessimo, significa che stiamo sbagliando qualcosa e dobbiamo astrarre di più.
2. **Testing:** facilita il testing perché l'interfaccia mi permette di sostituire un'implementazione reale con un'**implementazione fittizia** durante i test. Ovviamente questo riguarda gli **unit test:** testano che la logica di una singola classe funzioni correttamente, isolata da tutto il resto. I test che invece usano **implementazionio reali** si chiamano **integration test**, effettuati su databasi veri, rete vera, file system vero, etc. Verifica che tutti i pezzi funzionino correttamente. La pratica standard è avere **molti unit test** (veloci, girano a ogni modifica) e **alcuni integration test** (girano meno frequentemente).

```
List<Float> valori = new ArrayList<>();  // tipo dichiarato come interfaccia
```

Ereditarietà

Non è pratico copiare una classe e modificarla: si avrebbe una proliferazione di classi e tanto lavoro. Si vuole quindi riusare classi esistenti per implementare attributi e metodi leggermente diversi. Il **riuso delle classi** deve avvenire:

1. Senza dover modificare codice esistente e funzionante.
2. In modo semplice per il programmatore.

L'**ereditarietà** permette di definire una nuova classe **indicando solo cosa ha in più** rispetto a una esistente, come attributi o metodi nuovi, e modificando quelli esistenti.

```
public class Studente extends Persona { ... }
```

Studente è la **sottoclasse**, Persona è la **superclasse**.

Cosa può fare la sottoclasse?

- **Eredita** tutti i metodi e gli attributi della superclasse e può usarli come se fossero definiti localmente.
- **Aggiunge** nuovi attributi e metodi.
- **Ridefinisce (override)** metodi della superclasse.
- **Non può eliminare** metodi o attributi della superclasse.

Come funziona la visibilità nell'ereditarietà?

- **private**: visibile solo alla classe, non alla sottoclasse.
- **public**: visibile a tutti, anche alla sottoclasse.
- **protected**: visibile alle sottoclassi ma non alle altre classi.

Funzionamento di super e override:

- **super.nomeMetodo()** permette di chiamare il metodo della superclasse dall'interno di una sottoclasse. È fondamentale quando si fa override di un metodo e si vuole mantenere anche il comportamento del genitore.
- **@Override** è un'annotazione che dice al compilatore: "sto ridefinendo un metodo già esistente nella superclasse". Il compilatore verifica che il metodo sia davvero presente nella superclasse, proteggendo da errori di battitura. Se un metodo non è presente nella superclasse, @Override segnalerà un errore di compilazione.

Nota: il nome della sottoclasse deve comunicare a quale classe è simile e come è diversa. Inoltre, classi che servono come radici di una gerarchia devono avere

nomi concisi, mentre le classi che sono più in basso nella gerarchia possono avere un nome più lungo.

Compatibilità fra tipi e Casting

L'ereditarietà definisce una classificazione di tipi: una sottoclasse è un **sottotipo compatibile** con la superclasse. Una sottoclasse fa tutto ciò che fa la superclasse, e altre cose in più.

```
Studente s = new Studente("Alan", "Rossi", "M12345");
Persona p = s;   // OK: Studente è compatibile con Persona
p.printAll();    // a runtime chiama printAll() di Studente
```

Attenzione: non vale il contrario. Non si può usare la superclasse dove si usava la sottoclasse. Ovvero non posso assegnare una variabile di tipo Persona a una variabile di tipo Studente. Il compilatore lo **blocca.** Persona non ha i metodi specifici di uno Studente, così come non ha la matricola. **Ogni studente è una persona**, e quindi Studente è compatibile con Persona, **ma non ogni persona è uno studente**, e quindi Persona non è compatibile con Studente. Diciamo che Studente è compatibile con Persona perché possiamo assegnare a una variabile di tipo Persona una variabile di tipo Studente.
Se p è dichiarato di tipo Persona, non posso chiamare p.nuovoEsame() anche se a runtime p punta a uno Studente: **il compilatore lo vieta** perché a **compile time** conosce solo il tipo Persona.

Importante: tuttavia, ci potrebbero essere dei casi in cui siamo sicuri che la variabile di tipo Persona a cui ci stiamo riferendo è in realtà uno Studente a runtime. In questo caso, possiamo **forzare il casting esplicito.**

```
Persona p = new Studente(...);
Studente s = (Studente) p;   // OK a runtime, perché p
                                                    // p
unta davvero a uno Studente
```

Il cast è valido per il compilatore, ma a runtime genera **ClassCastException** se l'istanza non è del tipo corretto. Va usato con cautela, non va abusato.
Se forziamo un casting e in realtà si scope che quella Persona era veramente solo

una Persona e non uno Studente, allora verrà generato l'errore ClassCastException.

Nota importante: se p a runtime è di tipo Studente, non potrà comunque invocare il metodo nuovoEsame(), in quanto questo **a compile time verrebbe proibito,** dato che non è un metodo di Persona. Si potrebbe ovviamente risolvere tramite **casting forzato.**

Record

In Java il **record** è una classe speciale progettata per tenere **dati immutabili.**

```
public record Esame(String materia, int voto) { }
```

Caratteristiche:

- Fornisce automaticamente **l'implementazione dei metodi getter** per i suoi campi, con lo stesso nome del campo (non `getMateria()`, ma `materia()`).
- I **campi** sono **final**: non modificabili dopo l'assegnazione.
- Viene fornito automaticamente un **costruttore** che assegna tutti i campi. Non ci sono setter.
- I **record** sono **final**: **non possono essere estesi** da sottoclassi.
- Possono implementare **interfacce**, ma non possono estendere classi.
- Possono avere campi static, metodi static e metodi di istanza aggiuntivi.

Sono utili quando si ha bisogno di raggruppare dati all'interno di un'istanza senza volerli modificare.

Sintomi di buona e cattiva programmazione

Cattiva programmazione: Spaghetti Code

Rappresenta l'**antipattern.** Si parla di spaghetti code quando si verificano alcune delle seguenti condizioni:

1. Si ha davanti un codice con metodi lunghi, senza parametri e che usano variabili globali.

2. Flusso di esecuzione determinato dall'implementazione interna dell'oggetto, non dai chiamanti.
3. **Interazioni minime** fra gli oggetti.
4. Nomi classi e metodi indicano la **programmazione procedurale.** Ovvero pensare in termini di procedure, e non di oggetti. Vengono create classi, ma vengono poi usate come semplici contenitori di funzioni, senza sfruttare l'OOP. Il segnale classico sono i nomi. Classi con nomi che descrivono **azioni** e non **entità.** Invece di chiedere i dati a un oggetto e poi manipolarli fuori, bisogna dire all'oggetto di fare qualcosa (**Tell, Don't Ask).**
5. **Ereditarietà e polimorfismo** non usati, quindi riuso impossibile.
6. Gli oggetti non mantengono lo stato fra le invocazioni. Ovvero l'oggetto **non ricorda** cosa è successo nelle chiamate precedenti. Se non mantiene lo stato, ogni chiamate è indipendente e l'oggetto non ha memoria. Un oggetto incapsula sia i **dati (lo stato)** sia le **operazioni** su quei dati. Se le operazioni non modificano né leggono uno stato interno, allora non stiamo davvero usando gli oggetti nel modo corretto.

Cause: inesperienza con OOP, nessuna progettazione.

Qualità del codice

Vi sono vari segnali da cui si può evincere che si sta scrivendo codice nel modo corretto.

1. **Correttezza:** è stato possibile eseguire test che verificano il codice. **Soddisfare (e verificare) i requisiti** permette di valutare e ottenere la qualità del sistema software. I test **documentano le condizioni sotto le quali il codice funziona e proteggono il codice da modifiche indesiderate.** I test devono essere **indipendenti** fra loro e **auto valutarsi** (ovvero bisogna confrontare il risultato con il valore atteso).
2. **Test Driven Development (TDD):** implementare prima il test, e dopo il codice che risolve un requisito. Durante l'implementazione dei test, si scelgono nomi di classi e metodi (si sta facendo **progettazione).**
   Il motto è: **Red, Green, Refactor.**

- Red indica test non superato. Si possono implementare (e modificare) le classi. Quindi prima si scrive il test, **fallisce** perché la classe non esiste ancora o non è completa. A questo punto si lavora sull'implementazione finché il test non passa.
- Green indica test superato. Si possono implementare nuovi test per nuove funzionalità, oppure si può fare refactoring.
- **Refactoring:** miglioramento della struttura del codice senza aggiungere funzionalità. I test già verdi garantiscono che il refactor non ha modificato il comportamento esterno del codice (ovvero da parte di chi chiama il metodo). È fondamentale che il refactor venga fatto **solo su codice che già funziona e che è corretto.**

Nota: in Java non possiamo semplicemente non scrivere i metodi e far partire i test, perché il compilatore ci darà errore, dato che Studente non esiste. La soluzione è creare una **classe stub,** ovvero una versione minimale di Studente che compila ma non fa niente. Per esempio, scriviamo solo le firme dei metodi e facciamo ritornare ai metodi valori nulli (se hanno un tipo di ritorno diverso da void). Poi l'implementazione effettiva di questi metodi verrà dopo aver fallito i test.
Il flusso corretto quindi diventa: scrivere i test, poi creare lo stub, poi compilare (red), poi implementare, poi compilare e rieseguire (green) e infine fare refactoring se necessario.

3. Si sta usando bene il paradigma **OOP.**
4. **Responsabilità:** ogni metodo ha una **sola piccola responsabilità.** Separazione delle varie operazioni, ciascuna in un suo metodo. Ciò permette di verificare la correttezza delle operazioni e di riusarle. Questo permette di ottenere **coesione** del codice, ovvero non vi è un metodo (o una classe) che implementa tanti compiti.
   Principio di Singola Responsabilità (SRP): la singola responsabilità è cruciale per la comprensione, il riuso e l'ereditarietà. Bisogna avere un solo motivo per cambiare il codice di quella parte, ovvero non devono esserci più motivi possibili per modificarlo. Se ci sono più motivi per modificarla, allora la classe ha troppe responsabilità e va spezzata.

5. **Costruzione di astrazioni:** metodi di livello più basso e di livello più alto. Questi ultimi richiamano i precedenti. Lo stesso si fa per la classi. Lo sviluppatore OOP **costruisce astrazioni.** Le astrazioni sono classi che nascondono dettagli di implementazione.
6. Il flusso di chiamate ai metodi è indipendenti dagli algoritmi. Gestito dal chiamante.
7. I servizi offerti dai metodi si possono riusare (richiamandoli).
8. Lo stato dell'oggetto diventa **osservabile**, attraverso metodi opportuni, in modo da poter **fare i test.**
9. Utilizzo del **paradigma Command and Query.**

Command and Query

Consiste nella separazione delle operazioni che aggiornano lo stato (command) da quelle che restituiscono valori (query).

- I metodi **Query** restituiscono un risultato (si vede dal parametro di ritorno) e non modificano lo stato del sistema.
- I metodi **Command (o modificatori)** cambiano lo stato del sistema ma non restituiscono un valore.

I metodi Query si possono chiamare liberamente, senza preoccupazioni sulla modifica dello stato, mentre si deve stare più attenti quando si chiamano i metodi command.

Metriche del codice

LOC (Lines of Code): numero di linee che terminano con un punto e virgola. Indica la dimensione del codice. La media LOC per metodo è una metrica. In progetti open source come JUnit: media ~18 LOC per metodo. Metodi brevi (preferibilmente sotto le 30 righe) sono più leggibili e testabili.

Principi fondamentali dell'ingegneria del sw

- **KISS (Keep it Simple, Stupid):** produrre componenti semplici, facili da capire e gestire. Codice complesso non è sinonimo di codice scritto bene. Nella

codifica si premia la semplicità.

- **DRY (Don’t Repeat Yourself):** il codice non deve essere duplicato. Se c’è un bug in una parte di codice che è stata copiata, allora bisogna correggerlo in tutti i posti in cui è stato incollato. Bisogna evitare le ripetizioni nel codice.
- **YAGNI (You Ain’t Gonna Need It):** non bisogna progettare funzionalità che non sono attualmente utili. Bisogna scrivere codice solo per soddisfare **requisiti esistenti.** Non bisogna anticipare feature future.
- **LoD (Law of Demeter):** non bisogna parlare con gli estranei. Un oggetto dovrebbe avere conoscenza solo di oggetti correlati (amici). Una classe non dovrebbe dipendere da più di 3-5 classi diverse. Il problema è quando **una** classe dipende da troppe altre, non quando molte classi usano una classe centrale.

Principi per i Test

- **FIRST:** Fast, Isolated/Independent, Repeatable, Self-validating, Timely.
    1. **Fast**: un buon test deve essere veloce, dato che verrà eseguito spesso per avere feedback immediato dopo ogni modifica. Se i test sono lenti, il ciclo Red-Green-Refactor viene rallentato ed è un problema.
    2. **Isolated/Independent:** ogni test deve essere indipendente dagli altri. Il risultato di un test non deve dipendere dall’ordine in cui vengono eseguiti, né dallo stato lasciato da un test precedente.
    3. **Repeatable:** il test deve dare sempre lo stesso risultato a parità di input, indipendentemente da quando e dove viene eseguito.
    4. **Self-Validating:** il test deve dire da solo se ha passato o fallito, senza che il programmatore debba interpretare l’output. Il risultato deve essere binario: verde o rosso.
    5. **Timely:** i test vanno scritti nel momento giusto. Se si segue l’approccio TDD, per esempio, vanno scritti prima del codice. Se li scriviamo dopo, invece, tenderemo a scrivere dei test che si adattano al codice già scritto invece di testare davvero il comportamento atteso.
- **TDD:** vedi sopra. Prima implementare il test e poi il codice dell’applicazione.

- **BDD (Behaviour-Driven Development):** estensione del TDD che sposta il focus dal come testo il codice al **come si comporta il sistema** dal punto di vista dell'utente o del dominio. L'idea è descrivere i test in linguaggio naturale, usando tre clausole:

  1. **Given:** il contesto iniziale, lo stato del sistema prima che succeda qualcosa.
  2. **When:** l'evento o l'azione che scatena il comportamento.
  3. **Then:** il risultato atteso, cosa deve succedere dopo.

  Esempio: data una lista di pagamenti (given), faccio la somma (when) e verichiamo che il risultato sia x (then). Il vantaggio è la **leggibilità.** Il test diventa documentazione del comportamento atteso, non solo verifica tecnica.

Nota importante: il TDD è una pratica di sviluppo (scrivere i test prima del codice, seguendo cicli Red, Green, Refactor). Si concentra sul processo e sulla correttezza tecnica. Il BDD è un modo di scrivere e pensare i test. Usa il linguaggio Given/When/Then per descrivere il comportamento dal punto di vista dell'utente. Si concentra sul linguaggio e sulla comunicazione tra sviluppatori e non tecnici.
Il BDD usa il TDD come base: sto ancora scrivendo test prima del codice, ma cambio il modo in cui li esprimo e il livello a cui ragiono. Non penso "testo questo metodo" ma "descrivo questo comportamento del sistema". Questi due approcci sono **complementarsi.**
Posso seguire l'approccio BDD senza quello TDD, ma perdo il beneficio principale, ovvero guidare lo sviluppo in base al comportamento atteso.

Diagrammi UML

UML (Unified Modeling Language) è una notazione grafica standardizzata per rappresentare visivamente un sistema software. Esistono due tipi principali di diagrammi rilevanti per questo corso: diagramma delle classi e diagramma di sequenza.

Diagramma delle classi

Mostra le **interazioni fra le classi.** Un rettangolo rappresenta una classe, suddiviso in tre sezioni: nome, attributi, metodi.

Simboli di visibilità: `-` privato, `+` pubblico, `#` protetto.

```
- nomeAttributo : Tipo ---> indica un attributo privato
+ nomeMetodo(param : Tipo) : TipoRitorno ---> indica un metod
o pubblico
```

Gli attributi e i metodi **sottolineati** sono **statici** (appartengono alla classe, non alle istanze).

Rappresentazione di un'interfaccia: si disegna un rettangolo identico a quello di una classe, ma con la dicitura **<<interface>>** sopra il nome.

In alternativa, notazione a cerchio (**lollipop**): `[ Classe ]—○ (List)`. Nelle interfacce non si indicano attributi, solo metodi.

Frecce nel diagramma delle classi:

- Freccia con triangolo vuoto verso l'alto: **ereditarietà** (B è sottoclasse di A). È importante che l'asticella della freccia non entri dentro il triangolo, sarebbe un grave errore. Il triangolo all'interno deve essere vuoto.
- Freccia con triangolo vuoto tratteggiato (--▷): **implementazione** di un'interfaccia (freccia rivolta verso l'interfaccia).
- Freccia semplice (→): **associazione/dipendenza** (A → C significa che A usa C).

Diagramma di Sequenza UML

Mostra le **interazioni fra oggetti nel tempo** (a runtime). Si usa quando si vuole rappresentare come le cose avvengono durante l'esecuzione, non solo com'è strutturato il codice.

Elementi del diagramma:

- **Asse temporale:** verticale verso il basso. Ciò che viene prima è in alto.
- **Oggetti:** in alto in orizzontale, identificati come `nome:Classe`. I due punti indicano che è un'istanza. Se si conosce solo la classe e non il nome, si scrive `:Classe`.
- **Linea della vita (tratteggiata):** indica che l'oggetto esiste.

- **Barra di attivazione:** rettangolo sulla linea della vita, indica che l'oggetto è attivo (sta eseguendo o aspettando un ritorno).
- **Invocazione di metodo:** freccia piena dalla barra di attivazione di chi chiama verso quella di chi riceve. Il chiamante ha una barra di attivazione più lunga perché attende il ritorno.
- **Ritorno:** freccia tratteggiata con triangolo pieno, può essere omessa.
- `create`**:** parola scritta sulla freccia quando si crea una nuova istanza.
- `loop`**:** frame (riquadro) che racchiude un ciclo ripetuto.
- In Java non esiste un metodo distruttore: non si usa la X per indicare la distruzione dell'istanza. Il garbage collector della JVM gestisce automaticamente la memoria.

Nota: nei diagrammi UML non si può indicare se una chiamata è ricorsiva, e non bisogna illustrare tutto il codice: si rappresenta solo la parte rilevante.

Polimorfismo, Late Binding e DispatchPolimorfismo

Nei sistemi ad oggetti possono esistere **metodi con lo stesso nome** (e stessa signature) in classi diverse. Quando si usa l'ereditarietà e sono stati definiti metodi con lo stesso nome, chiamare un metodo può avere **effetti diversi**: il comportamento è **polimorfo**.

Senza polimorfismo, si dovrebbe inserire uno `switch` sul chiamante per valutare la classe di ciascuna istanza e chiamare il metodo corrispondente: molto più fragile e difficile da mantenere.

Early binding vs Late binding

- **Early binding:** il compilatore guarda il tipo dichiarato della variabile e collega staticamente il codice della classe di quel tipo.
- **Late binding (dinamico):** a runtime, si guarda il tipo **effettivo** dell'istanza (non quello dichiarato della variabile) e si collega il codice della classe dell'istanza. È questo il comportamento standard in Java per i metodi di istanza.

Consideriamo il seguente esempio:

```
Persona px;
if (i > 10) px = p; // p è un'istanza di Persona
else px = s;        // s è un'istanza di Studente
px.printAll();  // a runtime quale printAll() viene chiamato?
```

Il compilatore verifica che printAll() esista per il tipo Persona. A runtime, se px punta a s (uno Studente), viene chiamato printAll() di Studente. Questo è il **late binding**.

Compile time: il tipo dichiarato della variabile determina quali operazioni/signature si possono usare.
Runtime: l'istanza puntata dalla variabile ha sempre un tipo concreto (una classe, mai un'interfaccia), e questo non cambia mai.

Dispatch

Il **dispatch** è il meccanismo con cui Java decide **quale versione di un metodo** chiamare a runtime, basandosi sul tipo reale dell'oggetto e non su quello dichiarato.

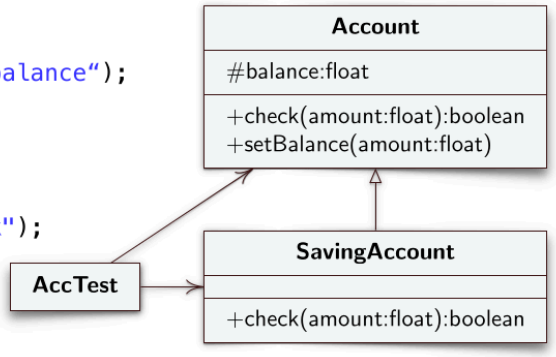
1. Quando un metodo `M()` viene chiamato su un oggetto, si controlla il tipo dell'istanza a runtime.
2. Si cerca `M()` sulla classe dell'istanza: se trovato, si esegue.
3. Se non trovato, si risale nella gerarchia verso le superclassi finché non si trova il metodo.

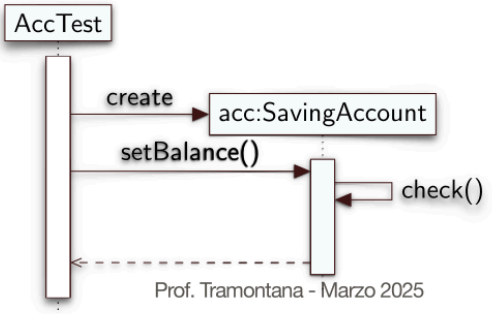
Esempio interessante (Sottoclassi e Dispatch):

```java
public class Account {
   protected float balance = 0;
   public void setBalance(float amount) {
      System.out.println("in Account set-balance“);
      if (check(amount))
         balance += amount;
   }
   public boolean check(float amount) {
      System.out.println("in Account check");
      return balance + amount >= 0;
   }
}

public class SavingAccount extends Account {
   public boolean check(float amount) {
      System.out.println("in Saving-Account check");
      return (balance + amount) >= 1000;
   }
}

public class AccTest {
   public static void main(String[] args) {
      Account acc = new SavingAccount();
      acc.setBalance(1234);
   }
}
```





19 Prof. Tramontana - Marzo 2025

Descrizione del meccanismo: dato che setBalance() non si trova in SavingAccount, allora ne segue che si trova in Account, e quindi si esegue. Dentro setBalance() viene chiamato check(), e quindi si cerca prima in SavingAccount, lo si trova e lo si esegue. È interessante notare che **anche dall'interno di un metodo della superclasse**, la chiamata a un metodo ridefinito nella sottoclasse eseguirà la versione della sottoclasse.

Nota importante: a compile time si dichiara una variabile di un certo tipo, e il tipo determina quali operazioni e quali signature si possono usare. Tale tipo può essere o una classe o un'interfaccia. Invece, il tipo a runtime di un oggetto non cambia mai.

Testing e JUnit

Definizione di test: programma che esegue il codice sotto certe condizioni (valori in input) e valuta se il risultato fornito è corretto.

QUI JUNIT

Modulo 2: Refactoring

Refactoring: è il processo che cambia un sistema software in modo che il comportamento esterno del sistema non cambi, ovvero i requisiti funzionali soddisfatti sono gli stessi, per far sì che la struttura interna sia migliorata.
Si migliora la progettazione a poco a poco, durante e dopo l'implementazione del codice. **Esempio:** eliminare frammenti di codice duplicati all'interno di rami condizionali.

- **Idea di base:** riconoscere che è **difficile** ottenere fin dall'inizio una **progettazione adeguata,** e quindi si deve cambiare. La nuova progettazione permetterà di incorporare più facilmente nuovi requisiti.

Vantaggi del refactoring:

1. Spesso la **dimensione del codice si riduce** dopo il refactoring.
2. Le strutture complicate si trasformano in **strutture più semplici** e più facili da capire e manutenere. **Esempio:** strutture logiche con condizionali molto annidati (metodi lunghissimi con tanti rami), strutture di classi con troppe responsabilità, strutture di dipendenza con componenti fortemente accoppiati fra loro.
3. Si evita di introdurre **debito tecnico** all'interno della progettazione. Ovvero soluzione che funziona a breve termine, ma nel tempo si accumulano degli **interessi**, ovvero il codice diventa sempre più difficile da modificare e da capire. Spesso capita di non riuscire a scrivere subito un'ottima versione del metodo che dobbiamo scrivere, e magari la scriviamo in un modo poco leggibile. Non correggere e refactorizzare quel metodo aumenta il debito tecnico.
   Esempio: scrivere una soluzione rapida e sporca invece di progettarla per bene, a causa delle scadenze.

Perché fare refactoring?

1. Serve per migliorare la progettazione, dato che **senza refactoring la progettazione si deteriora mano a mano che si apportano modifiche** al

codice.

2. Rende il codice più facile da capire, dato che **la struttura del codice viene migliorata** e viene rimosso il codice duplicato.
   Aiuta a scoprire bug, dato che fare refactoring **promuove la comprensione profonda del codice**. Permette di programmare più velocemente, perché una buona progettazione è più facile da cambiare.

Come fare refactoring?

Usando le tecniche di refactoring che vedremo a breve. Dopo aver fatto refactoring, bisogna verificare che tutti i test siano ancora superati. Se un test non è superato, il refactoring ha compromesso qualcosa che funzionava, e bisogna intervenire subito. È importante che il refactoring venga **fatto solo su codice funzionante.** Se il codice non è corretto, non ha senso fare refactoring.

Quando fare refactoring?

Per capire se è necessario fare refactoring, si possono controllare vari sintomi (**bad smell**, o **code smell):**

1. **Codice duplicato:** si usa la tecnica Estrai Metodo.
2. **Metodi lunghi o con commenti all'interno:** usare tecnica Estrai Metodo (insieme a Sostituisci Temp con Query).
3. **Classe di grandi dimensioni:** tiene tante istanze e/o ha tante linee di codice. Si usa la tecnica Estrai Classe.
4. **Lunga lista di parametri per un metodo:** usare tecnica Introduci Oggetto Parametro.
5. **Cambiamenti divergenti:** cambiamenti a parti diverse per motivi diversi. Si usa la tecnica Estrai Classe.

Tecniche di Refactoring

Tecnica 1: Estrai Metodo

Un frammento di codice può essere raggruppato. Far diventare quel frammento un metodo il cui nome spiega lo scopo del frammento.

Motivazioni:

1. Cercare **metodi lunghi** o codice che necessita di commenti per essere comprensibile.
2. Ridurre la lunghezza dei metodi, poiché **metodi piccoli** sono con più probabilità **utilizzabili** richiamandoli da altri metodi.
3. Scegliere un **buon nome** per i metodi piccoli, che esprime ciò che il metodo fa.
4. I metodi di alto livello, quelli che invocano i metodi piccoli, sono più facili da comprendere, sembrano essere costituiti da una sequenza di commenti.
5. Fare **overriding** (ridefinire nella sottoclasse) è più semplice se si hanno metodi piccoli.

Meccanismi:

1. Cercare un nuovo metodo il cui nome comunica l'intenzione del metodo.
2. **Copiare il codice estratto** dal metodo sorgente al nuovo metodo.
3. Guardare se il codice estratto ha **variabili locali** al metodo sorgente e far diventare tali variabili **parametri** del metodo nuovo.
4. Se alcune variabili sono usate solo all'interno del codice estratto, farle diventare **variabili temporanee** del nuovo metodo.
5. Se una variabile locale al metodo sorgente è modificata dal codice estratto, vedere se è possibile far sì che il metodo estratto sia una query e assegnare il risultato della chiamata alla variabile locale del metodo sorgente.
6. Sostituire nel metodo sorgente il codice estratto con una chiamata al metodo nuovo.

Tecnica 2: Sostituisci Temp Con Query

Problema: una variabile temporanea **temp** è usata per tenere il risultato di un'espressione.
Estrarre l'espressione e inserirla in un metodo. Sostituire tutti i riferimenti a temp con la chiamata al metodo che incapsula l'espressione. Il nuovo metodo può essere richiamato anche altrove.

Per esempio:

```
double prezzoBase = quantita*prezzo;
// diventa
private double prezzoBase() {
    return quantita*prezzo;
}
```

Motivazioni:

1. **Variabili temp** sono temporanee e locali, possono essere viste solo all'interno del contesto del metodo e per questo **inducono ad avere metodi lunghi,** perché è il solo modo per raggiungere tali variabili. Infatti, tutti i metodi che dovranno operare con la variabile temp e tutte le operazioni che dovranno essere fatte su di essa, dovranno stare dentro lo stesso metodo, dove è contenuta temp. In caso contrario, dovremmo riscrivere temp anche negli altri metodi, ma questo porta a duplicazione di codice, e quindi **conviene usare un metodo query**. Con la sostituzione effettuata tramite il refactoring, qualsiasi metodo può avere il dato.
2. Spesso si effettua il refactoring di Sostituisci T con Q prima della tecnica **Estrai Metodo.** Refactoring facile se temp è assegnata solo una volta. Capiamo bene, infatti, dopo la spiegazione (1) che tutti i metodi e tutte le operazioni che operavano su temp prima si trovavano in un solo metodo, e quindi è molto probabile per questo motivo che sarà necessario usare la tecnica Estrai Metodo.
   Una variabile temporanea è locale al metodo in cui è definita.
   Per definizione, quindi, tutto il codice che la usa deve stare nello stesso metodo. Non c'è alternativa: non possiamo accedere a una `temp` da un altro metodo.
   Conseguenza diretta: se la `temp` viene usata in molti punti, tutte quelle operazioni si accumulano nello stesso metodo. La `temp` agisce come una colla: tiene tutto il codice correlato inchiodato in un unico blocco.
   Risultato: il metodo cresce, fa troppe cose, e ha bisogno di essere spezzato → Extract Method.

Meccanismi:

1. Cercare una variabile temporanea assegnata solo una volta.

2. Dichiarare temp come **final**. Serve per verificare che la variabile non sia veramente riassegnata da nessun'altra parte. Se è così e il compilatore non dà problemi, si può procedere col refactoring.
3. **Compilare** (per verificare che è assegnata una volta sola).
4. Estrarre la parte destra dell'assegnazione e creare un metodo.
5. Sostituire tutte le occorrenze della variabile con la chiamata al metodo.

Nota importante: se la variabile viene assegnata più volte, e quindi modificata, allora si perde il senso di questa tecnica di refactoring e non è più applicabile. Se la variabile assume valori diversi in momenti diversi, non si può creare un metodo query che restituisca il valore di quella variabile, dato che non ne ha uno "fisso".

Nota: se le performance diventano un problema reale e non immaginato (fare qualche somma in più non è poi così problematico), allora **si ottimizza dopo**.

Alternative possibili:

1. Spaccare la variabile in altre variabili e creare altrettanti metodi query.
2. Lasciare perdere e non applicare questa tecnica di refactoring. Infatti, se la variabile viene riassegnata perché accumula un risultato progressivo (come un contatore o una somma), allora non ha senso estrarne una query. Teniamo la variabile temporanea così com'è e valutiamo se applicare altri refactoring.

Tecnica 3: Dividi Variabile Temp

Una **variabile temporanea** è **assegnata più di una volta**, ma non è una variabile assegnata in un loop o usata per accumulare valori.

Invece di usare sempre la stessa variabile per i vari assegnamenti, conviene usare variabili separate per ciascun assegnamento.

```
double temp = 2*(width+height);
// così abbiamo il perimetro
System.out.println(temp);

temp = height*width;
// così abbiamo l'area
System.out.println(temp);
```

```
// Diventa

final double perim = 2*(height+width);
System.out.println(perim);
final double area = height*width;
System.out.println(area);
```

Motivazioni:

- Le variabili temporanee **temp** hanno vari usi. Alcuni portano ad assegnare temp più volte, come variabili che **cambiano ad ogni passata di un ciclo** (es. per accumulare una somma o per tenere traccia di un contatore). Queste assegnazioni multiple vanno bene.
- Tuttavia, le variabili temp che tengono il risultato di un metodo lungo, per essere usate dopo, dovrebbero essere assegnate una volta soltanto.
- Se sono assegnate più di una volta allora **hanno più di una responsabilità** all'interno del metodo. Ogni variabile dovrebbe avere una sola responsabilità e dovrebbe essere **sostituita con una temp per ciascuna responsabilità.**
- Usare una stessa temp per due cose diverse **confonde il lettore.**

Meccanismi:

1. Bisogna **cambiare il nome** della variabile temp al momento della dichiarazione e alla sua prima assegnazione.
2. Dichiarare la nuova temp come **final.**
3. Cambiare tutti i riferimenti a temp fino alla sua seconda assegnazione.
4. Dichiarare una **nuova temp** per la seconda assegnazione, a cui faranno riferimento tutte le assegnazioni successive che prima venivano fatte sulla temp iniziale.
5. Compilare e **testare.**
6. **Ripetere per singoli passi:** ogni passo rinomina una dichiarazione e cambia i riferimenti fino alla prossima assegnazione.

Tecnica 4: Sposta Metodo

Scenario: un metodo usa più caratteristiche (attributi e operazioni) di un'altra classe che non di quella in cui è definito.

Soluzione: bisogna creare un nuovo metodo (con un corpo di metodo simile) nella classe che il metodo usa maggiormente. Nel corpo del vecchio metodo bisogna invocare il nuovo metodo, oppure rimuovere il vecchio metodo.

Motivazioni:

1. Due classi sono **strettamente accoppiate.**
2. È importante spostare il metodo per rendere le **responsabilità** delle classi **più chiare** e separate.
3. Un metodo usa i metodi di un'altra classe più di quanto usa i metodi della propria classe.

Meccanismi:

1. **Esaminare attributi e metodi** usati dal metodo che deve essere spostato per decidere se bisogna spostare anche questi (ha senso spostarli se sono usati solo dal metodo trasferito).
2. Controllare che il metodo da spostare non sia dichiarato anche in superclassi e sottoclassi.
3. Dichiarare il metodo nella classe target.
4. **Copiare il metodo** nella classe target, e sistemare le chiamate a metodi della classe originaria e della classe target.
5. Richiamare il metodo nella classe target con l'opportuno riferimento. Trasformiamo quindi momentaneamente il metodo della classe originaria in un delegato del metodo della classe target. Una sorta di step intermedio in cui possiamo eseguire i **test** e verificare che tutto stia continuando a funzionare. Quindi ci serve a **garantire che il metodo funzioni ancora** durante la transizione.
6. Una volta che abbiamo verificato nello step precedente che la delega funziona, allora possiamo decidere se **rimuovere il metodo** originario o tenerlo per **delegare** (chiamare al suo interno il nuovo metodo nell'altra classe).

Nota importante: se il metodo è dichiarato anche in superclassi/sottoclassi, allora in base al perchè il metodo è **polimorfico,** si agisce di conseguenza. Se tutto il comportamento polimorfico appartiene concettualmente all'altra classe, allora spostiamo l'intero comportamento nell'altra classe [attenzione: si cade in un'altra tecnica di refactoring più complessa]. Se è importante che il comportamento polimorfico resti nella classe originale, allora bisogna necessariamente lasciare quel metodo nella classe originale, ma potremmo valutare di spostare parte della logica interna nella nuova classe, magari con la tecnica **estrai metodo.**

Tecnica 5: Estrai Classe

Scenario: una classe fa il lavoro che dovrebbero fare due classi.

Esempio: la classe Persona ha attributi per tenere prefisso e numero di telefono.

Soluzione: bisogna creare una nuova classe e si spostano alcuni attributi e metodi dalla vecchia alla nuova classe.

Motivazioni:

- Una classe dovrebbe avere **una responsabilità,** ma durante lo sviluppo si aggiungono attributi e metodi e la classe cresce.
- Una classe **grande** è **difficile da comprendere.**
- Se un **sottoinsieme di dati** ed un **sottoinsieme di metodi** dovrebbero andare insieme, o se un sottoinsieme di dati di solito viene cambiato insieme, o tali **dati dipendono tra loro**, allora è bene **dividere la classe.**
- Un altro segno è che la sottoclasse di tale classe coinvolge solo poche caratteristiche della classe. Con caratteristiche intendiamo sia i metodi che gli attributi. Ricordiamo che si deve poter dire "[nome_sottoclasse] è un [nome_superclasse].

Meccanismi:

1. Decidere quali responsabilità separare.
2. Creare una nuova classe per tali responsabilità.
3. **Inserire una dipendenza** tra la vecchia e la nuova classe: un attributo nella vecchia classe che tiene un'istanza della nuova classe.

4. **Spostare attributi e metodi** dalla vecchia classe alla nuova classe, iniziando dai metodi di più basso livello.

---

13. TestCos'è un test

Un test è un programma che esegue il codice sotto certe condizioni (valori in input) e valuta se il risultato è corretto. Valida **una parte** del sistema. **Un test ha successo quando trova un difetto.** I test documentano le condizioni sotto le quali il codice funziona e proteggono il codice da modifiche indesiderate.

Osservabilità e precondizioni

Osservabilità: per fare test, lo stato dell'oggetto deve essere osservabile tramite metodi opportuni (es. getter). Se dall'interno del test non puoi osservare una variabile intermedia, bisogna modificare il codice per renderlo osservabile. Se un metodo è difficile da testare, probabilmente sta facendo troppe cose → SRP.

Precondizioni (Design by Contract — DbC): sono le condizioni che devono essere necessariamente soddisfatte per eseguire correttamente il codice. Esempio: se voglio accedere all'elemento con indice 5 di un array, prima devo verificare che l'indice sia ≥ 0 e che `array.size() > 5`. Bisogna sempre ragionare sulle precondizioni dei metodi.

Test negativi: bisogna esplicitare tutti i casi, anche quelli "negativi" (es. lista vuota, indice fuori range). Il programmatore deve leggere tutto il codice per capire cosa succede nei casi limite. Struttura consigliata:

java

```
// Prima verificare le precondizioni e ritornare subito
if (lista.isEmpty()) return null;   // caso negativo: esci subito
// poi il codice del metodo per il caso "normale"
```

Nei metodi che richiedono strutture dati, bisogna contemplare il caso del **limite minimo** e del **limite massimo**. L'idea è quella di tentare di non usare le eccezioni: se scrivo codice che gestisce le eccezioni con condizioni esplicite invece di `try/catch`, è generalmente meglio. Le eccezioni delle librerie devono comunque essere gestite: non possono essere lasciate propagare verso l'alto. L'importante è gestirle, non passarle al chiamante.

Oracle

Il **valore atteso** (oracle) è il riferimento per confrontare l'output del test. Come si determina:

- Per sistemi nuovi: ci si basa sui **requisiti** (se sono scritti bene).
- Per refactoring: l'output della versione precedente fa da oracle (il comportamento esterno non deve cambiare).

Struttura di un test (AAA / GWT)

GWT (Given-When-Then)	AAA (Arrange-Act-Assert)
Given — contesto iniziale	**Arrange** — prepara lo stato del sistema
When — l'azione che si testa	**Act** — chiama il metodo
Then — il risultato atteso	**Assert** — valuta il risultato

GWT è più orientato al processo (BDD), AAA è più orientato all'implementazione (codice).

Principi FIRST

- **F — Fast:** veloci.
- **I — Isolated/Independent:** ogni test indipendente dagli altri.
- **R — Repeatable:** stesso risultato su qualsiasi macchina.
- **S — Self-Validating:** dice da solo se è passato o no.
- **T — Timely:** scritti al momento giusto (idealmente prima del codice, vedi TDD).

Qualità dei test

- Almeno una classe di test per ogni classe del sistema, e almeno un test per ogni metodo significativo.
- I messaggi di output devono essere brevi e chiari.
- I test devono essere **semplici**: se un test è complicato e fallisce, non si capisce se il problema è nel test o nel sistema.

- Attenzione ai **float**: usare tolleranza `Math.abs(risultato - atteso) < 0.01`.
- I test di usabilità (interfaccia grafica) non si automatizzano facilmente.

TDD — Test Driven Development

Il ciclo **Red** → **Green** → **Refactor**:

1. **Rosso:** scrivi un test che fallisce. In questa fase si scelgono i nomi di classi e metodi: si sta già facendo progettazione.
2. **Verde:** scrivi il codice minimo per far passare il test.
3. **Refactoring:** migliora la struttura del codice senza cambiarne il comportamento.

Se seguendo TDD tutti i requisiti sono già coperti dai test, non si scrive altro codice (→ YAGNI). Non bisogna arrivare obbligatoriamente al 100% di copertura: con TDD, arrivare al 90-95% è generalmente accettabile.

BDD — Behaviour Driven Development

Variante del TDD orientata al comportamento del sistema, con il linguaggio GWT (Given-When-Then).

---

14. Copertura del codice (Code Coverage)

La **copertura del codice** è il rapporto percentuale tra le linee di codice eseguite durante i test e il totale delle linee. Una copertura del 70% indica che il 70% delle linee è stata eseguita almeno una volta durante i test.

Strumenti: da VSCode con JUnit si ottiene sia il valore di copertura sia il tracciamento riga per riga. Le linee con sfondo **verde** sono già state eseguite almeno una volta; quelle con sfondo **rosso** non sono mai state eseguite.

Il codice è corretto se la copertura è 100%? No. Neanche se i test sono scritti bene. Il codice potrebbe funzionare per gli specifici test scritti, ma non per altri casi. La copertura è una metrica utile, non una garanzia.

Con TDD: non si scrive codice se non si è già scritto il test. Arrivare al 90-95% è generalmente accettabile. Se in un metodo ci sono condizioni che richiedono una

struttura più articolata (es. gestione di limite minimo e massimo), bisogna scrivere test che contemplino entrambi i casi limite.

---

15. JUnit

JUnit è il framework standard per i test unitari in Java. La versione più recente indicata nelle slide del corso è **JUnit 6**. Fornisce un motore per l'esecuzione dei test e API per scriverli.

Annotazioni principali

- `@Test` — identifica un metodo di test.
- `@BeforeEach` — eseguito **prima di ogni** singolo test (per uno stato fresco a ogni test).
- `@BeforeAll` — eseguito **una sola volta** prima di tutti i test della classe. Il metodo deve essere `static`.

Asserzioni (classe `Assertions`)

java

```
assertEquals(expected, actual)   // due valori uguali
assertTrue(condition)             // condizione vera
assertNull(object)                // oggetto null
```

Esempio

java

```
public class AppTest {
    private static final Catalogo catalog = new Catalogo();

    @BeforeAll
    public static void init() {
        catalog.svuota();
        catalog.inserisci("album,12.00,Rick,2025-01-15");
        catalog.inserisci("zaino,21.50,Rick,2025-01-15");
    }

    @Test
    public void sommaValori() {
        catalog.calcolaSomma();
```

```
        assertEquals(33.50f, catalog.getSomma());
    }
}
```

Test parametrizzati

`@ParameterizedTest` con `@CsvSource` esegue lo stesso test con input diversi, evitando duplicazioni:

java

```
@ParameterizedTest
@CsvSource({"1, 1, 2", "2, 3, 5", "-1, 1, 0"})
void testAdd(int a, int b, int expected) {
    assertEquals(expected, calculator.add(a, b));
}
```

---

16. Maven

Apache Maven è uno strumento per la gestione di progetti Java e per automatizzare il build. Standardizza compilazione, gestione delle dipendenze, test, packaging e rilascio.

Struttura standard

```
progetto/
├── pom.xml
└── src/
    ├── main/
    │   └── java/
    │       └── com/example/   ← codice sorgente
    └── test/
        └── java/
            └── com/example/   ← classi di test
```

File POM

Descrive il progetto: nome, versione, dipendenze, plugin. Per aggiungere una libreria: cercarla su **Maven Central**, copiare il blocco `<dependency>` nel `pom.xml`. Maven scarica automaticamente la libreria e le sue dipendenze. Il `scope: test` indica che la libreria è usata solo per i test.

Comandi principali (ogni fase esegue anche quelle precedenti)

bash

```
mvn compile    # compila il codice sorgente
mvn test       # esegue i test unitari
mvn package    # produce un .jar distribuibile
mvn verify     # esegue i controlli di qualità
```

Un **.jar** è un archivio Java: un file compresso che contiene le classi compilate e le risorse del progetto.

Design PatternDefinizioni e caratteristiche

- I **Design Pattern** sono strutture software (ovvero micro-architetture) per un piccolo numero di classi che descrivono **soluzioni di successo per problemi ricorrenti.** Tali micro-architetture specificano le diverse classi ed oggetti coinvolti e le loro interazioni.
- Si mira a riusare un insieme di classi, ovvero la soluzione ad un certo problema ricorrente, che spesso è costituita da più di una classe.
- Durante la progettazione, prendiamo diverse scelte sulle classi, le cui conseguenze potrebbero non essere subito note, e le classi potrebbero diventare difficili da riusare o non esibire alcune proprietà (**riusabilità, estensibilità, sostituibilità).** Con sostituibilità intendiamo sostituire la classe in questione con un'altra classe compatibile senza rompere il codice che la usa. Per esempio, sostituire una LinkedList con un ArrayList.
- Esistono tanti cataloghi di Design Pattern, per vari contesti: sistemi centralizzati, concorrenti, distribuiti, real-time, etc.
- Un Design Pattern descrive un problema di progettazione ricorrente che si incontra in specifici contesti e presenta una **soluzione collaudata generica ma specializzabile.** Documentano quindi soluzioni già applicate che si sono rivelate di successo per certi problemi e che si sono evolute nel tempo.
- **Aiutano i principianti** ad agire come se fossero esperti.
- **Supportano gli esperti** nella progettazione su grande scala.
- Evitano di re-inventare concetti e soluzioni, riducendo il costo.

- Forniscono un **vocabolario comune** e permettono una comprensione dei principi di design.
- Non si occupano solo di cosa fa il codice (**requisiti funzionali**), ma analizzano le loro **proprietà non-funzionali** (come è realizzato): ovvero, come una funzionalità è realizzata, es. affidabilità, modificabilità, sicurezza, testabilità, riuso.

Descrizione di un Design Pattern

Un Design Pattern nomina, astrae ed identifica gli aspetti chiave di un **problema di progettazione**, le classi e le istanze che vi partecipano, i loro ruoli e come collaborano, ovvero la distribuzione delle responsabilità.

La descrizione include cinque parti fondamentali:

1. **Nome:** permette di identificare il Design Pattern con una parola e di lavorare con un **alto livello di astrazione**, e indicando lo scopo del pattern.
2. **Intento:** descrive brevemente le **funzionalità** e lo **scopo**.
3. **Problema (Motivazione + Applicabilità):** descrive il problema a cui il pattern è applicato e le condizioni necessarie per applicarlo.
4. **Soluzione:** descrive gli elementi (classi) che costituiscono il design pattern, le loro responsabilità e le loro relazioni.
5. **Conseguenze:** indicano risultati, compromessi, vantaggi e svantaggi nell'uso del Design Pattern.

Nota: nella sezione Problema si parla di **forze** (come in fisica). Ovvero, le forze sono **obiettivi e vincoli,** spesso contrastanti, che si incontrano nel contesto del Design Pattern.

Altre parti della descrizione di un Design Pattern possono essere:

- **Esempi di utilizzo:** illustrano dove il design pattern è stato usato.
- **Codice:** fornisce porzioni di codice che lo implementano.

Organizzazione

I design pattern sono organizzati sul catalogo (libro **GoF**) in base allo scopo. Ne esistono di diversi tipi:

- **Creazionali:** riguardano la creazione di **istanze**. Fanno parte di questa categoria design pattern come il Singleton, il Factory Method, l'Abstract Factory, Builder.
- **Strutturali:** riguardano la scelta della **struttura**. Fanno parte di questa categoria design pattern come l'Adapter, il Facade, il Composite, il Decorator, il Bridge.
- **Comportamentali:** riguardano la scelta dell'**incapsulamento di algoritmi**. Fanno parte di questa categoria design pattern come l'Iterator, il Template Method, l'Observer, lo State.

Design Pattern Creazionali

I design pattern creazionali presentano le seguenti caratteristiche:

- Permettono di **astrarre il processo di creazione oggetti**: rendono un sistema indipendente da come i suoi oggetti sono creati, composti e rappresentati.
- Sono importanti se i sistemi evolvono per dipendere più su **composizioni di oggetti** che su ereditarietà tra classi. L'ereditarietà è rigida e la gerarchia è fissa a compile time: la composizione, infatti, è più flessibile, dato che possiamo combinare oggetti diversi a runtime.
- L'enfasi va dal codificare un insieme fissato di comportamenti verso un più **piccolo insieme di comportamenti fondamentali componibili.** Invece di avere tante classi ognuna con un comportamento specifico fisso, bisogna avere **poche classi con comportamenti base che possiamo combinare fra loro.** Meno classi e più flessibilità.
- Incapsulano conoscenza sulle classi concrete che un sistema usa. **Il codice client non sa quali classi concrete vengono istanziate.** Questa conoscenza è nascosta dentro il pattern. Ad esempio, con un metodo factory il client chiama `createDocument()` senza sapere se ottiene un PdfDocument o un WordDocument. **Sposta la responsabilità di sapere quale classe concreta usare** dal client al pattern.

- Nascondono come le istanze delle classi sono create e composte. Il codice client non sa come l'oggetto è costruito internamente: se è un singleton, se viene creato da zero o recuperato da un pool. Vede solo l'oggetto finito e pronto da usare.

DP: Factory Method

Intento: definire una interfaccia per creare un oggetto, ma lasciare che le sottoclassi decidano quale classe istanziare. Factory Method permette ad una classe di rimandare l'istanziazione alle sottoclassi.

Problema:

- Un framework usa classi astratte per definire e mantenere relazioni tra oggetti. Il framework deve creare oggetti ma conosce solo classi astratte che non può istanziare. Un metodo responsabile per l'istanziazione (detto **factory**, ovvero fabbricatore) incapsula la conoscenza su quale classe creare. Quindi il framework lavora con classi astratte perché deve essere **generico e riusabile**, ma a un certo punto dovrà creare oggetti concreti, e non può farlo da solo perché non sa quali classi concrete esistono.
- Il problema quindi è che non può istanziare Document perché è astratto, e non può istanziare un oggetto specifico perché sarebbe vincolante. Quindi ha un'idea: rimanda l'istanziazione alle sottoclassi, e così poi, tramite il **dispatch a runtime,** verrà chiamata la versione corretta di `createDocument`.
- Ci sono due **esigenze che si scontrano:**
  1. Essere **generico:** ovvero il framework vuole funzionare con qualsiasi tipo di oggetto (sottoclassi), e per questo lavora con la superclasse generica e astratta, e non si lega a nessun tipo specifico.
  2. **Creare oggetti:** vuole creare un oggetto reale da poter usare.
- La soluzione è quella di delegare la creazione dell'oggetto alla sottoclasse. Lei si occupa di chiamare il **factoryMethod** per creare l'oggetto specifico corretto.
- **Nota:** con framework in questo contesto intendiamo una **struttura di classi astratte** che definisce uno **scheletro di applicazione**, lasciando alcuni dettagli da riempire a chi usa il framework.

Soluzione:

- **Product:** è l'interfaccia comune degli oggetti creati da factoryMethod().
- **ConcreteProduct:** è un'implementazione di Product.
- **Creator:** dichiara il factoryMethod(), e quest'ultimo ritorna un oggetto di tipo Product. Creator può avere un'**implementazione di default** di factoryMethod() che ritorna un certo ConcreteProduct. In questo caso, in UML ci sarebbe una freccia da Creator verso un ConcreteProduct specifico.
- **ConcreteCreator:** implementa il factoryMethod(), o ne fa override, e sceglie quale ConcreteProduct istanziare e ritorna tale istanza. Nell'esempio di sotto, il ConcreteCreator conosce entrambi i ConcreteProduct, e può istanziare quello che gli serve a seconda della logica interna.
  Tipicamente, esiste un ConcreteCreator per un singolo ConcreteProduct, quindi si parla di corrispondenza **1-a-1** (approccio più semplice e leggibile).
  Ogni ConcreteCreator esiste solo per creare un tipo specifico di ConcreteProduct, seguendo il principio **SRP**.

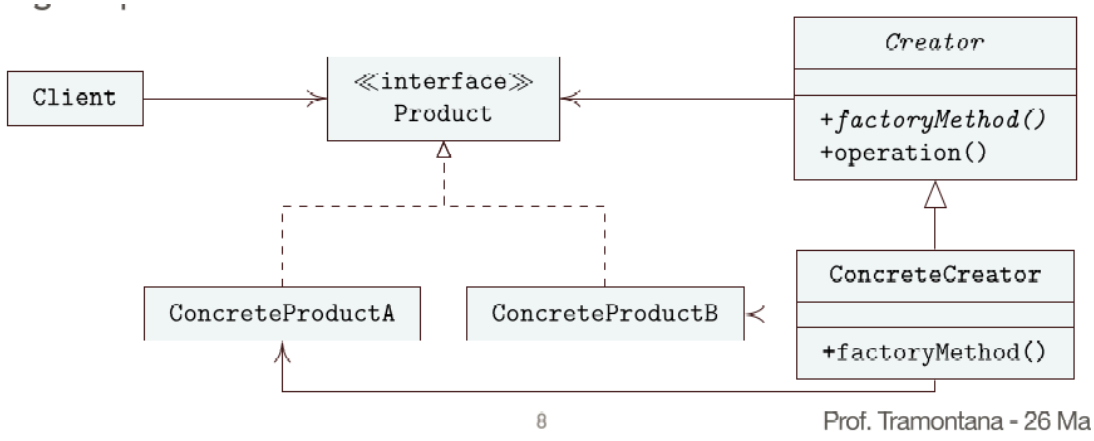


Diagramma UML delle classi

Diagramma UML di sequenza:

- `operation()`: è un metodo del Creator. Il Client chiama operation() su un riferimento che per lui è `Creator`, ma che punta a un oggetto il cui tipo dinamico è `ConcreteCreator`. Operation al suo interno ha bisogno di creare un oggetto e quindi chiama `factoryMethod()`. Operation è codice definito in Creator,

ma a runtime sta eseguendo sull'istanza `cc` (ConcreteCreator). Viene chiamato il metodo `factoryMethod()` sulla stessa istanza, e per **late-binding** entra in esecuzione la versione overriden di ConcreteCreator (e non quella di Creator). La freccia con la punta nera piena rappresenta un **messaggio sincrono**, cioè una **chiamata di metodo**. Il metodo scritto sulla freccia appartiene all'oggetto verso cui punta, ed è invocato dall'oggetto da cui parte. Poi c'è anche la **freccia riflessiva**, come quella di `factoryMethod()`, dato che il concreteCreator chiama un proprio metodo su sé stesso.

- `create`: viene creato il ConcreteProductA, specificato dal ConcreteClient. Rappresenta il **messaggio che istanzia l'oggetto**, in particolare l'**invocazione del costruttore** (sicuramente grazie al metodo chiamato poco prima, in questo caso grazie a factoryMethod).
- Successivamente, `cp` viene ritornato da factoryMethod e da operation. La freccia tratteggiata con la punta aperta rappresenta il ritorno (non è una chiamata di metodo), e l'etichetta sulla freccia rappresenta il valore restituito. In questo caso, il valore restituito è la variabile di nome `cp` , ovvero ConcreteProductA, che viene restituita al Client.
- Successivamente, dato che il Client possiede un **riferimento a Product** che punta dinamicamente a un oggetto `ConcreteProductA`, può utilizzare i metodi di questo oggetto, e in questo caso chiama il metodo `request()`.

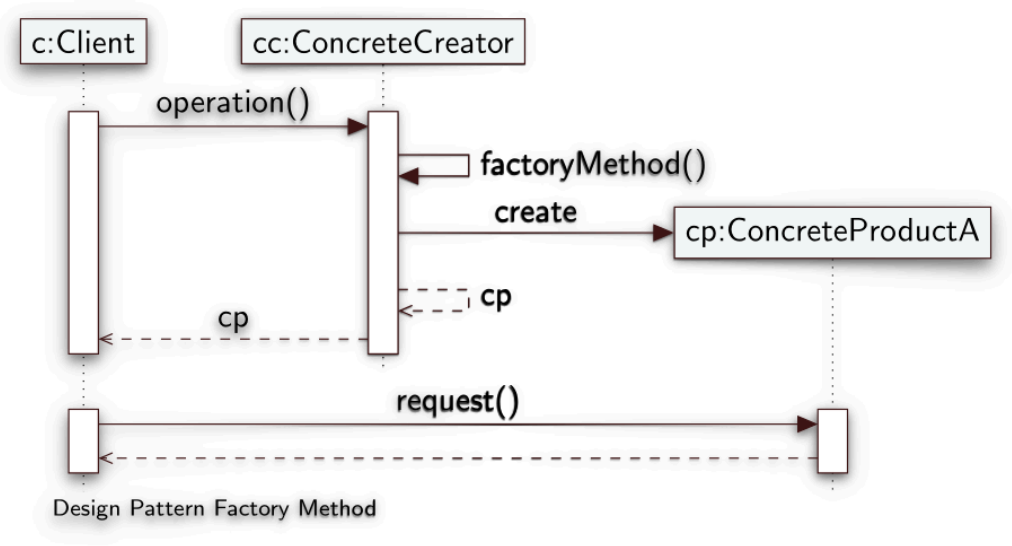


Diagramma UML di sequenza

Differenza tra UML di sequenza e UML delle classi

Il diagramma delle classi e il diagramma di sequenza non devono necessariamente combaciare freccia per freccia, perché rispondono a due domande diverse:

1. Il **diagramma delle classi** dei pattern catalogati è quasi sempre **idealizzato.** Disegna solo le dipendenze che costituiscono il cuore concettuale del pattern, e omette o semplifica il resto.
2. Il **diagramma di sequenza** mostra la **collaborazione a runtime completa**, e quindi è costretto a includere ogni messaggio reale, comprese le varie chiamate “di servizio”.

Quindi, la dipendenza `Client -> Creator` presente nel diagramma di sequenza esiste davvero. Se il Client invoca `operation()` su un Creator, allora ci sono pochi dubbi da avere: il Client dipende dal Creator, punto. Un diagramma delle classi **rigoroso** dovrebbe disegnare quella freccia. Una nota importante da considerare è che nel diagramma del Gamma questo box Client non è presente, e questa è un’aggiunta puramente didattica.

Digressioni su Product e Creator

- **Product:** è un'interfaccia perché definisce solo **cosa** un prodotto deve saper fare. I prodotti concreti **non condividono nulla**. Non c'è codice né stato comune da fattorizzato in una superclasse. Un **contratto puro** è la scelta giusta. Inoltre, un ConcreteProduct potrebbe implementare Product e altre interfaccia, oppure estendere già qualche altra classe. In questo modo l'interfaccia non consuma l'unica ereditarietà disponibile. Bisogna seguire il principio di "programmare per le interfacce".
- **Creator:** è una classe astratta e non un'interfaccia perché ha bisogno di tre cose che un'interfaccia non potrebbe dargli:
  1. **Stato d'istanza**: il creatore conta quante istanze ha prodotto. Le interfacce non possono avere campi d'istanza mutabili.
  2. **Metodo concreto** `final` **con un corpo:** capita spesso, così come anche nel nostro esempio di Studente, che il Creatore abbia un metodo di creazione che non deve essere modificato. Nel nostro caso, `creaStudente()` si occupa di incrementare il contatore delle istanze e di chiamare il costruttore per lo Studente. Le interfacce non possono avere metodi final.
  3. **Combinazione** di membri concreti (`creaStudente`, `getNum`) e membri astratti (`getStudente`, `sospensi`, `ripristina`). Questo "scheletro parziale con buchi da riempire" è esattamente ciò per cui esistono le classi astratte.

Bisogna quindi capire che l'interfaccia definisce "**cosa sai fare**", mentre la classe astratta "**cosa sei in parte**". Nell'esempio precedente, Studente potrebbe essere una classe astratta e funzionerebbe, ma renderebbe la struttura più rigida senza reali vantaggi. Creator, invece, non può essere un'interfaccia, perché ha bisogno di rendere **final** il metodo `creaStudente()` e di tenere il contatore delle istanze.

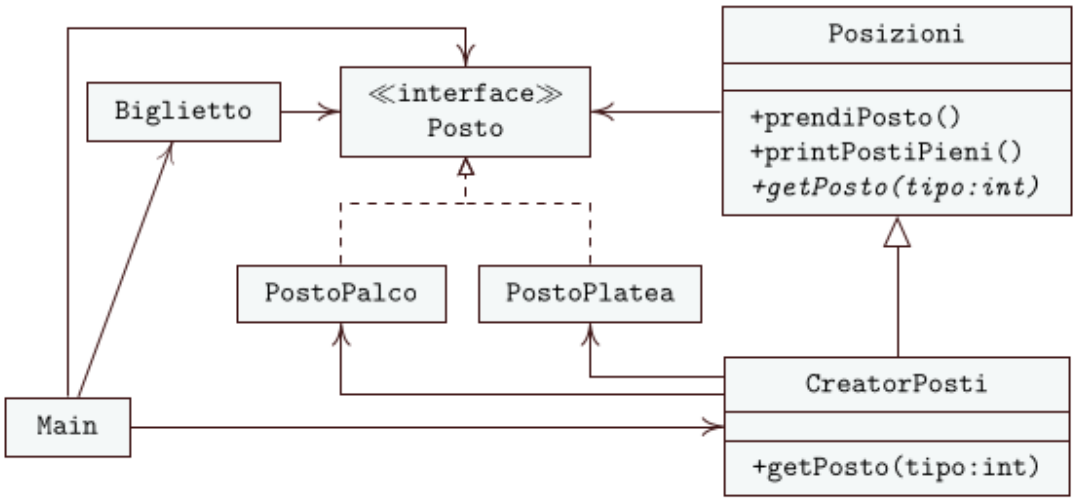
Conseguenze

- Il codice delle classi dell'applicazione conosce solo l'interfaccia Product e può lavorare con qualsiasi ConcreteProduct. I ConcreteProduct sono **facilmente intercambiabili.** Rappresenta il **beneficio principale.**
- Se si implementa una sottoclasse di Creator per ciascun ConcreteProduct da istanziare si ha una **proliferazione di classi.** Questo rappresenta uno svantaggio di questo design pattern, il costo che si paga per avere **disaccoppiamento, estensibilità Open/Closed** (poter aggiungere un nuovo

prodotto/una nuova coppia creator-prodotto è molto semplice e si può fare senza dover modificare il codice esistente) e **classi coese senza condizionali** (ogni ConcreteCreator fa una cosa sola, senza avere un'unica factory monolitica con un grande blocco if/switch). Il costo quindi è rappresentato da un maggior numero di classe. Quindi potremmo ritrovarci con tanti ConcreteCreator quanti sono i ConcreteProduct.

Esempio Concreto di Factory Method

Abbiamo visto poco prima che nel diagramma UML "generico" e idealizzato, veniva messo in luce solo il disaccoppiamento essenziale (Client → Product), e lasciava implicita la dipendenza Client → Creator. Nell'esempio seguente, questa dipendenza non viene lasciata implicita e offre interessanti spunti di riflessione.



È interessante notare che **Main** (ovvero il Client) dipende da:

1. **Biglietto:** il Main istanzia e manipola oggetti Biglietto. L'applicazione emette un biglietto per ciascun posto assegnato. Quindi il Main deve saper creare biglietti, ed ecco perché c'è la dipendenza `Main -> Biglietto`. Biglietto, a sua volta, dipende da Posto, e per questo c'è la dipendenza `Biglietto -> Posto`. Un biglietto è per un posto, e lo tiene come Posto, tramite astrazione. Quindi Main crea un Biglietto e gli passa il Posto ottenuto.

2. **CreatorPosti:** il Main dipende da CreatorPosti (ovvero il ConcreteCreator) perché **delega a lui la creazione**: chiede un posto, e il creatore decide e costruisce il tipo concreto giusto.
3. **Posto:** il Main lavora sui posti che ottiene, e li tratta attraverso l'interfaccia Posto. I metodi del creatore restituiscono un Posto, e Main lo usa come tale (lo passa al Biglietto, legge lo Stato, etc). Questo è il disaccoppiamento lato prodotto: Main non nomina mai PostoPalco né PostoPlatea, conosce solo il contratto Posto.

In questa variante, inoltre, il Main **dipende dal ConcreteCreator,** quindi è la variante meno disaccoppiata. Se volessimo "Programmare per le interfacce" dovremmo tipizzare la variabile come Posizioni (**tipo statico**) pur tenendo a runtime un oggetto CreatorPosti (**tipo dinamico**). In questo modo otteniamo il vantaggio di star programmando per le astrazioni, ma non abbiamo disaccoppiamento pieno perché comunque abbiamo la dipendenza da CreatorPosti.

Come fa il Client a specificare il tipo di Posto? Per farlo, il Client chiama il metodo prendiPosto() di Posizioni (Creator) passando come parametro un valore. Per esempio, il valore 1 rappresenterà il PostoPalco, e tutti gli altri valori possibili il PostoPlatea. In questo modo il Client non conosce in modo diretto né PostoPalco né PostoPlatea, ma solo una convenzione che serve per "mapparli" a dei numeri interi. In questo modo, se poi dovesse cambiare l'implementazione interna di un ConcreteProduct, il Client non ne risentirebbe, dato che lui si limita a chiamare la funzione prendiPosto di Posizioni specificando il tipo di Posto attraverso un semplice numero intero.

Object Pool

Descrizione e caratteristiche di un Object Pool:

- Un **Object Pool** è un deposito di istanze già create. Un'istanza sarà estratta dal pool quando una classe client ne fa richiesta.
- Il pool può **crescere** o può avere **dimensioni fisse**. Se è a dimensioni fisse e non ci sono oggetti disponibili al momento della richiesta, allora non creo di nuovi.
- Il client restituisce al pool l'istanza usata quando non è più utile.

Il Design Pattern **Factory Method** può implementare un Object Pool:

1. I client richiedono istanze, come visto per il Factory Method. Il punto interessante è proprio questo: in ogni implementazione concreta del Design Pattern Factory Method, il **Client non crea mai nessun prodotto**, ma chiede semplicemente al ConcreteCreator di fornirgliene uno, ma il Client non sa se questo verrà creato oppure no. Nel caso dell'object pool, solitamente non viene creato, ma viene fornito al client un oggetto già esistente nel pool: **dipende dallo stato del pool** in quel momento. Significa che l'interfaccia per ottenere oggetti non cambia, e lato client non vi è alcun tipo di modifica. Il pool **riusa quando può e crea quando deve.**
2. I client dovranno indicare quando l'istanza non è più in uso, quindi riusabile. Questa, lato client, è un'aggiunta interessante che si concretizza in una **nuova responsabilità** per il client, che dovrà occuparsi di fare un passo esplicito di **rilascio**.
3. Lo stato dell'istanza da riusare potrebbe dover essere riscritto. Questo significa che prima di consegnarla a un nuovo client, il pool deve **resettarne lo stato**, altrimenti l'utente vedrebbe dati vecchi. Un oggetto se si trova nel pool allora è libero, se è usato allora non si trova nel pool. Sono queste le uniche due opzioni possibili: **o è libero e quindi è nel pool o è in uso e quindi non è nel pool.**
4. L'object pool dovrebbe essere unico: potrei usare un **Singleton.**

Dependency Injection

- Il Design Pattern Factory Method può essere usato per inserire le dipendenze (**dependency injection**) necessarie alle istanze di ConcreteProduct.
- Tramite la Dependency Injection **un oggetto (client) riceve altri oggetti da cui dipende**, questi altri oggetti sono detti **dipendenze** (del client in questo caso).
- La tecnica di Dependency Injection permette di **separare la costruzione delle istanze dal loro uso**, e in questo modo **il client non crea l'istanza** di cui ha bisogno.
- Le dipendenze vengono **iniettate al client** per mezzo di parametri nel suo costruttore. Questo permette di **evitare complicazioni derivanti da metodi**

setter e da controlli per verificare che le dipendenze non siano null, di conseguenza il codice è più semplice.

- L'oggetto che fa Dependency Injection si occupa di **connettere** (fa **wiring** di) varie istanze. In un unico posto vediamo le connessioni fra gli oggetti.

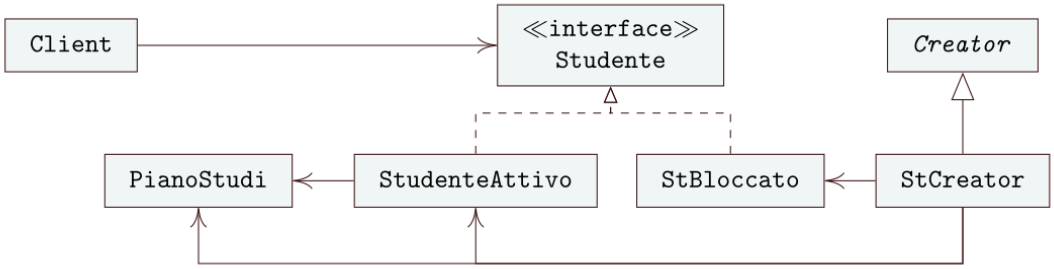
Quindi, con la dependency injection l'oggetto **non crea le sue dipendenze**, ma **le riceve da fuori**. C'è una **Constructor Injection:** la dipendenza arriva al momento della creazione. Può essere `final`, è garantita **non null** e l'oggetto è valido fin dal primo istante. Con la **Setter Injection**, invece, l'**oggetto nasce incomleto** (dipendenza null), finché qualcuno non chiama il setter. Ci sono diversi problemi in quest'ultimo approccio, come dimenticarsi di chiamare il setter e che l'oggetto è mutabile e non final.

Inoltre, dato che nessuno crea le proprie dipendenze, qualcuno dovrà pur crearle e collegarle. A questo ci pensa l'**injector** (anche noto come **composition root**).

Legame col Factory Method: dato che il factory method è l'unico punto dove il ConcreteProduct viene istanziato, è il posto naturale per iniettargli quelle dipendenze. Il factory in questo modo fa un doppio lavoro: decide quale classe concreta creare e la consegna già con il **wiring** fatto con le sue dipendenze. Il client chiama `factoryMethod()` e riceve un prodotto pronto all'uso, senza conoscere né la classe concreta né le sue dipendenze. In questo caso il client che riceve le dipendenze è `ConcreteProduct`, e l'injector è il factory method stesso.
Quindi, con la Dependency Injection, il client riceve il Creator già pronto (dato che si trova nel costruttore). Infatti, chi si occuperà di creare il Client dovrà passargli nel costruttore l'istanza del ConcreteCreator scelto. In questo modo le dipendenze (ovvero il Creator, che ha come tipo dinamico un determinato ConcreteCreator) vengono iniettate all'interno di Client. Ricordiamo infatti che in questo caso il Client conosce solamente il tipo statico Creator, ma poi dinamicamente questo si rivela essere un ConcreteCreator.

Esempio di Dependency Injection:

In questo abbiamo abbiamo che PianoStudi è una dipendenza di StudenteAttivo. Quindi nel costruttore di StudenteAttivo è presente PianoStudi, ma questa dipendenza verrà iniettata da chi si occupa di creare StudenteAttivo, ovvero StCreator, ovvero il factoryMethod del ConcreteCreator. StCreator costruirà StudenteAttivo e gli inietterà all'intero PianoStudi.



Dependency Injection

DP: Abstract Factory

Intento: fornire un'interfaccia per **creare famiglie di oggetti** che hanno qualche relazione senza specificare le loro classi concrete.

Problema:

- Il sistema complessivo dovrebbe essere **indipendente dalle classi usate,** così da essere configurabile con una di varie famiglie di classi. Le classi di una famiglia dovrebbero essere usate in modo consistente.
- Per esempio, lo strato di interfaccia utente permette vari tipi di look-and-feel (ovvero le varie **identità visive e interattive complessive**), così differenti comportamenti per accessori (widget, riquadri interattivi) dell'interfaccia utente sono possibili.
- Il sistema deve poter essere configurato con **una di più famiglie** possibili, e una volta scelta una famiglia, tutti i suoi pezzi devono provenire da quella famiglia. Quindi servono due cose insieme:
  1. Poter cambiare l'intera famiglia in un colpo solo (cambio **look-and-feel**).
  2. Garantire che non si mescolino pezzi di famiglie diverse (**coerenza**).

Relazione col Factory Method:

- Il Factory Method crea un prodotto, mentre l'Abstract Factory crea un'intera **famiglia di prodotti correlati**, che devono essere usati insieme in modo coerente, senza che il client conosca le classi concrete. Quindi l'Abstract Factory crea **più tipi** di prodotto correlati (e quindi un metodo per ciascun tipo).

- Mentre il Factory Method mi disaccoppia dalla classe concreta di un prodotto, l'Abstract Factory mi disaccoppia da un'intera famiglia di prodotti e **in più garantisce che siano usati in modo coerente**.

Soluzione:

- **AbstractFactory:** interfaccia astratta per creare famiglie di oggetti. Dichiara un metodo di fabbricazione per ciascun tipo di prodotto della famiglia.
- **ConcreteFactory:** classi che implementano operazioni per creare ciascuna famiglia di oggetti. Implementano l'interfaccia AbstractFactory, ciascuna producendo esclusivamente i prodotti della propria famiglia. Il client riceve un riferimento polimorfico a una ConcreteFactory e ne invoca i metodi di creazione senza mai sapere quale famiglia stia usando.
- **Interfacce AbstractProduct:** è l'interfaccia per una famiglia di oggetti. È una per ogni tipo di prodotto, e nascondono le implementazioni concrete. Quindi, per ogni tipo di Prodotto c'è un AbstractProduct.
- **Product:** definisce un oggetto, creato da ConcreteFactory, che implementa l'interfaccia AbstractProduct.
- **Client:** usa solo interfacce dichiarate da AbstractFactory e AbstractProduct.
- In questo modo è come se visualizzassimo i prodotti in una griglia bidimensionale, dove le righe sono i **tipi** di prodotto, e le colonne sono le **famiglie**.

Conseguenze:

- Permette di usare classi in modo consistente (per famiglie).
- Le famiglie di classi sono facilmente intercambiabili.
- Non è immediato supportare nuove classi Product, poiché bisogna aggiungere un metodo su AbstractFactory e su ogni ConcreteFactory.

Spiegazione complessiva:

- Il problema che risolve è tipico del design di sistemi software ad oggetti: un client che si occupa di istanziare correttamente le classi con cui lavora. Il problema sorge quando il client deve poter scegliere fra più famiglie di oggetti. L'accoppiamento fra il client è una singola famiglia di prodotti diventa un

debito tecnico architetturale significativo. A questo punto, **non può legarsi a ogni famiglia di oggetti**, sarebbe impensabile. Bisogna disaccoppiare il client e le famiglie di oggetti, e rendere facile e veloce per il client lo switch da una famiglia di oggetti all'altra. È questo il problema che questo Design Pattern risolve.

- Una **famiglia di oggetti** è un insieme di **oggetti distinti** per tipo ma correlati tra loro per appartenenza a una variante comune del sistema. I prodotti di famiglie diverse non devono essere mescolati, pena l'**incoerenza comportamentale o strutturale** del sistema.
- Un esempio tipico per l'applicazione di questo Design Pattern: sviluppare **toolkit grafico multi-piattaforma**. Il sistema deve produrre widget di interfaccia (Button, Checkbox, Scrollbar) in due varianti: Windows e macOS. Ogni variante forma una famiglia, infatti avremo i bottoni, le checkbox e le scrollbar in stile windows e gli stessi oggetti in stile macOS. Il client non può permettersi di scrivere `new WindowsButton()`, in quanto presente tre difetti gravi:
  1. **Accoppiamento alle classi concrete:** il client dipende direttamente dalla classe implementativa. Qualsiasi sostituzione richiede una modifica al codice client, violando il principio **Open/Closed Principle.** È uno dei principi **SOLID** dell'ingegneria del sw. Un'entità software, come una classe, un modulo o una funzione, deve essere **aperta all'estensione** (ovvero è possibile aggiungere nuovi comportamenti) e **chiusa alla modifica** (il codice sorgente esistente non deve essere alterato per accomodare tali estensioni). Ovviamente la chiusura rispetto alla modifica si riferisce alle classi astratte e alle interfacce, mentre le classi concrete possono naturalmente essere modificate per correggere bug o migliorare l'implementazione interna, purché rispettino il contratto.
  2. **Scarsa estendibilità**: aggiungere una terza famiglia come Linux richiede di rintracciare e modificare ogni punto del codice in cui viene effettuata un'istanziazione concreta.
  3. **Assenza di garanzie e coerenza:** nulla impedisce al client di creare un WindowsButton insieme a un MacScrollBar, ottenendo una combinazione incoerente che nessuna logica a runtime può facilmente rilevare.

Il problema quindi si generalizza così: come fare in modo che un sistema possa lavorare con famiglie intercambiabili di oggetti correlati, rimanendo completamente ignaro delle classi concrete che li implementano? La soluzione è quella proposta dal pattern.

Spiegazione del diagramma UML delle classi

- È importante notare che in AbstractProduct la lettera indica il tipo di oggetto (es. Button), mentre il numero che poi viene usato nei Product serve per distinguere le famiglie.
- ConcreteFactory non dipende da AbstractProduct nel diagramma UML in quanto già l'AbstractFactory dipende da questi, e **la dipendenza è transitiva**. Quindi nei suoi metodi è già vincolata per contratto a restituire qualcosa che sia compatibile con AbstractProductA e AbstractProductB. Aggiungere quindi una freccia diretta da ConcreteFactory verso AbstractProductA e AbstractProductB sarebbe un'informazione ridondante, e quindi rumore visivo.
- Il Client, invece, ha comunque una freccia verso AbstractProductA e AbstractProductB, perché non si parla di dipendenza transitiva ma di **dipendenza diretta**. Il client, infatti, chiama dei metodi sopra AbstractProductA e AbstractProductB, quindi ha bisogno di conoscerli. Non basta quindi la dipendenza transitiva che deriva da AbstractFactory, ma ne serve una diretta.
- Il client, in una situazione ideale, **non conosce ConcreteFactory**, in quanto sono dipendenze che gli vengono iniettate dall'esterno (tramite la dependency injection). Infatti, solitamente dovrebbe avere un tipo AbstractFactory nel costruttore, che poi a runtime punta a un oggetto ConcreteFactory specifico.

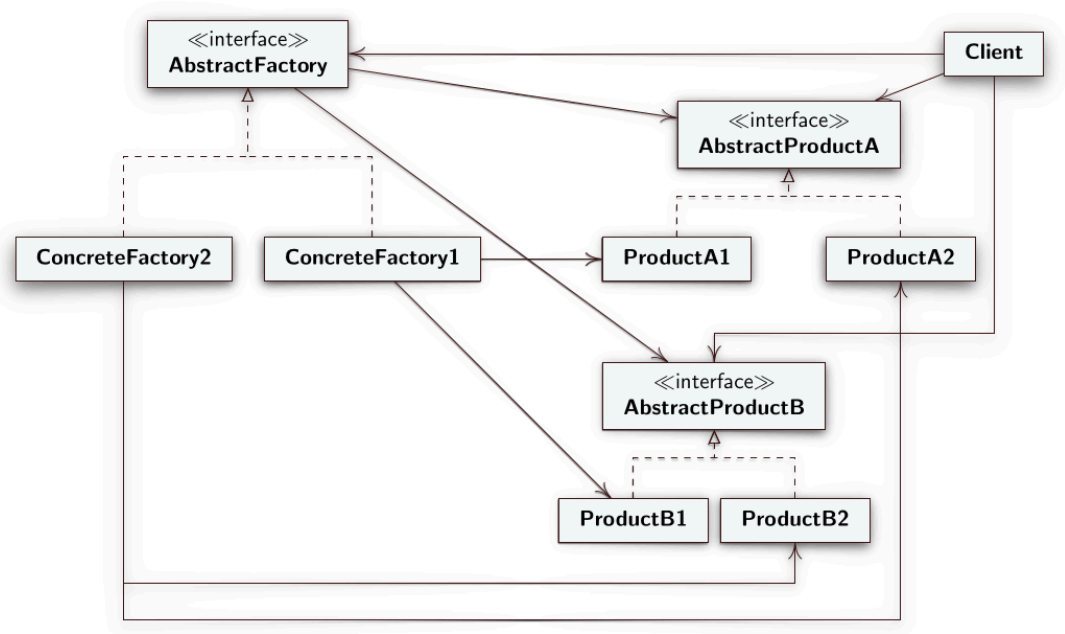


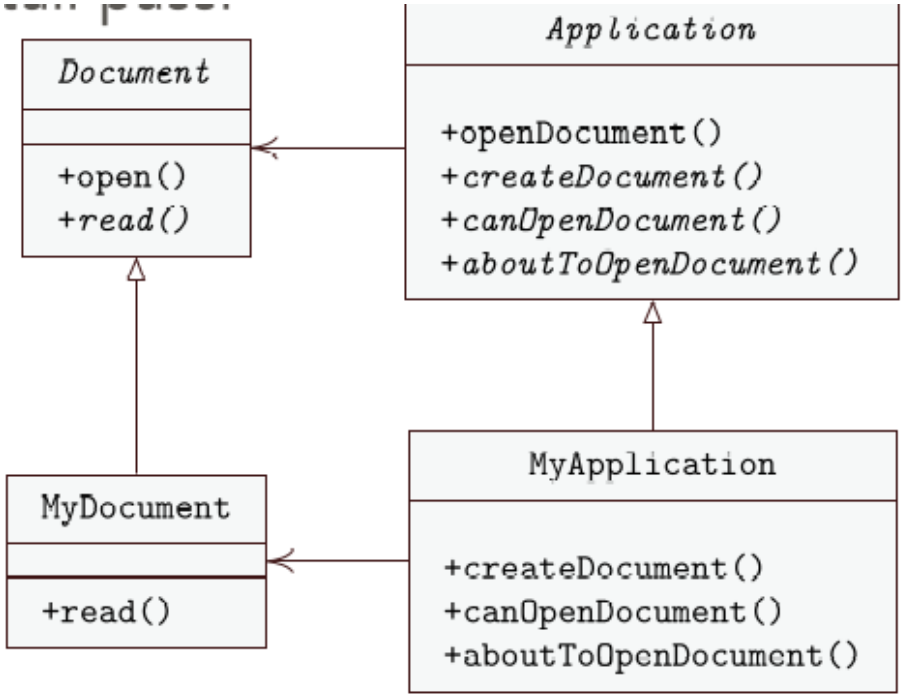
Diagramma UML di Abstract Factory

DP: Template Method

Intento: definisce lo scheletro di un algoritmo in un'operazione e rinvia la definizione di alcuni passi nelle sottoclassi. Il Template Method lascia che le sottoclassi ridefiniscano alcuni passi di un algoritmo senza cambiare la struttura dell'algoritmo.

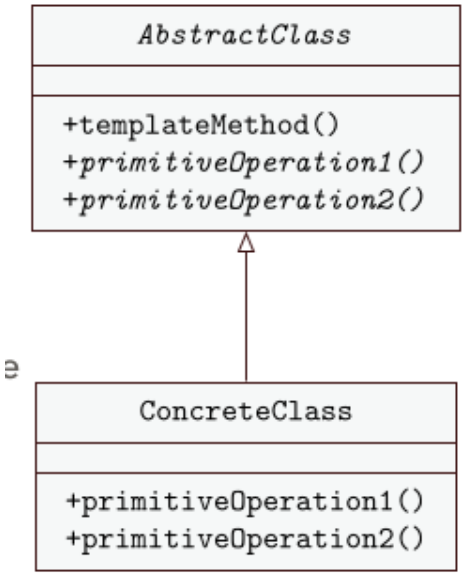
Problema (Motivazioni):

- Alcune volte si ha bisogno di classi che definiscono i passi necessari per un algoritmo, ma non i dettagli di tali passi.
- Per esempio, supponiamo di avere una classe Application che definisce l'algoritmo per aprire e leggere un documento in `openDocument()`, ovvero il **template method.** Tale algoritmo è definito tramite operazioni astratte che le sottoclassi implementano.
- **Template method:** metodo della superclasse che contiene lo **scheletro fisso** dell'algoritmo.



Soluzione:

- **AbstractClass** definisce operazioni primitive (come metodi astratti) che devono essere implementate dalle sottoclassi per definire i passi dell'algoritmo. Implementa un template method che definisce lo scheletro dell'algoritmo, che chiama le operazioni astratte.
- **ConcreteClass** implementa le operazioni primitive per svolgere i passi specifici.



Conseguenze:

- I metodi template sono una tecnica fondamentale per riusare il codice. Sono particolarmente importanti nelle classi di libreria poiché fanno sì che si possa esprimere il comportameno comune nelle classi di libreria.
- I metodi template portano ad avere una **struttura di controllo invertita**, che **permette a una superclasse di chiamare le operazioni di una sottoclasse**, e non viceversa. **Principio Hollywood:** “non chiamarci, noi ti chiameremo”. Questo perché sarà la superclasse a chiamare la sottoclasse, e non viceversa.
- I metodi template chiamano:
  1. **Operazioni concrete su ConcreteClass:** chiama metodi che hanno già un'implementazione nella superclasse ma chiama la versione overridata della sottoclasse. **Classi client:** chiama metodi accessori che possono aiutare l'algoritmo, e sono metodi di un’altra classe.
  2. **Operazioni concrete su AbstractClass:** ovvero metodi che hanno già un'implementazione dentro AbstractClass ma che non vengono overridati, magari ausiliari per l'algoritmo.
  3. **Operazioni primitive:** proprio le operazioni che ci interessano di più, in quanto non ne viene fornita un'implementazione e spetta alla sottoclasse scriverla. **Metodi Factory:** rientra nelle operazioni primitive di cui abbiamo appena parlato, solo che si occupa di creare oggetti. È primitivo perché il

suo scopo è creare un oggetto, e il tipo di oggetto dipende dalla sottoclasse specifica, quindi non può saperlo già nella superclasse. Ma niente impedisce che sia un hook se esiste un tipo di default ragionevole, ma solitamente è primitivo. Inoltre, dato che il metodo factory ha un tipo di ritorno, si potrebbe pensare di implementarlo e creare quell'oggetto, ma questa osservazione ha due problemi: il primo è che non è detto che quell'oggetto sia istanziabile, magari è un'interfaccia o una classe astratta. Il secondo problema è che non magari non esiste un oggetto "generico" che vada bene per tutti i suoi derivati, e quindi non si saprebbe come crearlo.

4. **Operazioni di aggancio (hook):** sono metodi con un corpo vuoto nella superclasse, il template method li chiama, ma di default non succede niente. La sottoclasse può sovrascriverli se vuole aggiungere un comportamento in quel punto dell'algoritmo, ma non è obbligata.
   La differenza tra operazioni primitve e di aggancio è la seguente: le operazioni primitive sono **indispensabili** per l'algoritmo, quelle di aggancio sono **opzionali** (permettono di personalizzare l'algoritmo in un determinato punto se se ne ha bisogno).

- Le operazioni primitive **devono** essere ridefinite, le operazioni di aggancio **possono** essere ridefinite, nelle sottoclassi.

Nota di dettaglio: quando diciamo qualcosa come "operazioni concrete su ConcreteClass" intendiamo operazioni concrete appartenenti a/definite in ConcreteClass.

```java
public abstract class AbstractClass {
    public void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
}
```

```java
public class ConcreteClass extends AbstractClass {
    @Override
    protected void primitiveOperation1() {
       // codice per l'operazione
    }
    @Override
    protected void primitiveOperation2() {
    }
}
```

Codice del D.P.

Classi e Oggetti

Le classi formano il **vocabolario del dominio di un problema**. Le classi sono statiche nel senso che per loro l'esistenza, la semantica e le relazioni sono fissate prima dell'esecuzione del programma. La classe per ciascun oggetto è statica, ovvero una volta che l'oggetto è creato la classe è fissata. Gli oggetti sono creati e distrutti durante l'esecuzione di un'applicazione.

D.P. Singleton

Intento: assicurare che una classe abbia una sola istanza e fornire un punto di accesso globale all'istanza.

Motivazioni:

- Alcune classi dovrebbero avere esattamente una istanza in tutta l'applicazione. Un po' di **esempi**: uno spooler di stampa (ovvero il sistema di memorizzazione della stampante), un file system (un solo gestore che conosce lo stato del disco), un window manager (un solo gestore che sa quali finestre esistono e dove), una lista clienti (un solo oggetto che rappresenta lo stato corrente della lista), etc.
- Una **variabile globale** rende un oggetto accessibile, ma non proibisce di avere più oggetti per una classe, quindi non va bene come soluzione.
- La classe stessa dovrebbe essere responsabile di tener traccia del suo unico punto di accesso.

Soluzioni:

- La classe che deve essere un **Singleton** dovrà implementare un'operazione **getInstance() sulla classe** (quindi in Java è un **metodo static**), che ritorna l'unica istanza creata.
- La classe Singleton è **responsabile per la creazione dell'istanza**. Il Singleton, quindi, si "auto-protegge", infatti è proprio lui stesso l'istanza che ci interessa.
- Il **costruttore** della classe Singleton è **privato**, così da non permettere la creazione tramite new ad altre classi.

Chi crea il Singleton? Ci pensa proprio getInstance(). Infatti, la prima volta che viene chiamato controlla se l'istanza, ovvero **uniqueInstance**, è NULL. Se lo è, chiama il costruttore (è l'unico a poter chiamare il costruttore, visto che è privato),

se invece non è NULL, allora ritorna l'istanza che già esiste, senza crearne una nuova.

Due versioni del Singleton:

1. **Eager:** l'inizializzazione dell'oggetto viene fatta non appena la classe viene caricata, e viene fatta fuori dai metodi. Si parla di **inizializzazione anticipata**. È thread-safe di default, a differenza della versione lazy.
2. **Lazy:** l'inizializzazione avviene solo quando viene chiamato il metodo getInstance(). Permette di risparmiare risorse se il Singleton non viene mai usato. È la versione di default del Singleton. Tuttavia, non è thread-safe di default, in quanto due thread potrebbero fare il controllo `if uniqueInstance == NULL` nello stesso momento, e creare quindi due istanze del Singleton.

Dubbio: stiamo chiamando un metodo su un oggetto che non esiste ancora? No, perché il metodo che stiamo chiamando, ovvero getInstance(), è statico, e infatti lo stiamo **chiamando sulla classe**, e non su un'istanza. Ricordiamo infatti che **i metodi statici sono metodi di classe**.

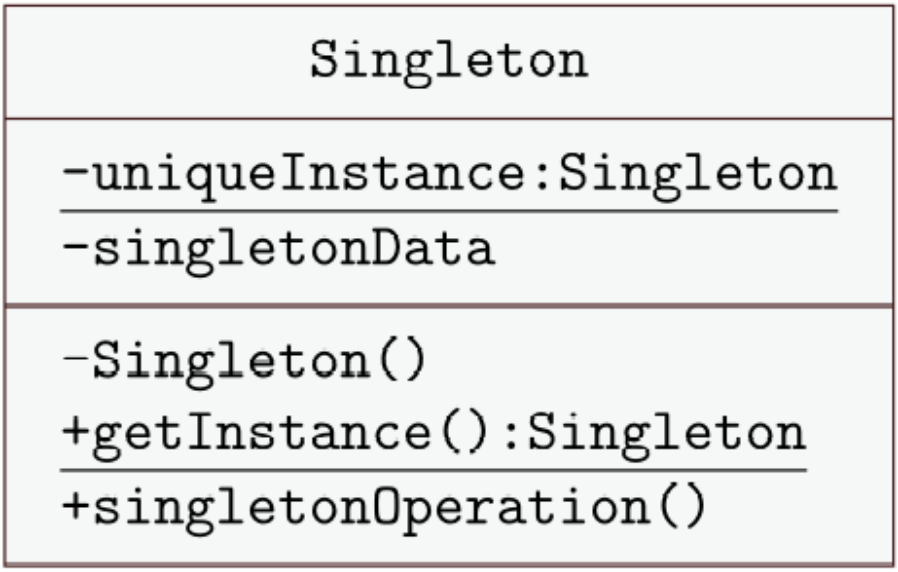


Diagramma UML del Singleton

Considerazioni e conseguenze:

- Grazie al Design Pattern Singleton, le classi non possono avere più di un'istanza a runtime.
- Nel caso dell'esempio delle slide, dove la classe che fa il ruolo del Singleton è Fib (per i numeri di Fibonacci), diremo che la classe Fib **implementa (o svolge)** il ruolo di Singleton.
- Se dopo aver realizzato una classe che svolge il ruolo di Singleton volessimo avere più istanze di quella classe anziché una sola, il codice delle classi chiamanti rimarrebbe invariato, e la sola classe da modificare è la classe che implementa il Singleton. La variante che permette un numero finito di istanze si chiama Multiton.
- **Principio delle Conseguenze Locali:** un cambiamento in qualche punto del codice non dovrebbe causare problemi in altri punti, questo è il principio che viene seguito anche dal Singleton, come abbiamo appena visto. I client del Singleton, infatti, chiamano getInstance con variabili diverse, ma puntano tutte allo stesso oggetto. Quando il Singleton viene modificato, queste variabili punteranno a oggetti diversi, in base al numero di istanze. Quindi i client non si accorgeranno nemmeno della modifica, in quanto la modifica sarà in termini di **comportamento**, e non di codice. L'unico codice modificato sarebbe quello della classe Singleton.
- **Multiton:** supponiamo di avere la classe Pagamenti come Multiton. Potremmo associare una istanza a ciascuna cartella, e la decisione sull'istanziazione dipende quindi dall'aver già creato (o meno) un'istanza per una data cartella.
- La classe che è un Singleton ha **pieno controllo** di come e quando i client accedono al valore della sola istanza. Inoltre, evita che esistano variabili globali che tengono la sola istanza condivisa.
- Permette di controllare il numero di istanze create in un programma, facilmente e in un solo punto.
- La soluzione è più flessibile rispetto a quella di **usare static per tutte le operazioni e le variabili**, poiché si può facilmente cambiare il numero di istanze consentite. Tuttavia, la soluzione con tutto static funzionerebbe comunque. Solitamente, con questo tipo di soluzione, il costruttore si mette privato per impedire che qualcuno possa chiamarlo. In questo caso, però, nemmeno getInstance potrà chiamarlo, in quanto non esisteranno mai delle

istanze della classe, ma lavoreremo solo sulla classe in modo completamente statico. Quindi **nessun costruttore chiamato**.

- L'unico frammento di codice da modificare quando si vuole variare il numero di istanze create è quello della classe che è Singleton, mentre usando static si dovrebbero modificare tutte le invocazioni.

D.P. Prototype

Idea: quando creiamo degli oggetti con `new NomeClasse()`, lo stiamo creando completamente da 0. A volte, invece di crearlo "vuoto", potrebbe esserci utile copiarne uno che esiste già. Questo oggetto che esiste già è noto come **prototipo**.

Intento: specificare i tipi di oggetti da creare usando una istanza di un prototipo, e creare nuovi oggetti copiando questo prototipo.

Motivazioni:

- **Stato iniziale complesso**: durante la creazione di un nuovo oggetto, può essere necessario avere uno stato iniziale che è simile a quello di un altro oggetto esistente, e che quest'ultimo ha ottenuto per mezzo di operazioni eseguite su esso. Per esempio, immaginiamo di avere un oggetto `Documento` che abbiamo configurato dopo 10 operazioni (impostato il font, margini, lingua, stile, etc). Vogliamo crearne un altro uguale come punto di partenza. Ovviamente non ci conviene rifare le stesse operazioni una seconda volta, ma ha più senso clonare l'oggetto esistente.
- **Accoppiamento stretto con la classe:** quando si crea un oggetto bisogna indicare una specifica classe, e così la classe che chiede l'istanza è strettamente accoppiata con la classe da istanziare.
  Cos'è l'accoppiamento stretto? Scrivendo la seguente:

  ```
  Forma f = new Cerchio();
  ```

  Il codice **conosce esplicitamente** `Cerchio`. Se poi vogliamo che f sia un `Quadrato`, dovremo modificare quel codice. Questo è l'accoppiamento stretto. Con il Prototype, invece, il nome dell'oggetto che vogliamo creare non lo specifichiamo con il nome della classe (scrivendo `new NomeClasse()`), ma lo

specifichiamo dando un'istanza già pronta da clonare. **Il tipo emerge dall'istanza, non dalla classe scritta nel codice.** Scriviamo, infatti, una cosa di questo tipo:

```
Forma f = prototipo.clone();
```

In questo modo, il codice che crea l'oggetto **non sa** qual è la classe concreta. Sa solo che può clonarlo. Rompe il **coupling**.

- **Difficolta ad aggiungere nuovi tipi:** lo **stretto accoppiamento** fra classi da istanziare e chi istanzia rende difficile l'aggiunta di nuovi tipi. Nel Factory Method, per esempio, bisogna creare una gerarchia di Creator per aggiungere nuovi tipi. Se avessimo, per esempio, `CerchioCreator` e `QuadratoCreator` e volessimo aggiungere `Triangolo`, dovremmo aggiungere una nuova classe `TriangoloCreator`. Quindi la gerarchia dei creator **cresce** insieme alla gerarchia dei prodotti. Col Prototype non serve nessun Creator. Per supportare Triangolo, registriamo semplicemente un'istanza prototipo di Triangolo. **Non aggiungiamo classi, aggiungiamo oggetti**.

Soluzione:

- **Prototype:** definisce l'interfaccia per le operazioni comuni e dichiara un metodo per clonare sé stessa.
- **ConcretePrototype:** classe che implementa l'interfaccia Prototype e l'operazione di clonazione.
- **Client:** classe che crea nuovi oggetti tramite clonazione dell'istanza prototipo, quindi senza indicare la classe esplicitamente. Client **conosce solo l'interfaccia Prototype**.

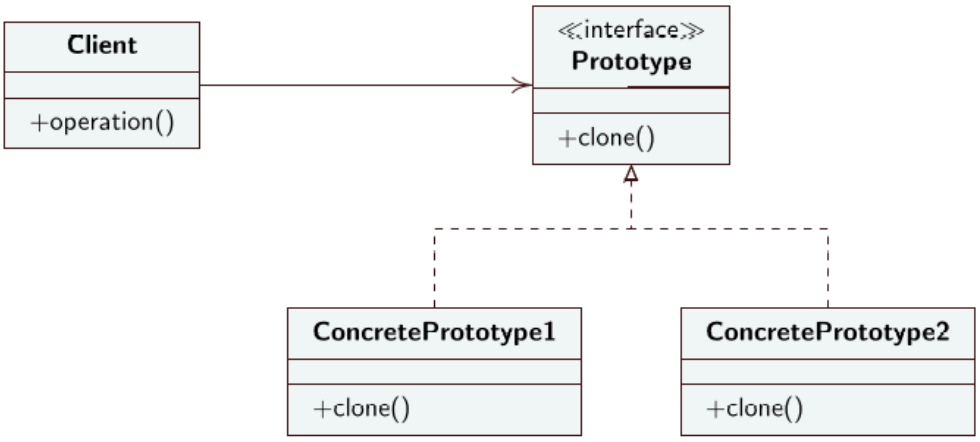


Diagramma UML del D.P. Prototype

- **Come funziona la registrazione dei Prototipi?** Si ha un **registro di prototipi**, ovvero una mappa (`nome -> istanza`), e può facilmente essere modificata mentre il programma è in esecuzione. Di registro ne esiste **uno solo**. Non appartiene né a Prototype né a ConcretePrototype. È una **classe separata**, solitamente nota come `PrototypeRegistry`. Internamente è come una mappa. Di solito questa classe è un **Singleton**, oppure viene passata come dipendenza. È il punto centrale da cui tutti i client pescano i prototipi.
- **Come fa il client a sapere quale voce gli interessa?** Il client conosce una **chiave semantica**, ovvero il nome logico di quello che vuole, ma non il nome della classe. Il client **sa cosa vuole funzionalmente**, ma non come è fatto internamente.
- **Perché servono più ConcretePrototype?** Perché ognuno poi implementa `clone()` in modo appropriato per sé stesso. Servono classi diverse perché **oggetti diversi hanno struttura e comportamento diversi**. Un cerchio ha un raggio, un Quadrato ha un lato, etc.
- Nell'esempio precedente, sul Cerchio e sul Quadrato, loro sarebbero i ConcretePrototype che implementano Prototype. Per questo non possiamo permetterci di avere una sola classe ConcretePrototype, perché sennò dovremmo schiacciare tutti gli attributi di più classi distinte in una sola classe, e ciò ovviamente non va bene. Abbiamo più `ConcretePrototype` perché abbiamo più **tipi di oggetti strutturalmente diversi** che vogliamo poter clonare.

Conseguenze:

- Il client non conosce la classe usata, poiché il client usa l'interfaccia comune, quindi il **numero di dipendenze del client è ridotto**. Come conseguenza di ciò, il client può usare classi specifiche (sottoclassi), pur non conoscendone il nome.
- Solitamente, il codice che crea oggetti è fisso a compile-time. Così, invece, si può **registrare un nuovo prototipo** (o rimuovere un prototipo) **a runtime**. Ovviamente, possiamo anche costruire l'oggetto prototipo a runtime, per poi poterlo registrare come prototipo. Questo è possibile grazie al registro di prototipi di cui abbiamo parlato poco sopra.
- Si possono specificare nuovi oggetti con valori diversi, **senza creare nuove classi**. Senza Prototype, infatti, potremmo essere tentati di creare una classe per ogni possibile variante. Per esempio, classi come `CerchioRosso`, `CerchioBlu`, `CerchioGrande`, sono "sprecate". Conviene invece clonare un Cerchio e **cambiare l'attributo** dopo la clonazione. La variazione sta nello **stato**, non nella classe. **Evitiamo l'esplosione di sottoclassi**.
- **Prototipo con struttura composita:** un nuovo prototipo può essere usato per tenere una struttura diversa di oggetti (**composizione**). Gli oggetti della composizione sono attributi del prototipo, come in un **Composite**. Si avrà quindi come Prototipo un oggetto **composto da altri oggetti**. Per esempio un `Pannello` che contiene al suo interno `Bottoni`, `Label`, `TextField`. Questi sotto-oggetti sono attributi del prototipo. Clonando il prototipo otteniamo una copia dell'intera struttura composita in un colpo solo, senza doverla ricostruire pezzo per pezzo. Utile quando il Prototype collabora con il Composite.
- Pur avendo tante classi diverse da poter usare (i ConcretePrototype) non si ha la necessità di **implementare una gerarchia di Creator** (come per il Factory Method), in quanto un nuovo oggetto si ottiene tramite clonazione. Con Prototype, infatti, **aggiungiamo solo un'istanza al registro**, non una classe.

Prototype: Implementazione

- L'uso di un **manager di prototipi** (ovvero il registro di istanze visto poco prima) permette ai client di immagazzinare e recuperare i prototipi, prima di clonarli.
- L'implementazione dell'operazione di clonazione può essere la parte più difficile quando le strutture contengono **riferimenti circolari**. Il problema si

verifica in situazioni come le seguenti: una classe `Dipendente` con attributo Manager, e una classe `Manager` con attributo lista dipendenti. Il **problema emerge durante la deep copy:** per clonare Dipendente dobbiamo clonare prima il suo Manager, ma per clonare il Manager dobbiamo prima clonare i suoi Dipendenti, che a loro volta richiedono di clonare il Manager, e così via. Per gestirlo correttamente bisogna tenere traccia degli oggetti già clonati durante la copia, e riusare il clone già creato invece di ricominciare. Parte più delicata dell'implementazione. Si usa una **mappa degli oggetti già clonati**.

- Sebbene un'operazione di clonazione potrebbe essere fornita dal linguaggio di programmazione, l'operazione di clonazione potrebbe effettuare una **copia superficiale (shallow copy)** anziché una **copia profonda (deep copy)**.
  In una shallow copy i riferimenti tenuti negli attributi dell'oggetto da clonare sarebbero condivisi fra originale e clone, e questo spesso non è sufficiente, quindi occorre una deep copy, dove ognuno degli oggetti ha effettivamente la propria copia personale degli attributi in questione.
- I cloni potrebbero necessitare di essere inizializzati. La classe prototipo potrebbe definire operazioni per impostare dati. Dato che il clone nasce con lo stesso stato del prototipo, **a volte alcune informazioni dovranno essere diverse per ogni clone**. Per esempio: il prototipo `Contratto`, template configurato con clausole standard. Lo cloniamo per creare un contratto reale, ma il clone ha ancora l'ID del prototipo, la data di creazione del prototipo, nome del cliente del prototipo e così via. Cioè, è ancora un oggetto "generico" e ci serve renderlo più "personale", aggiungendo dei dati che sono specifici per ogni clone. La classe ConcretePrototype, quindi, potrà definire metodi appositi per questa **inizializzazione post-clonazione**. Questi metodi ovviamente potrebbero essere dichiarati e definiti anche in Prototype.

Esempio Prototype

Si vuole tenere un'agenda di impegni. Varie voci possono essere aggiunte all'agenda. Lo stato dell'agenda è un insieme di impegni presi. Una nuova istanza dell'agenza deve poter avere accesso agli impegni precedentemente inseriti.

In modo simile, si vogliono inserire note in un quaderno e accedere alle note precedenti quando si ha una nuova istanza di quaderno.

D.P. Adapter

Intento: converte l'interfaccia di una classe in un'altra interfaccia che i client si aspettano. Adapter permette ad alcune classi di interagire, eliminando il problema di interfacce incompatibili.

Problema:

- Alcune volte una classe di una libreria non può essere usata poiché incompatibile con l'interfaccia che si aspetta l'applicazione. Ovvero nome metodo, parametri, tipo parametri, di chiamate all'interno dell'applicazione che non sono corrispondenti a quelli offerti da una classe di libreria.
- Non è possibile cambiare l'interfaccia della libreria, poiché non si ha il sorgente (e comunque non conviene cambiarla).
- Non è possibile cambiare l'applicazione (il chiamante), e si può voler cambiare quale metodo chiamare, senza renderlo noto al chiamante.

Soluzione (versione Object Adapter):

- **Target:** è l'interfaccia che il chiamante si aspetta.
- **Client:** us oggetti che sono conformi all'interfaccia Target.
- **Adaptee:** è l'oggetto di libreria che necessita l'adattamento.
- **Adapter:** converte, ovvero adatta, la chiamata che fa una classe client all'interfaccia della classe di libreria. Il chiamante usa l'Adapter come se fosse l'oggetto della libreria. La classe con ruolo **Adapter implementa Target, tiene il riferimento all'oggetto di libreria (Adaptee)**, e sa come invocarlo, ovvero chiama i metodi di Adaptee.
- Segue il meccanismo della **composizione**.

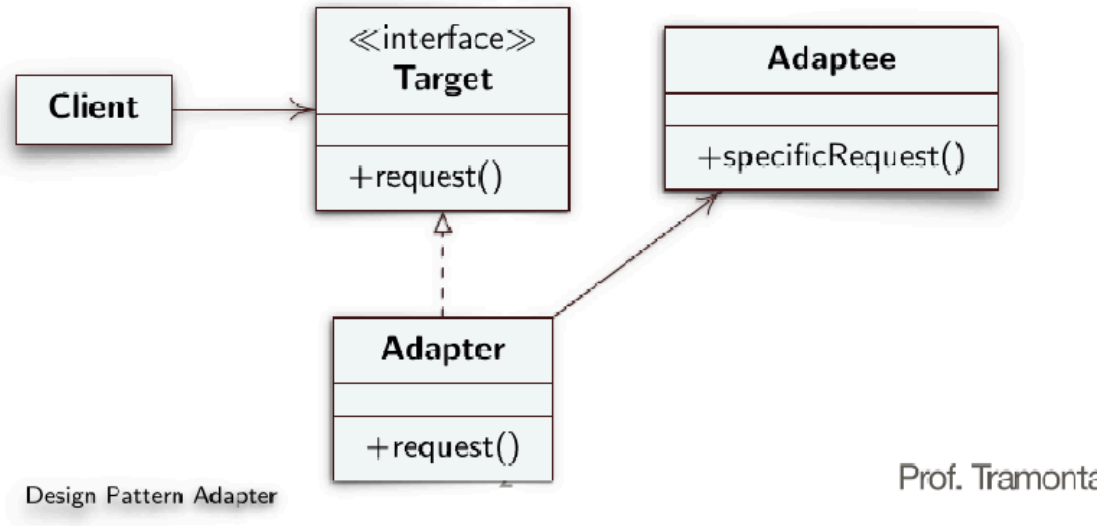


Diagramma delle classi dell'Adapter

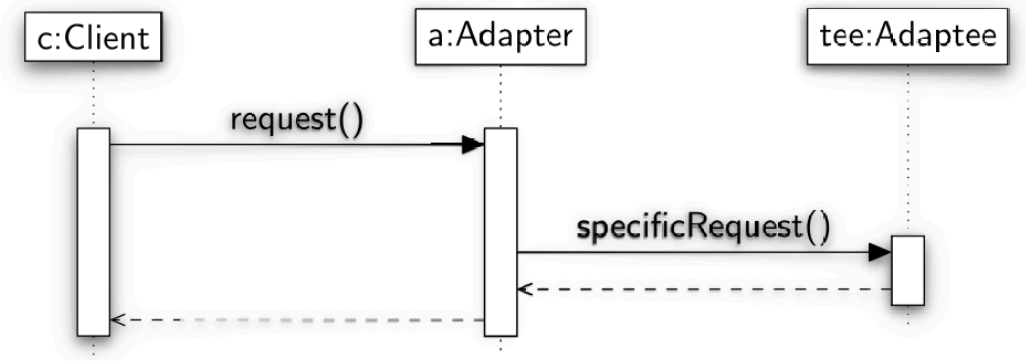


Diagramma di sequenza dell'Adapter

Soluzione (versione Class Adapter):

- Identica alla precedente, ma qui Adaptee è superclasse di Adapter, e quindi Adapter non tiene un riferimento a un oggetto di tipo Adaptee, ma si limita a chiamare i metodi della superclasse. La **differenza** è che non **posso scegliere a runtime quale Adaptee usare**, perché è già "hardcoded" nella gerarchia. Segue il meccanismo dell'**ereditarietà**.

- Una proprietà di questa versione è il **two-way Adapter**. Adapter espone entrambe le interfacce allo stesso tempo, in quanto implementa Target ed estende Adaptee, in quanto può essere passato a codice che si aspetta un Target e a codice che si aspetta un Adaptee. **Non adatta solo in una direzione**, ma può essere in mezzo a due sistemi che parlano linguaggi diversi e fare da ponte.

Conseguenze:

- Client e classe di libreria Adaptee rimangono indipendenti. Client, infatti, conosce solo Target, e Adaptee conosce solo sé stesso. Possiamo cambiare l'Adaptee modificando semplicemente il codice di Adapter, senza toccare il client, e viceversa.
- Il ruolo Adapter può cambiare il comportamento dell'Adaptee, in quanto può aggiungere **test di precondizioni e postcondizioni**. Precondizioni: cosa si deve soddisfare prima di eseguire. Postcondizioni: cosa è verificato se tutto è andato bene.
- L'Object Adapter può implementare la tecnica di **Lazy Initialisation**, ovvero si aspetta che avvenga un'invocazione a un metodo di Adaptee prima di istanziarlo. Alternativa al costruire l'Adaptee direttamente nel costruttore. Conviene come approccio perché potremmo sprecare risorse e non è detto che lo useremo.
- L'Adapter aggiunge un **livello di indirettezza**. Ogni invocazione del Client ne scatena un'altra fatta dal ruolo di Adapter. Possibile **rallentamento** (trascurabile), e codice più difficile da comprendere, quindi paghiamo un costo di **leggibilità**.
- Se Target e Adapter coincidessero, il Client si ritroverebbe direttamente accoppiato alla libreria, e quindi non favorirebbe il disaccoppiamento, e quindi porterebbe a una cascata di cambiamenti nel caso in cui la libreria venisse aggiornata, in quanto dovremmo modificare Client, Target, e le eventuali sottoclassi di Target. Una modifica che sarebbe potuta essere locale e solo su Adapter si propaga invece su tutto il sistema.

Altro esempio di Diagramma:

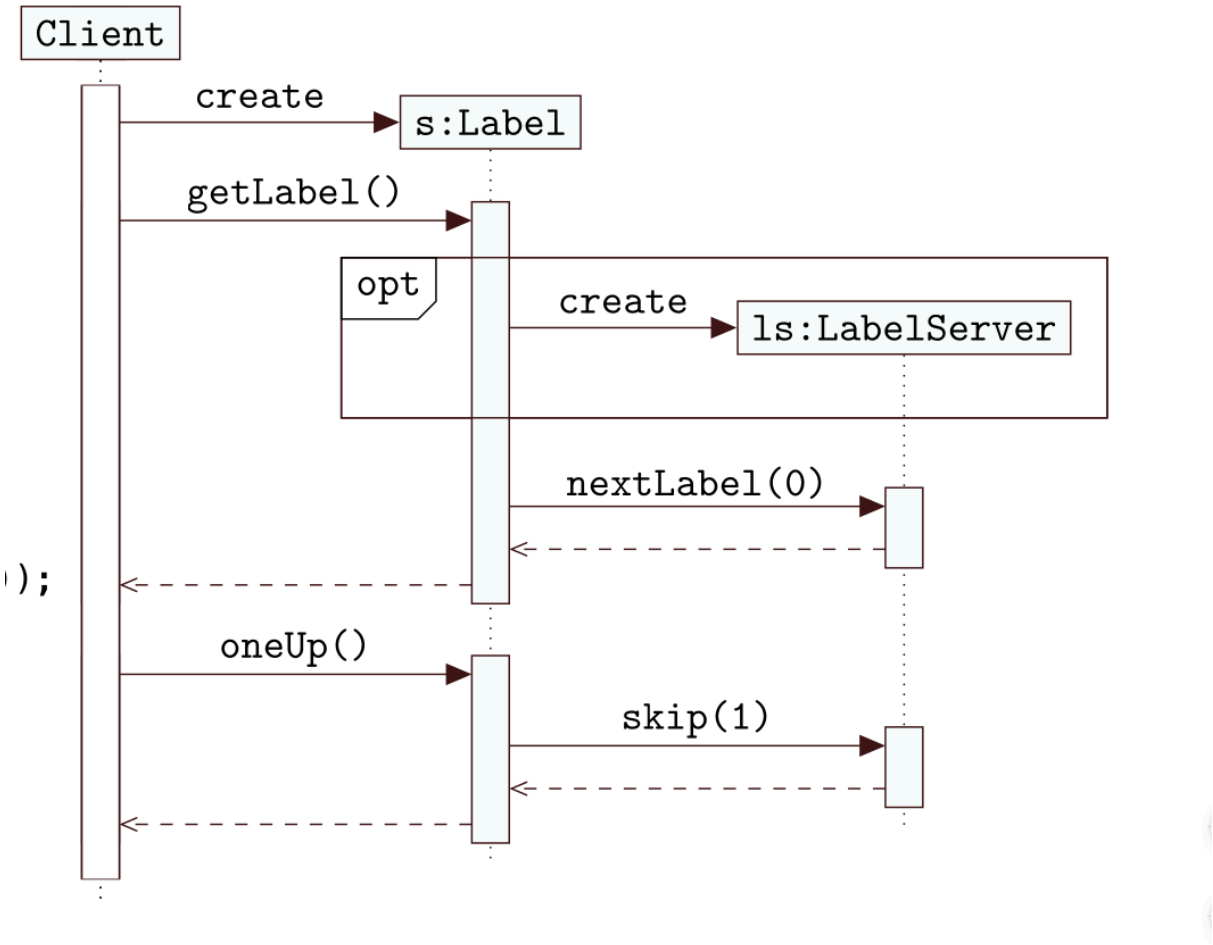


Diagramma UML di sequenza su un esempio specifico

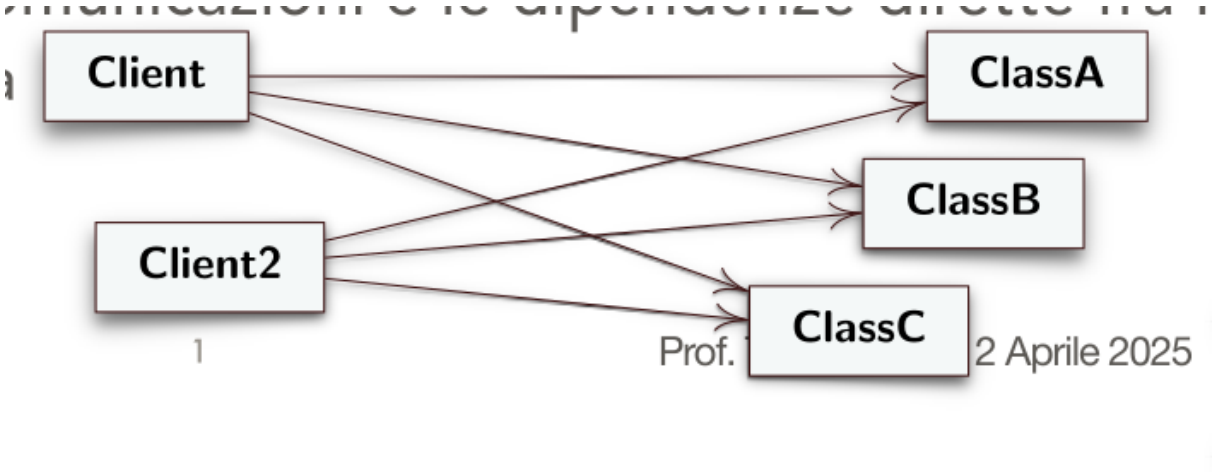
Nota: quest'ultimo diagramma è interessante in quanto ci mostra una notazione dell'UML di sequenza. Il rettangolo con opt in alto a sinistra indica un'operazione **opzionale (optional)**. Equivale a un if senza else. Per fare if/else si usa il rettangolo con **alt**, e per rappresentare i loop si usa **loop**.

D.P. Facade

Intento: fornire un'**interfaccia unificata** al posto di un insieme di interfacce in un **sottosistema** (consistente di un insieme di classi). Definire un'interfaccia di **alto livello** (semplificata) che rende il sottosistema più facile da usare.

Problema:

- Spesso si hanno tante classi che svolgono funzioni correlate e l'insieme delle interfacce può essere complesso.
- Può essere difficile capire qual è l'interfaccia essenziale ai client per l'insieme di classi.
- Si vogliono ridurre le comunicazioni e le dipendenze dirette fra i client ed il sottosistema.



La situazione in diagramma UML prima di usare il Facade

Soluzione:

- **Facade:** fornisce un'unica interfaccia semplificata ai client e nasconde gli oggetti del sottosistema, questo riduce la complessità dell'interfaccia e quindi delle chiamate. Facade si occupa di invocare i metodi degli oggetti che nasconde.
- **Client:** interagisce solo con l'oggetto Facade.

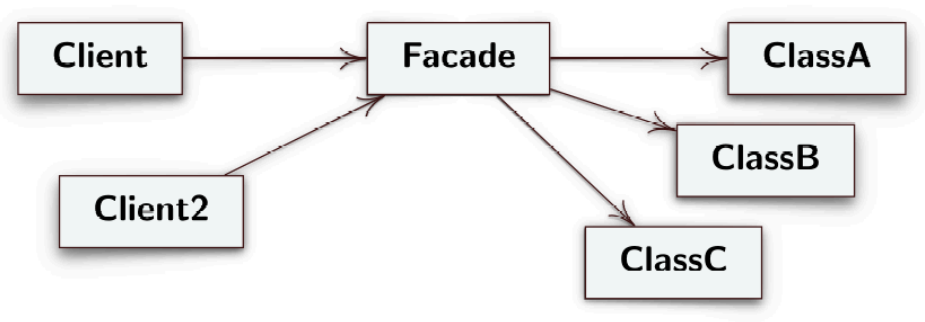


Diagramma UML del Facade

Conseguenze:

- **Nasconde ai client l'implementazione** del sottosistema.
- Promuove l'**accoppiamento debole** tra sottosistema e client. In questo modo, se un domani le classi del sottosistema dovessero subire modifiche, il client rimarrebbe invariato (isolato dai cambiamenti), e gli unici aggiustamenti sarebbero da fare sul Facade.
- Riduce le dipendenze di compilazione in sistemi grandi. Se si cambia una classe del sottosistema, si può ricompilare la parte di sottosistema fino al Facade, quindi non i vari client. Es. se cambia `English.h`, il compilatore ricompila tutti i file che lo includono direttamente. Con Facade, solo `Facade.cpp` include le classi del sottosistema, e quindi solo lui viene ricompilato, non tutti i client.
- Non prevenie l'uso di client più complessi, quando occorre, che accedono ad oggetti del sottosistema. Facade **non obbliga** nessuno a usare solo l'interfaccia semplificata. Se un client avanzato ha bisogno di fare qualcosa che Facade non espone, può comunque accedere direttamente alle classi del sottosistema. Facade **è una comodità, non un blocco**.
- **Implementazione:** per rendere gli oggetti del sottosistema non accessibili al client, le corrispondenti classi possono essere annidate dentro la classe Facade. Quindi, se vogliamo che i client **non possano mai accedere** direttamente al sottosistema, possiamo annidare le classi del sottosistema dentro Facade.

- Il Facade **non elimina la complessità:** la **sposta e la incapsula** in un posto solo. Il client ci guadagna in semplicità, e il sottosistema resta invariato.

D.P. Composite

Intento: comporre oggetti in strutture ad albero per rappresentare gerarchie di parti o del tutto. Composite permette ai client di trattare oggetti singoli e composizioni di oggetti uniformemente.

Problema:

- Immaginando di lavorare su un File System, abbiamo due tipi di oggetti: File e Cartelle. Senza Composite, il codice Client che vuole fare un'operazione (come `stampaContenuto`) deve distinguere i due casi: se l'elemento considerato è un file oppure una cartella.
- Supponendo di gestirla tramite if/else, se a un certo punto notassimo che l'elemento non è un file, significa che è una cartella. A questo punto, stampiamo il contenuto della cartella e poi procediamo a stampare il contenuto dei singoli elementi della cartella. Tuttavia, non sappiamo se i singoli elementi della cartella sono a loro volta cartelle o file, e quindi dovremmo scrivere altri **if/else annidati**, e chiaramente questo non è un approccio corretto per risolvere il problema.
- Vogliamo fare in modo che **file e cartelle (Leaf e Component) implementino la stessa interfaccia**, e così il client non chiederà mai se l'elemento è un file o una cartella, e in questo modo **si può chiamare semplicemente** `stampaContenuto` su ciascuno. Che siano file o cartelle, non importa: è **ricorsione naturale**.

Motivazione:

- È necessario **raggruppare** elementi semplici tra loro per formare elementi più grandi.
- Se nell'implementazione c'è **distinzione** tra classi per elementi semplici e classi per contenitori di questi elementi semplici, il codice che usa queste classi deve trattarli in modo differente. Questa distinzione rende il codice più complicato.

- Composite permette di descrivere una composizione **ricorsiva**, in modo che i client non debbano fare distinzione tra tipi di elementi. I client tratteranno tutti gli oggetti della struttura uniformemente.

Esempio 1, i File:

- Le operazioni su disco permettono la gestione di file (immagini, testo, etc) e la gestione di cartelle. Esempi di operazioni sono: lettura, scrittura, esecuzione, creazione.
- Le classi client devono poter fare **operazioni su file e cartelle**, senza distinguere fra essi.
- Una cartella deve poter contenere al suo interno sia file che altre cartelle, queste a sua volta contengono altri elementi, e così via.

Esempio 2, le Figure:

- In un editor di figure ho sia figure semplici, come Linea, Box, etc, che figure composte, ovvero **raggruppamenti** di figure semplici e composte.
- Le classi client devono poter trattare allo stesso modo sia le figure semplici che quelle composte, quando avviano operazioni come disegna, sposta, etc.

Soluzione:

- **Component:** interfaccia che rappresenta elementi semplici e non. Si occupa di:
  1. **Dichiarare le operazioni** degli oggetti della composizione, ovvero le operazioni che il client chiamerà su tutti;
  2. **Implementare le operazioni comuni alle sottoclassi** in modo appropriato. Così se c'è qualcosa che Leaf e Composite fanno allo stesso modo, possiamo metterlo qui una volta sola;
  3. **Dichiarare le operazioni per l'accesso e la gestione degli elementi semplici**. Queste operazioni servono solo a Composite, ma stanno su Component così permettono al client di usarle senza dover fare `instanceof`. Sono operazioni come `aggiungi()`, `rimuovi()`, `getFigli()` etc.
  4. Può anche definire un'operazione che permette ad un elemento di **accedere all'oggetto padre** nella struttura ricorsiva.

Grazie a Component, **il client non deve sapere con chi sta parlando**.

- **Leaf:** classe che rappresenta **elementi semplici** (detti **child/children**). È sottoclasse di Component. Implementa il comportamento degli oggetti semplici, infatti implementa concretamente le operazioni dichiarate da Component. Operazioni come `aggiungi()` e `rimuovi()` o non le implementa, o le implementa lanciando un errore. Quando il client chiama `stampaContenuto()` su una Leaf, semplicemente esegue l'operazione e basta, senza nessun tipo di ricorsione.
- **Composite:** classe che rappresenta **elementi contenitori**. È sottoclasse di Component.
  1. Definisce il comportamento per l'aggregato di elementi child.
  2. Tiene il riferimento a ciascuno degli elementi child e **implementa operazioni per gestire elementi child**. Tiene una lista di Component, non di Leaf o di Composite specifici. Questo rende la struttura arbitrariamente profonda.
  3. Si occupa anche di **delegare operazioni**: quando il client chiama `stampaContenuto()` su un Composite, lui si occupa di iterare questa istruzione sui figli e di chiamare il metodo su ciascuno.

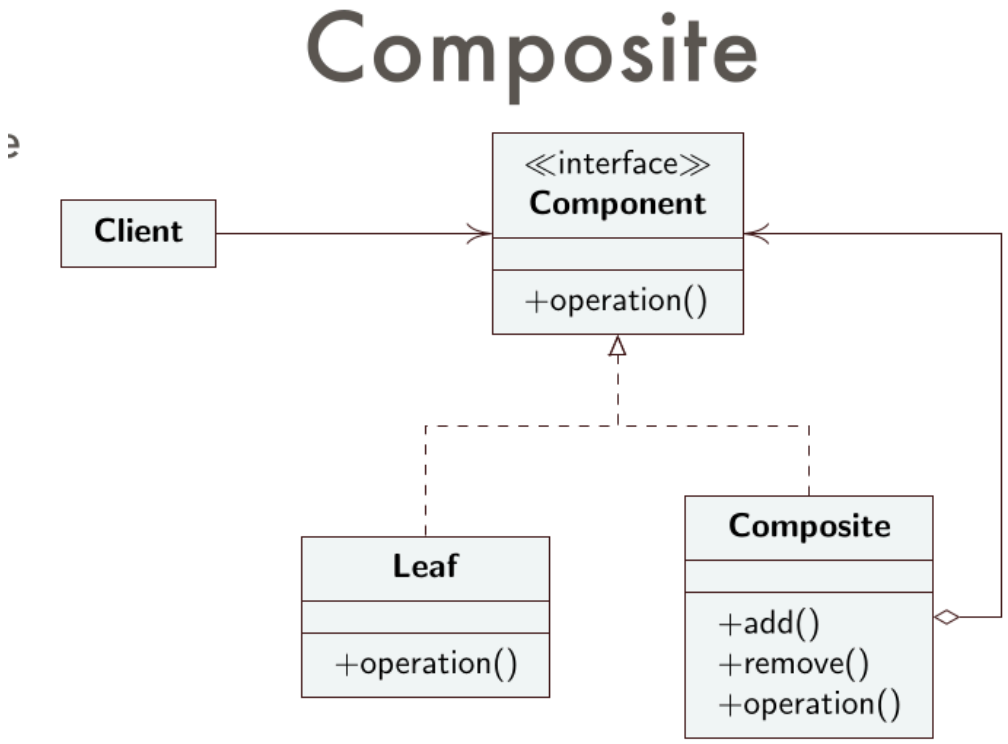


Diagramma UML delle classi del D.P. Composite

Tipi di relazioni:

- **Associazione:** rappresenta una relazaione in cui una classe **usa** un'altra classe. Es. Client che usa Component.
- **Aggregazione:** rappresenta una relazione in cui una classe (**whole**) consiste di altre classi (**parti**). Ovvero, il **Composite aggrega (e consiste di) Leaf** (parti). Chi aggrega e le parti **esistono indipendentemente** l'uno dall'altro. Quindi le parti **hanno vita propria**. Si rappresenta con un **rombo vuoto** sul lato del whole. Nel nostro esempio, possiamo prendere un File e spostarlo in un'altra Cartella, non viene distrutto insieme al contenitore.
- **Composizione:** una forma di aggregazione in cui la parte non può esistere senza l'intero (whole). La parte appartiene all'intero ed è distrutta quando l'intero è distrutto. Si rappresenta con un **rombo pieno**. In generale, usiamo il rombo pieno se l'oggetto A possiede/contiene un oggetto B che senza l'oggetto A non avrebbe senso di esistere. Per esempio, uno Stato è **intrinsecamente legato** al suo Ordine, e Stato come informazione a sé stante,

senza il suo ordine, non ci dice niente. Se muore Ordine, muore Stato. Quindi siamo nel caso in cui B **non ha vita propria** fuori dal suo contenitore, c'è un **vincolo di composizione**.
Nota: A potrebbe tranquillamente continuare a esistere anche senza B.

Collaborazioni:

- I client usano l'interfaccia di Component per interagire con elementi della struttura composita. Se il ricevente del messaggio è Leaf, la richiesta è gestita direttamente. Se il ricevente è un Composite, questo invia la richiesta ai suoi child e possibilmente avvia operazioni addizionali prima e dopo.

Conseguenze:

- Elementi semplici possono essere composti in elementi più complessi, questi possono essere composti, e così via. **Composizione ricorsiva**.
- Un client che si aspetta un elemento semplice può riceverne uno composto.
- **I client sono semplici**, trattano strutture composte e semplici uniformemente. I client non devono sapere se trattano con un Leaf o con un Composite.
- Nuovi tipi di elementi (Leaf o Composite) possono essere aggiunti e potranno funzionare con la struttura e i client esistenti. Per esempio, quando aggiungiamo `Shortcut extends Component`, ci basta implementare `operation()` al suo interno, e il client esistente **non va toccato**. Il client così riuscirà a chiamare operation() su Shortcut esattamente come su File o Cartella, senza nemmeno sapere che Shortcut esiste. Segue il principio **Open/Closed:** aperto all'estensione (aggiungendo nuovi tipi), ma chiuso alla modifica (il codice client esistente non cambia mai).
- Non è possibile a **design-time** vincolare il Composite solo su certi elementi Leaf (in base al tipo), invece dovranno essere fatti dei controlli a runtime.
Esempio: supponiamo di voler fare una Cartella che contiene solo File e non altre Cartella. Non possiamo esprimerlo nella gerarchia delle classi, in quanto Composite tiene una lista di Component generici, e quindi sarebbe necessario fare un **controllo a runtime**.

Ulteriori dettagli:

- Per facilitare la navigazione degli oggetti, gli elementi child (ovvero **qualsiasi nodo che ha un padre**) mantengono il riferimento all'oggetto che li contiene. Il

riferimento è inserito in Component, mentre Leaf e Composite possono implementare le operazioni per gestirlo.

- Per avere client che non distinguono se trattano con Leaf o Composite (è uno degli obiettivi), la classe Component dovrebbe **definire quante più operazioni in comune possibile**. Le classi Leaf e Composite faranno override delle operazioni.
- Dove implementare le **operazioni di gestione di child,** `add()` **e** `remove()`**?**
  1. Se dichiarate in Component si ha **trasparenza**, ma per le classi Leaf tali operazioni non hanno significato. La **sicurezza** nell'uso è quindi compromessa, poiché i client potrebbero avviarle su Leaf. Se i client avviano un'operazione add() su Leaf si ignora la richiesta, questo potrebbe indicare un difetto del programma.
  2. Se dichiarate in Composite si ha **sicurezza**, poiché a compile time si verifica che tali operazioni non possono essere chiamate su Leaf, ma si perde la **trasparenza**.
- Si può inserire una operazione `getComposite()` in Component che ritorna null e che è ridefinita in Composite per ritornare il riferimento a sé stesso. I client dovrebbero comunque distinguere il tipo di risultato e fare operazioni differenti, **niente trasparenza**. Stiamo semplicemente sostituendo `instanceof` con un controllo sul `null`.
- La lista che contiene elementi child deve essere definita in Composite, altrimenti se fosse definita in Component **si sprecherebbe spazio**, poiché ogni Leaf avrebbe tale variabile anche se non deve usarla mai.
- L'**ordinamento di child** per un Composite potrebbe essere importante e va tenuto in considerazione su certe implementazioni.
- Il Composite potrebbe implementare una **cache**, per ottimizzare le prestazioni. È utile quando si implementa un'operazione che deve ricercare tra tutti i suoi componenti child, come per esempio un metodo che somma la dimensioni di tutti i child. I componenti child devono poter accedere ad un'operazione che permette di invalidare la cache del Composite, in quanto se qualcuno di loro cambia qualcosa, la cache diventa **obsoleta**. Serve un'operazione del tipo `invalidaCache()` che risale al padre e gli dice: "il tuo valore cached non è più

valido, ricalcolalo la prossima volta”. La propagazione deve arrivare **fino alla radice**. Ovviamente ci si ferma quando il padre è uguale a null.

D.P. Decorator

Intento: aggiungere ulteriori **responsabilità ad un oggetto dinamicamente**. I decorator forniscono un’alternativa flessibile all’implementazone di sottoclassi per estendere funzionalità.

Motivazioni:

- Alcune volte si vogliono **aggiungere responsabilità a singoli oggetti**, e non all’intera classe. Per aggiungere responsabilità si potrebbe far uso dell’ereditarietà, tuttavia non si avrebbe flessibilità: un client non può controllare come e quando decorare un componente, in quanto la decisione è stata presa a **design-time**.
- Un approccio più flessibille è racchiudere (**wrap**) il componente in un altro oggetto che aggiunge una responsabilità. L’oggetto **wrapper è un decorator**. Il decorator è conforme all’interfaccia che decora, quindi **la sua presenza è trasparente agli oggetti che usano il componente**. Invece di modificare un oggetto esistente, lo avvolgiamo in un secondo oggetto che lo contiene come campo interno.
- Le **responsabilità possono essere sottratte dinamicamente**. I decorator possono essere **annidati ricorsivamente**, per aggiungere più responsabilità.
- A volte la **creazione di sottoclassi non è praticabile**. Un numero grande di estensioni produrrebbe un numero enorme di sottoclassi per gestire tutte le combinazioni. Usando l’ereditarietà avremmo bisogno di un numero enorme di sottoclassi per poter gestire tutte le combinazioni possibili.

Esempio:

- Si vuole aggiungere la proprietà Bordo ad un componente Testo. **Se si eredita Testo dalla classe Bordo**, gli oggetti della sottoclasse avranno questa proprietà, ma **non si avrà flessibilità**: non si possono avere oggetti senza tale proprietà.
- Alternativa flessibile, inserire il componente Testo dentro un oggetto Bordo.

- L’oggetto che racchiude, chiamato DecoratorBordo, manda la richiesta al componente Testo e aggiunge altre attività prima o dopo l’invio della richiesta.
- Il **Decorator non sostituisce** il comportamento del componente: lo delega e poi aggiunge qualcosa intorno.

Soluzione:

- **Component:** definisce l’interfaccia per gli oggetti che possono avere aggiunte le responsabilità dinamicamente. Questa interfaccia è il contratto che devono rispettare sia gli oggetti reali che i loro decorator. Grazie a ciò, il client non sa mai con chi sta parlando: se con un oggetto non wrappato o wrappato.
- **ConcreteComponent:** definisce un oggetto su cui poter aggiungere responsabilità. È l’oggetto che vogliamo decorare, e non sa niente dei decorator che lo avvolgeranno. Si occupa di implementare Component.
- **Decorator:**
    1. Mantiene un riferimento a un oggetto Component, quindi può wrappare qualsiasi cosa implementi Component, sia un ConcreteComponent che un altro Decorator.
    2. Si occupa anche di implementare Component, e quindi dall’esterno è indistinguibile da un ConcreteComponent, dato che implementano la stessa interfaccia.
    3. Decorator inoltra le richieste al suo oggetto Component e può fare altre operazioni prima e dopo l’inoltro della richiesta.
- **ConcreteDecorator:** implementa la responsabilità aggiunta al Component. Può decorare un oggetto in due modi: aggiungendo dati (stato) o aggiungendo logica (comportamento).

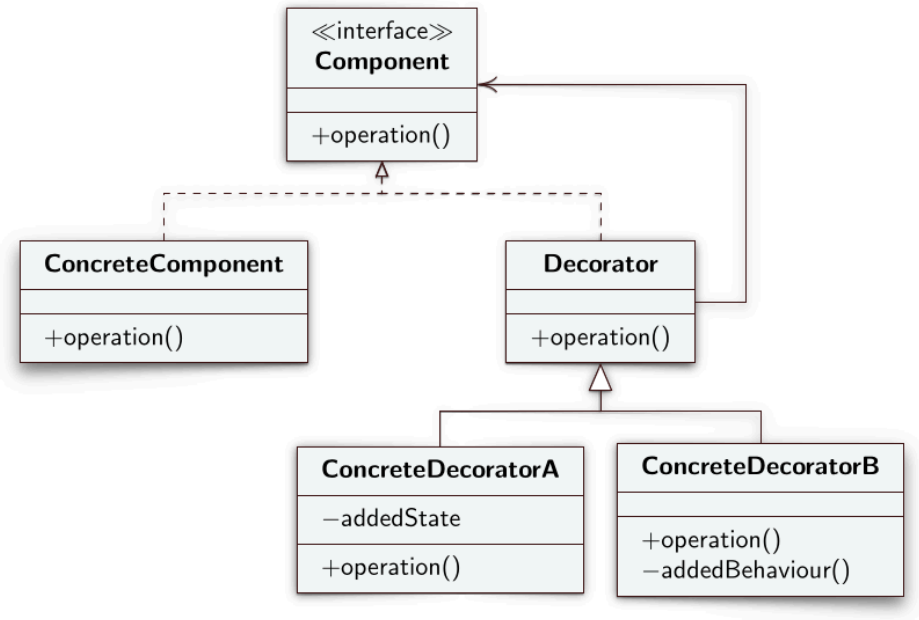


Diagramma UML delle classi di Decorator

Doppia relazione di Decorator verso Component: si occupa sia di implementarlo che di contenerne uno. Lo implementa così il client lo vede come un Component normale, e quindi garantisce **trasparenza**. Ne contiene uno perché così può wrappare qualsiasi Component, anche un altro Decorator (**annidamento**). Se non ne contenesse uno, non avrebbe modo per wrappare questi Component. Inoltre, poiché Decorator implementa Component, soddisfa quel tipo, e quindi nel campo component di Decorator si può mettere qualsiasi cosa: un ConcreteComponent o un ConcreteDecorator.

Conseguenze:

- **Più flessibilità** rispetto all'ereditarietà poiché si possono aggiungere responsabilità dinamicamente.
- La stessa responsabilità può essere aggiunta più volte, semplicemente aggiungendo due istanze dello stesso ConcreteDecorator.
- Prevedere per le classi in alto nella gerarchia tutte le responsabilità che servono significherebbe avere per esse **troppe responsabilità**. Mettere tutto in ConcreteComponent o in Component ci porta ad avere classi gonfie e che

fanno troppe cose, violando il principio di Singola Responsabilità. ConcreteDecorator sono invece indipendenti e permettono di aggiungere responsabilità successivamente.

- **Svantaggio:** l'identità (tipo e riferimento) del ConcreteDecorator non è quella del ConcreteComponent, quindi **non si dovrebbero confrontare i riferimenti**. Quando wrappiamo un ConcreteComponent dentro un ConcreteDecorator, l'oggetto risultante non è più un ConcreteComponent, e quindi se in qualche punto del codice facciamo confronti basati sul tipo o sul riferimento, il Decorator rompe quella logica.
- Si avranno tanti piccoli oggetti, che differiscono nel modo in cui sono interconnessi. Questo perché una catena di decorator produce tanti oggetti distinti in memoria, ognuno che contiene il riferimento al precedente. Ciò rende il **debug più difficile**, e infatti se qualcosa va storto bisogna seguire la catena di deleghe per capire dove non funziona. Inoltre, la **comprensione del sistema è più difficile**: un sistema con molti decorator annidati può diventare difficile da leggere se non si conosce il pattern.

D.P. Bridge

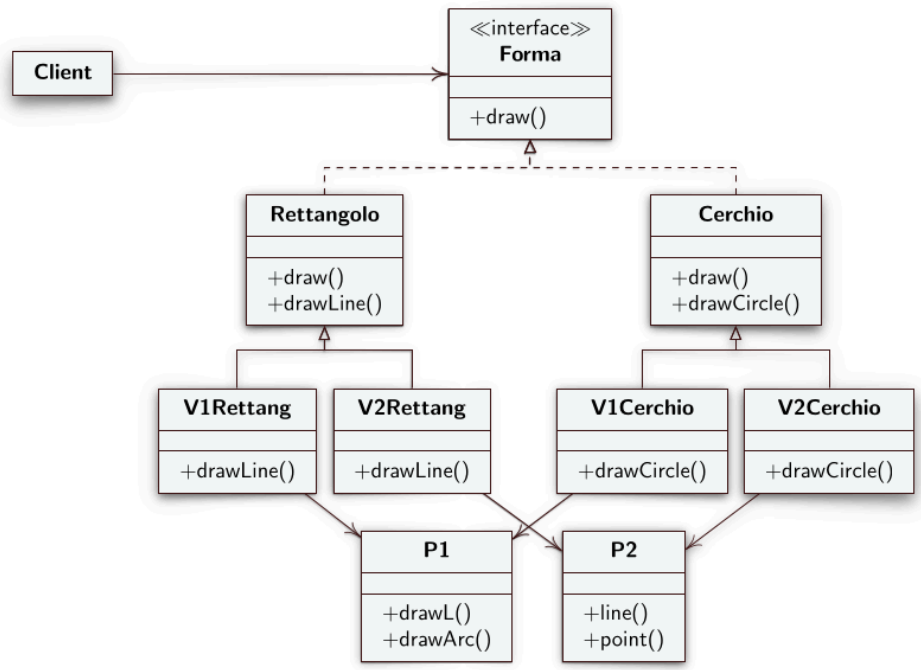
Intento: disaccoppiare una astrazione dalla sua implementazione così che le due possano variare indipendentemente.

Motivazione:

- Quando una astrazione può avere varie implementazioni, di solito si usa l'**ereditarietà**. Una classe astratta definisce l'interfaccia dell'astrazione, le sottoclassi concrete la implementano in modi diversi.
- Questo approccio **non è flessibile**, poiché **collega l'astrazione all'implementazione permanentemente**, e rende difficile modificare, estendere e usare astrazioni e implementazioni indipendentemente. Le conseguenze dell'ereditarietà sarebbero: esplosione combinatoria (con M forme e N piattaforme, necessito di M $*$ N classi concrete) e accoppiamento rigido: un'istanza di V1Rettang non può mai usare P2 a runtime, la piattaforma è decisa staticamente, a compile-time, e quindi non possiamo cambiare implementazione senza cambiare classe.

Esempio:

- In un sistema, occorre avere Rettangoli e Cerchi (astrazioni). Inoltre, occorre che Rettangoli e Cerchi siano disponibili per due piattaforme P1 e P2 (implementazioni).
- La piattaforma P1 mette a disposizione drawL() per disegnare linee, mentre la piattaforma P2 fornisce line().
- Struttura delle classi iniziale (prima del Bridge):



Considerazioni:

- La soluzione appena vista porterebbe a una **proliferazione di classi**.
- Se introducessi un'altra piattaforma P3 (implementazione) avrei bisogno di 6 classi concrete (2 forme * 3 piattaforme). Se introducessi un'altra forma (astrazione), avrei bisogno di 9 classi concrete (3*3).
- Per ogni variazione da introdurre vorrei invece un incremento lineare del numero di classi. Inoltre, presa una classe, per esempio la classe V1Rettang, essa è legata a una certa piattaforma P1 in modo permanente, ovvero una istanza di V1Rettang non può usare piattaforma diversa da P1. Vogliamo cambiare questo comportamento.

Idea del Bridge:

- Invece di fondere le due dimensioni (forme e piattaforme) in un'unica gerarchia, il Bridge le separa in **due gerarchie distinte**, e le collega attraverso un riferimento (la composizione).
- Forma tiene un riferimento a DrawingAPI (interfaccia implementata dalle piattaforme), e così il numero di classi scende da M*N a M+N.
- **Astrazione:** non intendiamo classe astratta nel senso Java. Significa la parte del sistema che definisce **cosa vogliamo fare**, ovvero l'interfaccia ad alto livelli che il client usa. Nel nostro esempio è `draw()`.
- **Implementazione:** non si intende implementazione concreta nel senso Java. Significa la parte che definisce **come farlo a basso livello**, ovvero i "mattoncini" su cui l'astrazione si appoggia. Nel nostro esempio è `drawL()` e `line()`.
- Grazie al Bridge, possiamo:
  1. Aggiungere una forma senza toccare il codice delle piattaforme.
  2. Aggiungere una piattaforma senza toccare il codice delle forme.
  3. **Cambiare implementazione a runtime**, perché il riferimento è un campo dell'oggetto, non qualcosa di intrinseco alla gerarchia. Ciò sarebbe impossibile con l'ereditarietà pura: se abbiamo `V1Rettang`, non possiamo farlo diventare un `V2Rettang` a runtime.

Soluzione:

- **Abstraction**: definisce l'interfaccia per i client e mantiene un **riferimento a un oggetto di tipo Implementor**. Abstraction inoltra le richieste del client al suo oggetto Implementor. Abstraction **non è un'interfaccia**. In sostanza, si occupa di:
  1. Dichiarare operazioni ad alto livello che il Client può chiamare;
  2. Tenere al suo interno un campo di tipo Implementor. Quando `operation()` viene chiamato, internamente delega il lavoro basso-livello a `impl.operationImpl()`.

- **RefinedAbstraction:** estende l'interfaccia definita da Abstraction. Possono aggiungere operazioni specifiche, ma continuano ad appoggiarsi al campo ereditato da Abstraction. Per esempio, Rettangolo può definire un metodo specifico che chiama `impl.operationImpl()` più volte, magari per disegnare i quattro lati.
- **Implementor:** definisce l'interfaccia per le classi dell'implementazione. Questa interfaccia **non deve corrispondere esattamente ad Abstraction**, di solito Implementor **fornisce operazioni primitive**, mentre Abstraction definisce operazioni di più alto livello basate su tali primitive. Implementor **è un'interfaccia**.
- **ConcreteImplementor**: implementa l'interfaccia di Implementor e fornisce le operazioni concrete.

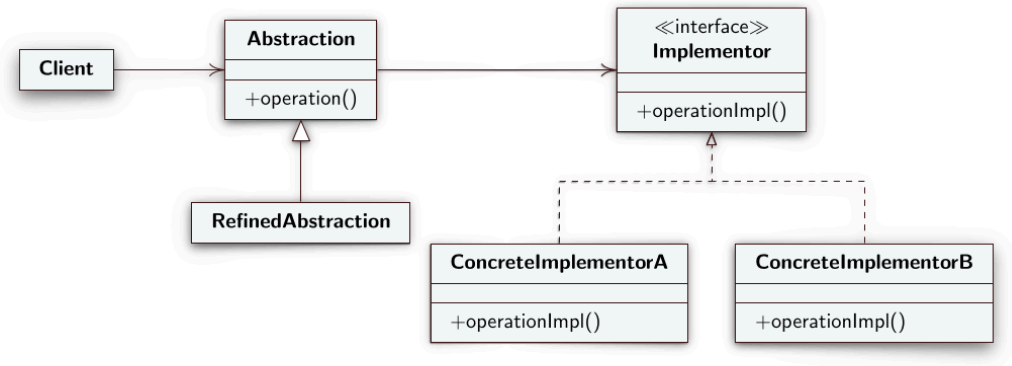


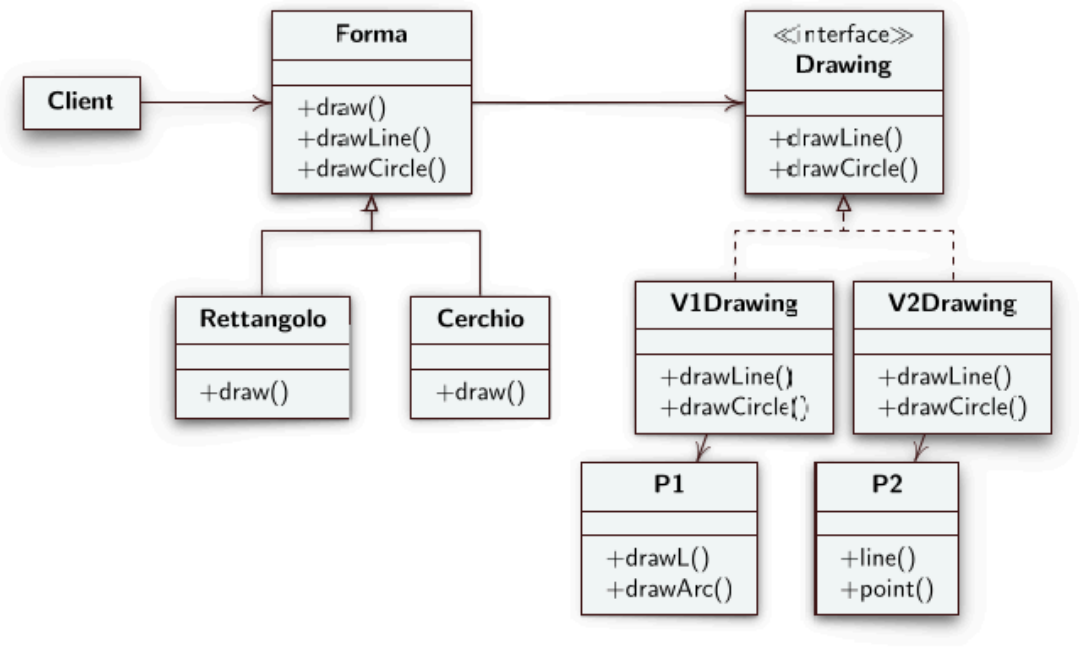
Diagramma UML delle classi del D.P. Bridge

Conseguenze:

- Bridge permette a una implementazione di **non essere connessa permanentemente a una interfaccia**, l'implementazione può essere configurata e anche cambiata a runtime.
- Il disaccoppiamento permette di cambiare l'implementazione **senza dover ricompilare Abstraction e i Client**, in quanto Abstraction dipende dall'interfaccia Implementor, e non dalle classi concrete.

- Solo certi strati del software devono conoscere Abstraction e Implementor. Il client, per esempio, conosce solo Abstraction (non conosce e non deve conoscere Implementor). Ciò favorisce l'**information hiding**.
- Notiamo che a sinistra c'è la gerarchia delle Astrazioni, e a destra la gerarchia delle Implementazioni.
- Le gerarchie di Abstraction e Implementor possono evolvere in modo indipendente. Possiamo infatti aggiungere una nuova RefinedAbstraction senza il bisogno di aggiungere un nuovo ConcreteImplementor, e viceversa.
- Il Bridge è la risposta pratica a una regola generale del design OO: **preferiamo la composizione all'ereditarietà**. Con l'ereditarietà diciamo **B è un A**, mentre con la composizione diciamo che **B ha un A**, e questo lo preferiamo in quanto è una relazione **flessibile**. Le variazioni delle due gerarchie sono **ortogonali**, ovvero completamente indipendenti le une dalle altre. Le variazioni per la parte di Abstraction riguarda le forme, e quindi sono variazioni legate a motivi geometrici/logici. Le variazioni per la parte di Implementor riguardano le piattaforme, che variano per motivi tecnici (es. aggiungo P3 perché ho un nuovo sistema operativo). Questi motivi di variazione non hanno niente a che fare l'uno con l'altro, e per questo si dicono ortogonali.

Struttura delle classi del nostro esempio dopo l'uso del Bridge:



- P1 e P2 rappresentano le **piattaforme già esistenti** (es. librerie di sistema, SDK grafici, etc). Hanno i loro metodi con nomi e firme fissati da altri:

  1. P1 espone drawL() e drawArc();
  2. P2 espone line() e point().

  Non possiamo modificarle: semplicemente esistono, dato che sono piattaforme già esistenti con metodi già pronti (magari scritti persino da noi), e i ConcreteImplementor si appoggiano a questi.

- L'interfaccia Drawing (Implementor) si aspetta drawLine() e drawCircle(). Le piattaforme reali P1 e P2 hanno metodi con nomi diversi, e quindi dobbiamo adattare P1 e P2 all'interfaccia che il Bridge si aspetta.

- V1Drawing e V2Drawing sono i ConcreteImplementor del Bridge, e internamente fanno da **wrapper** verso le piattaforme reali. Sono come degli adattatori delle piattaforme reali. Da fuori (ovvero da Forma) esistono solo V1Drawing e V2Drawing, che parlano il linguaggio di Drawing, e P1 e P2 sono completamente nascosti.

- **Nota importante:** possiamo notare l'utilizzo del Design Pattern **Adapter**. Infatti, V1Drawing svolge il ruolo di Adapter, Drawing il ruolo di Target e P1 il ruolo di

Adaptee. Il client dell'adapter, in questo contesto, è Forma, che vede solo Drawing (cioè Target). Adapter infatti risolve il problema che le piattaforme reali non parlano il linguaggio che il Bridge si aspetta, e quindi bisogna adattarle.

- Usare P1 o P2 internamente è un dettaglio implementativo di V1Drawing e V2Drawing: il Bridge non ne sa nulla e non gli interessa.

D.P. Observer

Design Pattern **comportamentale**, noto anche come **Publisher-Subscribe**.

Intento: definire una **dipendenza uno a molti** fra oggetti, così che quando un oggetto cambia stato tutti i suoi oggetti dipendenti sono notificati e **aggiornati automaticamente**.

Problema:

- Senza Observer, se abbiamo un oggetto Dati e tre viste (Tabella, Grafico, Monitor), dovremmo scrivere ogni volta `tabella.aggiorna()`, `grafico.aggiorna()`, `monitor.aggiorna()`. Dati è strettamente accoppiato a tutte le viste. Se aggiungiamo una quarta vista, dobbiamo modificare Dati. Se rimuoviamo Grafico, bisogna modificare Dati. Le classi **non sono più riusabili indipendentemente**.
- L'intento dell'Observer è quello di risolvere esattamente questo problema: Dati non deve sapere chi dipende da lui. Deve solo sapere che il suo stato è cambiato e **notificare chiunque si sia registrato**.

Motivazioni:

- Un sistema che è stato partizionato in un insieme di classi che cooperano deve mantenere la **consistenza** fra oggetti che hanno relazioni.
- Es. uno strumento di **presentazione** è separato dai **dati** dell'applicazione. Le classi che tengono i dati e quelle di presentazione sono **separate** e riusabili. Uno spreadsheet e un diagramma sono dipendenti dall'oggetto che contiene i dati e dovrebbero essere notificati dei cambiamenti dei dati.

Spiegazioni:

- L'Observer descrive come stabilire relazioni fra oggetti dipendenti.
- Gli oggetti chiave sono **Subject** e **Observer**.
- Un **Subject può avere tanti Observer** che dipendono da esso e gli Observer sono notificati quando lo stato del Subject cambia.
- **Meccanica della registrazione**: il Subject mantiene una lista di Observer registrati. Questo è il motivo per cui si chiama anche **Publisher-Subscribe**. Gli Observer si **iscrivono** (`attach`) al Subject quando vogliono ricevere notifiche. Gli Observer si **disiscrivono** (`detach`) quando non sono più interessati. Quando lo stato cambia, il Subject chiama `notify()` su tutti gli iscritti.
- Il Subject non sa nulla di chi siano i suoi Observer: li tratta tutti come oggetti che implementano l'interfaccia Observer. Questa è la ragione del **disaccoppiamento**.

Applicabilità:

- Un'astrazione ha due aspetti (es. dati e presentazione), ciascuno dipendente dall'altro. **Incapsulare questi aspetti** in oggetti separati permette di **riusarli indipendentemente**. Se mettiamo dati e presentazione insieme, nella stessa classe, allora non potremo avere due presentazioni diverse degli stessi dati, e non potremo riusare il modello dati con un'altra vista.
- **Un cambiamento** su un oggetto **richiede il cambiamento di altri**, non si conosce quanti oggetti sarà necessario cambiare. Si ha **scalabilità a runtime**: potrebbero esserci 1 o 100 Observer. Aggiungiamo e rimuoviamo dipendenze dinamicamente senza toccare il Subject.
- Un oggetto deve notificare altri oggetti senza fare assunzioni su chi sono tali oggetti, quindi gli oggetti non devono essere strettamente accoppiati. Promuove il **disaccoppiamento debole:** possiamo aggiungere un nuovo ConcreteObserver che non esisteva quando avevamo scritto il Subject, e funzionerà benissimo lo stesso.

Soluzione:

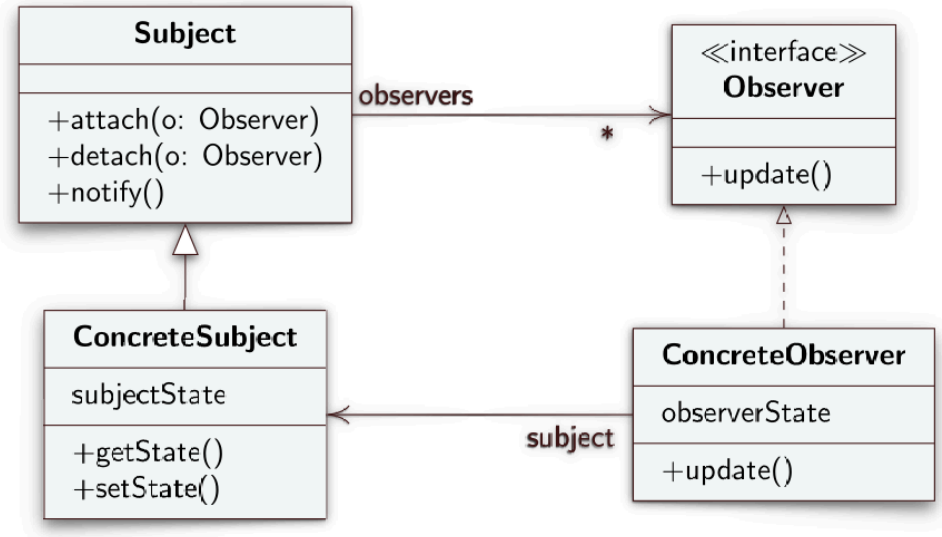
- **Subject:** conosce i suoi osservatori, un qualsiasi numero di oggetti Observer può osservare un Subject. Implementa le operazioni per aggiungere e togliere oggetti Observer e per notificarli.

- **Observer:** definisce una interfaccia (operazione `update()`) comune a tutti gli oggetti che necessitano la notifica.
- **ConcreteSubject:** tiene lo stato che interessa agli oggetti ConcreteObserver. Notifica i suoi osservatori quando il suo stato cambia, grazie all'operazione `notify()` che eredita da Subject. Con notify() il Subject itera sulla sua lista e chiama update() su ciascun Observer. Lo stato del ConcreteSubject viene modificato dal Client attraverso il metodo `setState()`. Invece, `getState()` è quello che gli Observer usano per interrogare il Subject nel modello pull.
- **ConcreteObserver:** tiene un riferimento all'oggetto ConcreteSubject, e tiene lo stato che deve rimanere consistente con quello del Subject. Infatti, per questo motivo ha un campo `observerState` che deve rispecchiare il `subjectState`. È questo il problema di consistenza che il pattern deve risolvere. Implementa Observer per ricevere notifiche dei cambiamenti del Subject. Dopo la notifica può interrogare il ConcreteSubject per ottenere il nuovo dato.

Flusso dell'Observer:

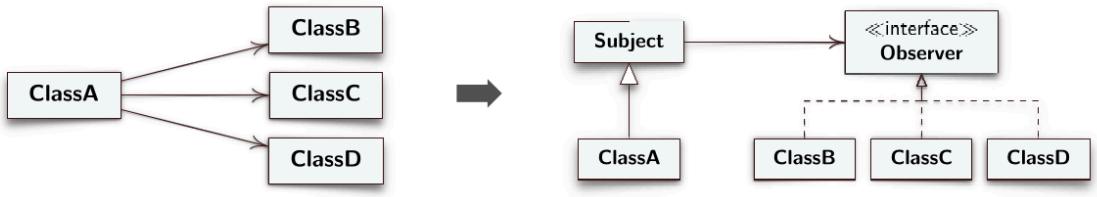
- Quando lo stato dell'oggetto ConcreteSubject cambia, l'oggetto ConcreteSubject chiama notify(), che chiamerà update() sugli oggetti ConcreteObserver per aggiornarli. Chi chiama notify()? Di questo se ne occupa setState() del ConcreteSubject, che chiama notify() internamente.

- Diagramma delle classi



Qual è il **significato dell'asterisco (*)?** L'asterisco in UML indica la **molteplicità**. Ovvero, indica quante istanze di un tipo sono associate a un'istanza dell'altro tipo. Ovvero, un Subject è associato a **zero o più** Observer.

- In generale, la situazione prima e dopo l'uso di Observer è la seguente:



Conseguenze:

- Il Subject conosce solo la classe Observer e non ha bisogno di conoscere le classi ConcreteObserver. ConcreteSubject e ConcreteObserver non sono accoppiati, quindi sono più facili da riusare e modificare. **Loose coupling**. La conseguenza è che possiamo aggiungere un nuovo ConcreteObserver senza toccare una riga di Subject o ConcreteSubject: principio **Open/Closed**.
- La notifica inviata da un ConcreteSubject è mandata a tutti gli oggetti che si sono iscritti, e il ConcreteSubject non sa quanti sono. Gli osservatori possono essere rimossi in qualunque momento. L'osservatore sceglie se gestire o ignorare la notifica. Fenomeno di **broadcast:** la notifica non è indirizzata a nessuno in particolare, ma a tutti gli iscritti.
- **Conseguenza negativa:** l'aggiornamento da parte del Subject può far avviare tante operazioni sugli Observer e altri oggetti per gli aggiornamenti. I cosiddetti **unexpected updates**.
- Nel modello pull, la notifica non dice agli Observer cosa è cambiato nel ConcreteSubject: è un evento che indica il completamento di un'operazione del ConcreteSubject.

Avvertenze:

- Con tanti Subject e pochi Observer, anziché tenere riferimenti a Observer nei Subject, si può usare una sola **tabella associativa** (in comune fra i Subject) per ridurre lo spazio impegnato. Si tratta di **ottimizzazione**.
- Un Observer potrebbe osservare più oggetti, per sapere chi ha mandato la notifica, come parametro di update(), si manda il riferimento al Subject. Quindi si passa `this` come parametro di `update()`.
- Subject chiama **notify()** dopo un cambiamento, oppure **aspetta un certo numero di cambiamenti**, in modo da evitare continue notifiche agli Observer. Quindi c'è l'approccio **automatico**, dove setState() chiama notify internamente a ogni modifica: overhead inutile se facciamo tanti setState di fila. Approccio **manuale** dove il client chiama notify() esplicitamente dopo tutti i setState. Più controllo, ma c'è il rischio di dimenticarsi di farlo.
- Per mantenere la consistenza fra Observer e Subject, un cambiamento dell'Observer deve essere comunicato al Subject (con setState()). Il Subject deve essere in uno **stato consistente prima di chiamare notify()**. Se lo chiamiamo nel mezzo di un aggiornamento parziale, si verificheranno problemi.
- Quando si vuole eliminare un Subject, gli Observer dovrebbero essere avvisati per cancellare il loro riferimento al Subject. Il Subject dovrebbe chiamare `detach()` su tutti i suoi Observer. Prima si distruggono gli Observer, poi il Subject, per evitare problemi di **zombie observer**.
- Il Subject può passare ulteriori informazioni quando chiama update(), e questo è il **modello push**. Nel modello **pull**, aspetta che l'Observer legga lo stato.
- Gli Observer che si registrano possono specificare gli **eventi di interesse**, in modo che con l'update() si manda solo ciò che è cambiato.

Vi sono **due varianti** di questo Design Pattern:

1. **Push model:** il Subject invia i dati aggiornati direttamente nella chiamata a `update()`.
   Pro: gli Observer non devono interrogare il Subject.
   Contro: il Subject deve sapere di quali dati hanno bisogno gli Observer. Il Subject dovrà quindi fare **assunzioni sui loro bisogni**.

2. **Pull model:** il Subject notifica solo che qualcosa è cambiato, e gli Observer interrogano il Subject per ottenere i dati.
   Pro: gli Observer prendono solo quello che serve.
   Contro: richiedono un riferimento al Subject.
   Questo è il modello **più flessibile** e anche il **più usato** nella pratica.

Observer in Java

- Il problema affrontato da questo D.P. è così comune che la sua soluzione è fornita nella libreria `java.util`, con i tipi **Observable** e **Observer**.
- Observable svolge il ruolo di Subject, quindi tiene traccia di tutti gli oggetti ConcreteObserver che vogliono essere informati di un cambiamento. Observable notifica il cambiamento di stato quando il metodo `notifyObservers()` viene chiamato.
- Perché notifyObservers() e non notify()? Perché notify() è già un metodo di `java.lang.Object`, la classe base di tutto. Esiste per **garantire sincronizzazione tra thread**.
- La classe Observable ha una variabile (**flag**) che indica se lo stato è cambiato dalla precedente notifica, ed è impostato dal metodo `setChanged()`. La chiamata a setChanged() è da effettuare all'interno dei metodi della classe ConcreteSubject secondo la logica desiderata. Notiamo che se chiamiamo `notifyObservers()` alla fine del metodo senza controllare se lo stato è davvero cambiato, notifichiamo gli Observer per niente, e quindi `setChanged()` risolve questo problema. `notifyObservers()` non fa nulla se il flag non è impostato.
- Observer è una interfaccia che ha solo il metodo `update()` e può avere un argomento che indica quale oggetto ha causato l'aggiornamento.
- Quando il ConcreteSubject chiama notify(), questo chiama update() che **esegue sul thread del chiamante**, costringendo il chiamante ad aspettare l'esecuzione di update() di ciascun ConcreteObserver.
- Più recentemente, con i **Reactive Streams**, si è cercato di risolvere il problema del passaggio di un insieme di item da un **publisher** a un **subscriber**, **senza bloccare il publisher** e **senza inondare il subscriber**. Quindi i problemi che i Reactive Streams vogliono risolvere sono: publisher troppo veloce e publisher che si blocca.

- Con la versione 9 di Java, Observable e Observer sono disponibili per compatibilità con versioni precedenti, ma sconsigliati.
- Java 9 fornisce nella libreria `java.util.concurrent` le interfacce `Publisher<T>`, `Subscriber<T>`, `Subscription`, e la classe `SubmissionPublisher`. Quest'ultima implementa un Publisher dedicando un thread per mandare ciascun messaggio ai Subscriber.
- **Publisher** è l'astrazione per fornire item, definisce il metodo `subscribe()`, che prende in input `Subscriber<T>`
- **SubmissionPublisher** implementa Publisher, e invia item ai Subscriber in **maniera asincrona**. Quando sul SubmissionPublisher è chiamato il metodo `submit()`, esso esegue in un thread dedicato asincrono la chiamata al metodo `onNext()` dei Subscriber, e si blocca se il subscriber non può ricevere l'item.
- **Subscriber** è usato per ricevere item, e definisce i metodi `onComplete()` (chiamato quando non arrivano più item), `onError()`, `onNext()` (chiamato per ogni item ricevuto), `onSubscribe()`.
- **Subscription** definisce il collegamento fra Publisher e Subscriber, e i metodi `cancel()` e `request(n)` permettono al Subscriber di fermare l'invio di messaggi e di richiedere l'invio dei prossimi n messaggi. Con cancel() il Subscriber dice "non voglio più messaggi", con request(n) dice "sono pronto a ricevere altri n item".

D.P. Model View Controller (MVC)

Intento: separare le **responsabilità** di un'applicazione interattiva in tre componenti distinti: **dati e logica** (Model), **presentazione** (View), **gestione dell'input** (Controller), così che ognuno possa essere modificato, esteso e riusato indipendentemente dagli altri.

È considerato un **pattern architetturale** per le applicazioni interattive, che individua tre componenti: **Model** per funzionalità principali e dati; **View** per mostrare i dati; **Controller** per prendere gli input dell'utente.

Si chiama pattern architetturale perché non risolve un singolo problema di design come gli altri pattern, ma definisce come **organizzare l'intera struttura** di

un’applicazione interattiva. L’idea è tenere separati tre ruoli che in un’applicazione naif finirebbero tutti nella stessa classe.

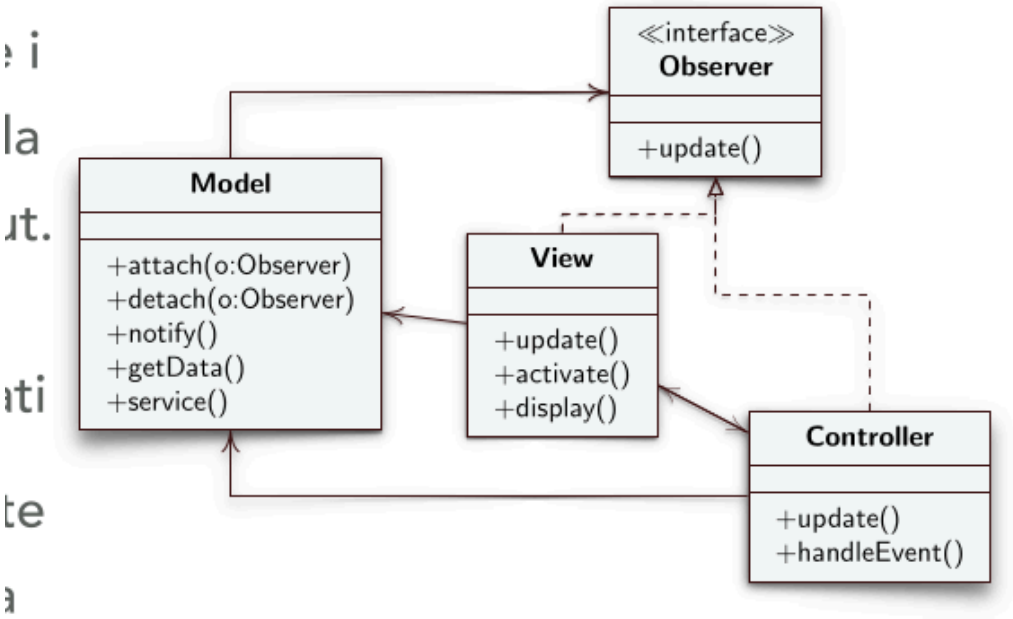
Motivazioni:

- Le interfacce utente possono cambiare, poiché funzioanlità, dispositivi o piattaforme cambiano.
- Le stesse informazioni sono presentate in finestre differenti (per es. sotto forma di grafici diversi).
- Le visualizzazioni devono subito adeguarsi alle manipolazioni sui dati.
- I cambiamenti all’interfaccia utente dovrebbero essere facili.
- Il supporto ai diversi modi di visualizzare non dovrebbe avere a che fare con le funzionalità principali.

Model: incapsula le funzionalità principali e i dati dell’applicazione. È indipendente dalla rappresentazione degli output e dagli input. Registra View e Controller. Avvisa View e Controller registrati dei cambiamenti di dati. **Contiene i dati e la logica di business**, ovvero le operazioni che hanno senso nel dominio del problema. Tuttavia, **non sa nulla di come i dati vengono mostrati**, né di come l’utente interagisce.

View: mostra i dati all’utente. Generalmente ci sono tante View, e ogni View è associata a un Controller. View inizializza il proprio Controller, e mostra i dati che legge da Model. Si occupa **solo di mostrare i dati** all’utente. Nel diagramma, View implementa l’interfaccia Observer, infatti **è un ConcreteObserver**.

Controller: riceve gli input dell’utente (da mouse e tastiera) sotto forma di eventi. Traduce gli eventi in richieste di servizio per Model o avvisa View. Si occupa di **gestire l’input dell’utente**. Traduce gli eventi fisici in operazioni. Anche Controller **è un ConcreteObserver**.



Collegamento con Observer:

- **Model** ha operazioni `attach()`, `detach()`, `notify()`. Rappresenta il **Subject**.
- **View** e **Controller** rappresentano i **ConcreteObserver**.
- Model non sa che esistono View e Controller come tipi concreti: sa solo dell'esistenza di Observer, e li conosce come tali. **Disaccoppiamento**.
- **Dipendenza reciproca ma asimettrica** tra View e Controller: View inizializza il proprio Controller, e quindi crea e tiene un riferimento al suo Controller perché l'utente fa qualcosa sull'interfaccia (click, tasto). È la View che riceve fisicamente l'evento. Non sa come gestirlo, e quindi chiama `handleEvent()`.
  Il Controller, invece, tiene un riferimento a View per poter chiamare `activate()` direttamente su di essa, senza passare per il Model. Serve per comandi UI che non riguardano i dati: mostrare un dialogo di errore, evidenziare un campo, aprire un pannello. Queste sono **operazioni di interfaccia pura** che non hanno senso nel Model. View e Controller **sono una coppia**: ogni View ha esattamente un Controller associato, e viceversa. Si occupano dello stesso pezzo di interfaccia: uno dalla parte dell'input e uno dalla parte dell'output.
- Ogni **pagina** è tipicamente una View. Non esistono View aperte simultaneamente che si aggiornano: ogni richiesta produce una nuova View da

zero. Una View è un **modo distinto di rappresentare i dati**: tabella, grafico, form, pagina. Questa relazione con Observer **ha senso solo se View ha bisogno di dati dinamici**. Se non è così, e la View mostra un contenuto statico (come pagina di login, schermata "about", pagina di errore, etc), allora non ha nessun motivo di fare `attach()` sul Model, non chiama `getData()` e il Model non la notifica. Il pezzo Observer della relazioen non si attiva. Tuttavia, è ancora una View nel senso MVC, dato che ancora mostra qualcosa all'utente. Deve semplicemente **non registrarsi come Observer sul Model**. Tuttavia, comunque useremo l'MVC per **organizzazione generale**, dato che questa rappresenta l'architettura dell'intera applicazione, e non qualcosa che accendiamo o spegniamo per la singola pagina. È la struttura dentro cui tutto vive, anche le parti semplici come quelle statiche.

Flusso completo:

- L'utente clicca qualcosa sulla View.
- View delega al suo Controller tramite `handleEvent()`.
- Il Controller chiama `service()` sul Model per richiedere un'operazione.
- Model aggiorna il suo stato interno, e poi chiama `notify()`.
- A questo punto, `notify()` chiama `update()` su tutti gli Observer registrati (View e Controller).
- View in `update()` chiama `getData()` sul Model per leggere i nuovi dati, poi `display()` per disegnare.

Perché Controller implementa Observer?

- Il Controller non serve solo per passare input al Model. Può anche dover **reagire ai cambiamenti del Model** per aggiustare il suo comportamento.
- **Esempi:** Model entra in uno stato "sola lettura", e il Controller deve sapere che non può più accettare certi input. Oppure Model carica dei dati, e il Controller abilita certi pulsanti che prima erano disabilitati.
- Se il Controller non implementasse Observer, dovrebbe interrogare il Model attivamente o ricevere notifiche in modo diverso.

La separazione di Model, View e Controller serve a rendere le tre parti **modificabili e riusabili indipendentemente**. MVC è Observer applicato su larga scala a

un’intera applicazione.

Esempi reali di MVC:

- Nei **sistemi distribuiti e web**, non c’è l’MVC classico (GUI desktop), dove tutti e tre i componenti vivono sulla stessa macchina. Nel web MVC la separazione diventa anche fisica: Model e Controller risiedono **lato server**, e la View viene renderizzata nel **browser del client**. La comunicazione avviene via HTTP: il Controller riceve richieste HTTP, interroga il Model e restituisce una View (pagina HTML) al browser. **Spring Boot** è il frameword Java più usato per realizzare questa architettura.
- **Model svolge il ruolo di Subject e ConcreteSubject**: è Subject perché gestisce attach, detach, notify, ed è ConcreteSubject perché tiene anche i dati e la logica al suo interno.

D.P. State

Intento: permettere a un oggetto di alterare il suo comportamento quando il suo stato interno cambia. Far sembrare che l’oggetto abbia cambiato la sua classe.

Stream Java

Introduzioni Java:

- Da Java 1.1 si possono usare **classi anonime** per implementare interfacce. Le classi anonime servono quando l’implementazione è usa e getta.
- Da Java 5 è stato introdotto il ciclo for avanzato.
- Da Java 7 si può evitare di indicare il tipo di elementi nella collection al momento dell’istanziazione, lasciando al compilatore l’**inferenza**, ovvero `List<String> nomi = new ArrayList<>();`
- Java è un **linguaggio imperativo**, ma da Java 8 include caratteristiche tipiche della **programmazione funzionale**. Questo tipo di programmazione è in genere **più concisa, espressiva**, e facile da **parallelizzare** rispetto alla programmazione ad oggetti.

- Da Java 8 le espressioni lambda si evitano i preamboli (codice **boilerplate**) e si implementa solo il comportamento, questo però vale solo quando l'interfaccia ha un singolo metodo. Il compilatore capisce che quella espressione lambda serve per implementare il singolo metodo esistente dell'interfaccia.

Espressioni Lambda

Un'espressione Lambda è una **funzione anonima**. I parametri in ingresso sono a sinistra del segno freccia, e il corpo della funzione è a destra. È **simile a un metodo**: ha una lista di parametri, un corpo e un tipo di ritorno. Può essere passata come argomento di un metodo. È **concisa**.

Nei linguaggi funzionali puri, un'espressione lambda è una **funzione pura**, ovvero il risultato **dipende solo dagli input (non ha uno stato)**. In Java, un'espressione lambda può accedere a uno stato esterno (ovvero può non essere pura).

Il tipo dei parametri a sinistra della freccia è determinato automaticamente. Quando non ha parametri o ne ha più di uno, occorre racchiudere i parametri fra parentesi tonde. Il corpo della funzione, nel caso abbia più di un'istruzione, va racchiusa fra parentesi graffe.

Esempio su differenze tra imperativo e dichiarativo:

- Con lo stile imperativo, per fare la ricerca di un nome bisognerebbe scrivere un ciclo for, e salvare se il nome è stato trovato dentro una variabile boolean. Per fare la stessa cosa in stile dichiarativo, mi basta usare il metodo `.contains()` e vedere il risultato.
- L'implementazione **imperativa** dà al programmatore il controllo di quel che il programma deve fare, ma bisogna implementare varie linee di codice, e molto spesso i hanno cicli che scorrono liste per trovare valori, calcolare somme, etc. Per capire cosa vogliono fare quelle righe di codice, bisogna prima leggere tanti dettagli all'interno del corpo del ciclo. Il ciclo è **esterno** al codice che implementa la lista (**l'iterazione è esterna**).
- Nella versione **dichiarativa**, vari dettagli dell'implementazione sono nella libreria sottostante (come col metodo `.contains()` della classe ArrayList). **L'iterazione è interna**.

Stile Funzionale

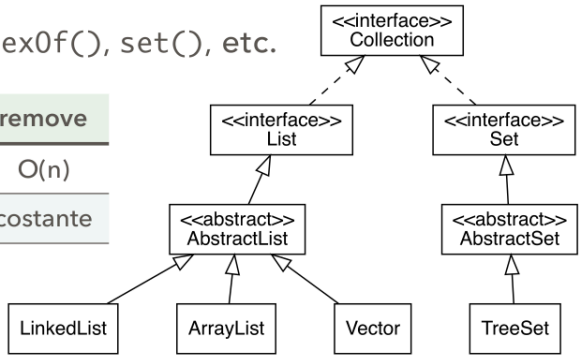
- Lo stile di programmazione funzionale è dichiarativo. La programmazione funzionale aggiunge allo stile dichiarativo **funzioni di ordine più alto**, ovvero funzioni che hanno come parametri altre funzioni.
- In Java si possono passare oggetti ai metodi, creare oggetti dentro i metodi e ritornare oggetti dai metodi.
- Da Java 8 si possono **passare funzioni ai metodi, creare funzioni dentro i metodi e ritornare funzioni dai metodi**.
- Ricordiamo che un metodo è una parte di una classe, mentre una funzione non è associata ad una classe.
- Un metodo o una funzione che ricevono, creano o ritornano una funzione, si considerano essere **funzioni di ordine più alto**.

Java Collection

- Le librerie di Java hanno tante interfacce e classi collection. **Collection** è un'interfaccia che definisce i metodi `add()`, `remove()`, `size()`, `contains()`, `containsAll()`, etc.
- **List** definisce i metodi `get()`, `indexOf()`, `set()`, etc.
- **ArrayList** è un **array espandibile**: cresce del 50% quando non vi è spazio. Gli **elementi sono contigui**, quindi l'accesso al generico elemento è veloce.
- **LinkedList** è una lista, ogni elemento ha i riferimento al successivo e al precedente.
- **Vector** è simile ad ArrayList, ma è **synchronized** (ovvero thread-safe, e quindi più lento).

- `List` definisce i metodi `get()`, `indexOf()`, `set()`, etc.

	get	add	contains	remove
ArrayList	costante	costante	O(n)	O(n)
LinkedList	O(n)	costante	O(n)	costante
TreeSet		O(lg n)	O(lg n)	



Gerarchia di Collection e complessità asintotica delle singole operazioni

Metodi di Default: Stream

- Java 8 introduce i metodi di default per le interfacce. Ciò ha permesso di includere nuovi metodi su interfacce già esistenti, senza compromettere la compatibilità delle aplicazioni conformi a precedenti versioni di Java.
- **stream()** è un **metodo di default** dell'interfaccia **Collection**.
- I metodi di default **non agiscono sullo stato**, invece **le classi astratte possono**, in quanto i loro metodi implementati possono avere costruttori e metodi che agiscono sullo stato.
- `stream()` restituisce uno **Stream<T>** che è una sequenza di elementi di tipo T (T è definito dalla collection su cui è invocato stream()). stream() permette di fare operazioni sugli elementi presenti sull'istanza di collection su cui è invocato.
- Il tipo Stream **definisce vari metodi che prendono in ingresso funzioni**.

Metodo Filter

- `of()` è un metodo statico di List (introdotto da Java 9) che restituisce una **lista non-modificabile** contenente gli elementi passati. Tuttavia, esiste anche il metodo `of()` di Stream, metodo static che restituisce uno Stream.
- `filter(Predicate<T> p)` è un metodo di Stream, **prende come argomento una funzione che ritorna un boolean**. filter() ritorna uno Stream costituito da tutti gli elementi della lista che soddisfano il predicato passato in input.

- `equals()` è un metodo di String che restituisce un boolean.
- **filter()** è **lazy**, ovvero è un'**operazione intermedia**, cioè può essere eseguita in un momento successivo rispetto a dove è inserita, e quindi viene eseguita quando necessario.
- **count()** è un metodo di Stream. È **eager**, ovvero è un'**operazione terminale**, genera un valore e forza l'esecuzione delle precedenti operazioni.
- Tutte le operazioni che restituiscono uno Stream sono **operazioni intermedie**.

Tipo Predicate

- **Predicate:** funzione che ritorna un boolean.
- Rappresenta un'**interfaccia funzionale**, ovvero un'interfaccia che definisce un solo metodo.
- L'annotazione `@FunctionalInterface` indica al compilatore che l'interfaccia ha solo un metodo astratto, così il compilatore controlla che abbia un solo metodo (evita che versioni successive inseriscano altri metodi).
- Predicate definisce il metodo **test()** che prende in ingresso un parametro di tipo **T** (internamente trattato come Object). Il metodo test() non è altro che il metodo definito da Predicate, per esempio: `Predicate<Integer> positive = x -> x >= 0;` ha come metodo test() la funzione $x \geq 0$. Il Predicato positive può essere passato a un metodo che prende in input una Predicate, come per esempio Filter.

Metodo Reduce

- `reduce(T identity, BinaryOperator<T> accumulator)` è un metodo di Stream, prende un valore dello stesso tipo degli elementi dello stream, e un'espressione lambda che ha due valori in ingresso e ritorna un valore.
- reduce() si usa quando si vuol passare da un insieme di valori ad un **singolo valore**, per esempio per ottenere la somma. Reduce applica la **funzione di accumulazione** che gli viene passata (secondo argomento) a tutti gli elementi dello stream e ritorna un risultato dello stesso tipo dei valori nello stream.
- **Esempio:** `reduce(0, (accum, v) -> accum + v);`

- Ciascun valore dello stream è rappresentato dal parametro v. La somma calcolata per ciascun v è tenuta dalla variabile accum e inizialmente vale 0 (valore fornito dal primo parametro di reduce()). reduce() è un'operazione terminale.

Riferimento a Metodi

- La sintassi **::** permette di avere il riferimento a un metodo, che si può quindi passare a un altro metodo.
- Es. **Integer::sum** è il riferimento al metodo statico sum() di Integer, che prende due parametri Integer e ne ritorna la somma.
- Passando il riferimento Integer::sum alla reduce(), verrà calcolata la somma degli elementi dello stream, come quando si passa l'espressione lambda `(accum, v) -> accum + v`. Scriveremo `reduce(0, Integer::sum)`.
- **Variante di reduce()**: può prendere in ingresso solo un `BinaryOperator<T>`, ovvero sia un'espressione lambda che un riferimento a un metodo, e in tal caso restituisce un tipo **Optional<T>** che può contenere il risultato. Scriveremo `reduce(Integer::sum)`.
- Il risultato non esiste se lo stream di partenza è vuoto, e per questo **Optional** rappresenta un valore che **può esistere o meno**, e se esiste, il metodo **isPresent()** darà un valore true. Per estrarre il valore si userà **get()**.

Metodo Map

- `map(Function<T, R> mapper)` di Stream prende in ingresso una funzione mapper, e restituisce uno stream contenente i risultati dell'esecuzione della funzione su ciascun elemento dello stream iniziale.
- Ovvero, **map()** chiama su ciascun elemento dello stream iniziale la funzione passata e dà in uscita il risultato della funzione. Ciascun risultato è inserito in un nuovo stream.
- Map è un'**operazione intermedia**.
- **Esempio:** `map(x -> x*2)` restituisce uno stream con i valori iniziali raddoppiati.

- **Esempio:** `map(Persona::getEta)` eseguito su uno stream di istanze di Persona, restituisce uno stream contenente i valori in uscita da getEta(), quest'ultimo è un metodo di Persona ed è invocato su ciascun elemento dello stream iniziale.

Comparator e Max

- Java sa confrontare numeri interi, ma **non sa confrontare oggetti** (come `Persona`). Per sapere quale tra due oggetti Persona è più grande deve saperlo da noi. Per fare ciò, usiamo un Comparator.
- **Comparator<T>** è un'interfaccia funzionale. Una sua implementazione permette di **stabilire un ordine su una collezione di oggetti**. Il metodo che definisce è `compare(T o1, T o2)`.
- Se o1 == o2, restituisce 0. Se o1 > o2, restituisce un valore positivo.
  Se o1 < o2, restituisce un valore negativo.
- Per confrontare due Persona, quindi, possiamo fare la seguente:
  `Comparator<Persona> myComp = (p1, p2) -> p1.getEta() - p2.getEta();`
  Ovvero sottraiamo le loro età. Il risultato della sottrazione rispetta il contratto di `compare()`, e quindi se p2 ha 25 anni e p1 ne ha 10, la differenza è 15, un numero positivo, e quindi p1 > p2.
- **Max:** metodo che scorre lo stream e usa il Comparator definito da noi per confrontare gli elementi a coppie, tenendo traccia del massimo corrente. È un'**operazione terminale**. Prende un Comparator e restituisce un **Optional**. Max al suo interno chiama il metodo `compare()` del Comparator in input.
- **Alternativa** alla sottrazione per il confronto delle età: possiamo usare `comparing()`, che prende in input il metodo per ottenere il valore da confrontare. In particolare, scriveremo
  `Comparator<Persona> c1 = Comparator.comparing(Persona::getEta)`
  che dice: "costruisci un Comparator che confronta le Persona estaendo l'età di ciascuna e confrontando quella".

Metodo Collect

- `collect()` è un **metodo di Stream** che permette di raggruppare i risultati e prende in ingresso un Collector. Collector è un'interfaccia che descrive **come**

raccogliere gli elementi.

- La classe **Collectors** implementa **metodi utili per raggruppamenti**, il metodo `toList()` restituisce un Collector che accumula elementi in una List.
- Da Java 16 è stato introdotto il metodo Stream.toList().

Programmazione Parallela

- La programmazione parallela è più difficile di quella sequenziale, e usare bene l'hardware risulta più complicato.
- In Java, la classe Thread permette di lanciare un nuovo thread di esecuzione, ma spesso bisogna risolvere i problemi di race condition, usando opportunamente **synchronized, wait, notify**. Inoltre, a partire da Java 5 sono stati messi a disposizione Lock, Esecutori, etc.
- Le operazioni map() e filter() di Stream sono **stateless**, ovvero non tengono uno stato durante l'esecuzione, quindi il risultato non dipende dall'ordine in cui i singoli elementi su cui eseguono vengono prelevati.
- Lo **stream di origine non viene modificato** con le operazioni map(), filter(), etc, ma generano uno stream distinto. Questo **facilita molto la parallelizzazione**.
- **Attenzione:** se durante l'esecuzione di un'operazione, per es. di map(), il codice aggiorna uno stato globale, l'operazione non è più stateless.

Stream Paralleli

- Collection ha i metodi di default **stream()** e **parallelStream()**.
- Il metodo parallelStream() **possibilmente dà uno stream parallelo**. Quindi si può avere l'esecuzione parallela basata su parallelStream(), senza bisogno di thread e sincronizzazione.
- Le prestazioni dipendono da diversi fattori:
  1. Numero di elementi dello stream;
  2. Operazioni da svolgere;
  3. Hardware;

4. Tipo di Collection su cui si invoca l'operazione.

- La Collection potrebbe essere ArrayList, LinkedList, etc. Con ovviamente tempi di accesso diversi.
- Eseguendo su milioni di elementi un codice che conta le persona il cui nome in UpperCase è "NOBI", si vede che le prestazioni migliorano (su hardware con più core) per ArrayList, Vector, TreeSet. Migliorano poco, o non migliorano, quando si usa LinkedList, a causa dell'**accesso sequenziale**. I parallel stream infatti funzionano **dividendo lo stream in chunk**, e assegnano ogni chunk a un thread diverso. Per dividere in modo efficiente, bisogna poter saltare a metà della struttura dati, e per farlo con la LinkedList bisogna **scorrere nodo per nodo**.

Stateless-Stateful

- Le operazioni map e filter sono **stateless**, ovvero non hanno uno stato interno, prendono un elemento dello stream alla volta e forniscono un risultato. Sono parallelizzabili facilmente, dato che ogni elemento viene elaborato **in isolamento**.
- Le operazioni come reduce e max accumulano un risultato. Mantengono uno **stato interno**, ma di **dimensione fissa**. Un accumulatore che viene aggiornato elemento per elemento. Lo stato (l'accumulatore) non cresce con la dimensione dello stream.
- Le operazioni come `sorted()` e `distinct()` sono **stateful**. Infatti, non possiamo ordinare senza aver visto **tutti gli elementi** prima. Non possiamo rimuovere i duplicati senza ricordare **tutti** gli elementi precedenti. Lo stato cresce proporzionalmente al numero di elementi. sorted() e distinct() sono **operazioni intermedie stateful**.
- Vi sono poi operazioni stateless come `findAny()` e `findFirst()` che sono **terminali**, e restituiscono un **Optional**, e per valutarle non è necessario esaminare tutto lo stream, per questo si dicono **short-circuiting** (ovvero può far sì che alcune parti non vengano analizzate).

Generazione di Stream: iterate()

- Gli stream possono essere generati a partire da una funzione tramite le operazioni `iterate()` e `generate()`. Queste operazioni creano stream infiniti.
- **iterate()** restituisce uno stream infinito e ordinato dato dall'esecuzione iterativa di una funzione f applicata inizialmente ad un elemento **seme**, quindi produce uno stream di seme, poi uno stream di f(seme), poi di f(f(seme)) e così via. L'operazione iterate è **sequenziale**, ovvero ogni elemento dipende da quello precedente. Ovviamente non può essere parallelizzato.
- **limit()** tronca lo stream a una lunghezza pari al numero indicato.
- **generate()**: permette di produrre uno stream infinito di valori, tramite una funzione di tipo **Supplier**, ovvero che fornisce un valore. Non applica una funzione a ogni nuovo valore prodotto, come invece fa iterate(). Utile per generare valori casuali, insieme a Math::random.

Debug

- Per esigenze di debug, **potremmo voler conoscere come è fatto lo stream** mano a mano che si eseguono le operazioni. Sarebbe utile capire cosa produce ciascuna operazione. Il metodo **forEach() consuma l'elemento dello stream**, quindi non si può usare.
- Usiamo `peek()`: operazione **intermedia** che prende un `Consumer<T>`, ovvero una funzione che riceve un elemento, fa qualcosa (es. stamparlo), e non restituisce nulla. L'elemento passa avanti nello stream **invariato**. Lo stream **processa un elemento alla volta**, attraverso tutta la pipeline, non un'operazione alla volta. peek è **lazy**: se non c'è un'operazione terminale alla fine, non viene mai eseguito.

IntStream

- IntStream rappresenta uno stream di valori int.
- Su IntStream possiamo usare metodi come **rangeClosed()**, che restituisce una sequenza di int nell'intervallo specificato (estremi inclusi) con incremento pari a 1. Possiamo anche usare **sum()**, che somma i valori presenti nello stream IntStream.
- **mapToInt()** esegue la funzione passata e restituisce un IntStream.

- **boxed()** restituisce uno Stream di Integer a partire da un IntStream. Uno stream di Integer non è altro che uno stream di oggetti. IntStream, invece, lavora direttamente con il primitivo int. Questo metodo ci serve quando abbiamo un IntStream ma abbiamo bisogno di uno `Stream<T>` generico, es. per usare `collect()`.
- **mapToObj()** restituisce uno stream di oggetti a partire da un IntStream. Ovvero restituisce `Stream<T>`

Interfacce Function<T, R> e Supplier<T>

- L'interfaccia funzionale Function<T, R> definisce un metodo chiamato **apply()** che prende in ingresso un oggetto di tipo T e ritorna un tipo R.
- Per esempio: `Function<String, Integer> stringLength = x -> x.length()`
  Qui il metodo apply() prende in ingresso un parametro String, l'implementazione è x.length(), e restituisce il valore dato da length().
- **Supplier<T>** è un'interfaccia funzionale che ha un singolo metodo chiamato **get()**, e rappresenta una funzione che non prende in ingresso alcun parametro e restituisce un valore di tipo T. Anche il metodo generate() di Stream prende in ingresso un Supplier, infatti generate() non ha parametri di ingresso, ma solo un valore di ritorno.

Processi di sviluppo del software

Un **processo software** descrive le **attività** (o task) necessarie allo sviluppo di un prodotto software e come queste attività sono collegate tra loro. Quindi definisce le attività che vanno fatte durante lo sviluppo del prodotto software.

Quali sono queste attività? Ve ne sono diverse. Si parla di attività che vengono fatte per ogni processo di sviluppo software, **alternate in modo diverso e trasversale per ogni processo di sviluppo**. A volte i processi di sviluppo consentono dei cicli, e quindi permettono di ritornare ad attività precedenti.

Attività o fasi dello sviluppo:

- Analisi dei requisiti (specifiche). A volte si parla di documenti, altre volte di qualcosa di più snello/piccolo.

- Progettazione (design). Più dettagliata è, più guidiamo l'implementazione.
- Codifica o implementazione (codice);
- Convalida o testing (approvazione);
- Manutenzione (subito dopo la consegna del prodotto al cliente).

Fase di Analisi dei Requisiti

L'**analisi dei requisiti** è la fase che porta a definire le specifiche, stabilisce i servizi richiesti e i vincoli del software.

- **Requisito:** ciascuna delle caratteristiche che il software deve avere. I requisiti tendono ad essere **granulari** (ovvero molti e piccoli). Descrizione più orientativa rispetto alle specifiche. Il requisito è una signola caratteristica atomica.
- **Specifica:** descrizione dettagliata e precisa delle caratteristiche del software, e quindi di cosa dobbiamo fare. Descrizione formale e precisa di una singola caratteristica atomica.
- **Feature**: un **insieme di requisiti** correlati tra loro. Una feature permette all'utente di soddisfare un obiettivo. Meno precisa dei requisiti, utile a livello di comunicazione col cliente.
- Requisiti, Specifiche e Feature sono **caratteristiche** del sistema software.

Passi per l'**ingegneria dei requisiti:**

1. **Studio di fattibilità:** il software che ci è stato chiesto di sviluppare è un software che possiamo sviluppare o che ci interessa fare? In tempi ragionevoli e costi concordati. Si risponde o sì o no. Va fatto quanto prima possibile. Questa sezione del lavoro, solitamente, non costa denaro a nessuna delle due parti. Poi dipende quanto dura lo studio. Se si risponde di sì, si passa alla fase successiva.
2. **Analisi dei requisiti:** si analizzano i requisiti, esplicitando meglio quello che era rimasto omesso/implicito nella fase di studio di fattibilità.
3. **Specifica dei requisiti:** si analizzano i requisiti in modo più preciso, scrivendo delle vere e proprie specifiche.

4. **Convalida dei requisiti:** si verifica che i requisiti siano **completi** (nessuna lacuna), **consistenti** (non si contraddicono fra loro), **realizzabili** (fattibili con il budget/tecnologia di cui si dispone) e **verificabili** (si può testare che il sistema li soddisfi).

Più il sistema software è critico, più la descrizione dei requisiti e delle specifiche deve essere precisa (e quindi dura di più). **Definizione di critico:** si definisce critico se da esso dipendono delle vite umane. Es. somministrazione endovena di un farmaco al paziente. Sistema software tramite cui si scambiano grandi quantità di denaro è anch'esso critico. Possono anche esserci perdite di vite umane. Oppure, software di guida autonoma.

Esistono **due tipi di Requisiti:**

1. **Funzionali:** ovvero **cosa** il sistema deve fare (funzionalità).
2. **Non-funzionali:** ovvero **come** il sistema lo fa (es. affidabilità, efficienza, prestazioni, manutenibilità, etc). Quindi parliamo di sicurezza, gestione dei dati, velocità di esecuzione (prestazioni), complessità (in base a varie metriche).

Fase di Progettazione

Fase di Progettazione: stabilisce la struttura software che realizza le specifiche.

Attività della progettazione:

- **Suddivisione dei requisiti**, ovvero mappare i requisiti ai sottosistemi/componenti, per capire quale parte del sistema è responsabile di soddisfare un determinato requisito.
- **Identificazione dei sottosistemi**, ovvero progettazione dell'architettura software.
- **Specifica delle responsabilità** dei sottosistemi.
- Progettazione di: interfacce, componenti, strutture dati, algoritmi.

Ognuna delle attività suddette produce un documento corrispondente (o integra un documento già esistente) che descrive un modello: modello degli oggetti, di sequenza, di transizione stati, strutturale, data-flow. Si tratta di **diagrammi UML**.

Vi sono due tipi di progettazione:

1. **Progettazione architetturale:** definisce la struttura ad alto livello, ovvero quali sottosistemi esistono e come comunicano.
2. **Progettazione dettagliata:** scende nel singolo componente, ovvero tratta degli algoritmi interni, di strutture dati, di interfacce precise etc.

Fase di Implementazione

Fase di implementazione: produce un programma eseguibile a partire dalla struttura stabilita.

- Progettazione e implementazione sono **attività correlate e spesso sono alternate**.
- Consiste nella programmazione, ovvero nella traduzione dei modelli della progettazione in un programma (codice) e della rimozione degli errori dal programma.

I programmatori effettuano alcuni test sul programma prodotto per scoprire errori (o bug) e rimuoverli.

Step per il Debugging:

1. **Localizzare** l'errore nel codice.
2. **Rimuovere** l'errore nel modello e successivamente nel codice. Se si corregge solo il codice senza aggiornare il modello, progetto e implementazione diventano inconsistenti. Conviene modificare prima il modello in quanto ci si potrebbe dimenticare di aggiornarlo se modificassimo prima il codice. Non possiamo dimenticarci, invece, di aggiornare il codice, in quanto l'errore ci verrebbe notificato subito dopo l'esecuzione dei test.
3. Effettuare nuovamente i **test** nel programma. Qui si parla di **unit test** (ovvero di test sul singolo modulo/funzione).

Fase di Convalida (Verifica & Validazione) e Testing

- La fase di convalida o Verifica e Validazione (V & V) del sistema software intende mostrare che il sistema software è conforme alle specifiche e che soddisfa le richieste (aspettative) del cliente.
- Vlene condotta tramite processi di revisione e test del sistema software.

- I test mirano ad eseguire il sistema software in condizioni derivate dalle specifiche di dati reali che il sistema software dovrà elaborare.

La **Fase di Test** si articola nei seguenti punti:

- **Test di componenti** o unità **(unit test)**: i singoli componenti sono testati indipendentemente. I **componenti** possono essere funzioni, oggetti o loro raggruppamenti.
- **Test di sistema:** l'intero sistema è testato, dando speciale importanza alle proprietà emergenti.
- **Test di accettazione (alpha test):** test condotti dagli sviluppatori con dati del cliente per verificare che il sistema soddisfi le esigenze del cliente.
- **Beta Test:** test condotti da alcuni clienti sul prodotto quasi completo.
- Prodotti software corrispondenti alle varie fasi di test:
    1. **Versione Alfa:** versione integrata (con difetti e parti mancanti).
    2. **Versione Beta:** versione che include tutte le feature (ma con difetti).
    3. **Versione Gold:** versione finale del prodotto, pronto a essere venduto/commercializzato.

Evoluzione: il software è **intrinsecamente flessibile** e può cambiare. Al cambiare dei requisiti per cambiamenti dell'ambiente a cui è rivolto (business, hardware, etc), il software deve evolvere se deve rimanere ad essere utile.

Lezione 27/05 e 29/05

Si riscontra un difetto (bug) quando il codice che è stato scritto non si comporta come previsto. Come scopriamo se il codice ha un difetto o meno? Tramite il documento dei requisiti (conformità coi requisiti) e il testing.

Fase di Convalida (Verifica & Validazione)

La fase di Convalida o Verifica e Validazione **(V & V)** del sistema mira a valutare se il sistema software si comporta come richiesto, il risultato è quello documentato come corretto e risponde correttamente ai requisiti. Solitamente si usa il termine **Validation.**

Se abbiamo raccolto e soddisfatto tutti i requisiti, ma il cliente ci comunica che il sistema software così non va bene (nonostante i requisiti fossero stati approvati dal cliente), e vuole delle modifiche, come ci si comporta?

Fase di Test

Ci sono vari tipi di test.

Test di sistema: quando ci sono tante classi che devono interagire fra di loro, di cui magari alcune sono state sviluppate da altri programmatori, bisogna scrivere test di ssitema o **di integrazione.** Mirano a testare delle **funzionalità** dette **complessive.** Non si esegue semplicemente il singolo metodo.

NDA: obiettivo non è sapere i dettagli dei dati/servirmi dei dati. Si firmano degli accordi NDA in modo che gli sviluppatori possano testare il sistema software che hanno sviluppato, quindi tramite i dati del cliente, detti **alfa test,** e mirano a testare il sistema con dati reali, per scoprire possibili vulnerabilità, naturalmente senza poi "sfruttare" in altri modi i dati del cliente. Versione prodotta dopo aver superato gli alfa test prende il nome di alfa.

Beta: versione che invece include tutte le feature (ma con difetti). Versione prodotta dopo aver superato i beta test prende il nome di beta.

Naturalmente, il sistema software può subire modifiche anche dopo il rilascio della prima versione, per i motivi più disparati, anche per esempio per una modifica sulla macchina su cui vuole essere fatto girare il sistema, e quindi magari modifiche dovute a motivi hardware.

Processo di sviluppo Waterfall

Tipo di sviluppo illustrato nel 1970, anche se ad oggi non è più uno dei più utilizzati. Royce si era ispirato ad altri processi di sviluppo di sistemi che non sono di software, prima di allora non c'erano processi di sviluppi di software. Si è ispirato ad altri processi di produzione, es. le automobili, dato che già quel tipo di industria era molto organizzata.

1969: coniato il termine **crisi del software.** In quel momento si era assistito, per circa un decennio, a un continuo sviluppo degli hw. Migliorava continuamente le prestazioni etc, notando che ogni 18 mesi, secondo la legge di Moore, le prestazioni dell'hw raddoppiano, e i costi erano sempre minori. Non si riusciva

però a produrre del sw in tempi utili e in modo ragionevole. Si volevano creare sw che sfruttassero al pieno gli hw di quei tempi. Tuttavia, si notò che nonostante si scrivesse più codice, non si riusciva a terminare il software nei tempi previsti (e con i costi previsti). Questo portava a grandi sprechi di denaro.

Sviluppato per questo motivo questo processo di sviluppo. Si chiama a cascata perché ogni fase viene attivata ed eseguita per essere compeltata. Una volta completata uan fase, si va avani alla fase successiva senza però poter tornare indietro. Le fasi che si decidono di realizzare durante il processo di sviluppo a cascata sono:

1. **Requirements definition** (analisi e organizzazione dei requisiti): fai tutta l'analisi all'inizio, e solo quando sei sicuro che tutti i requisiti sono stati ben raccolti e convalidati, su quella fase non ci tornerai più e passerai alla fase successiva. I requisiti raccolti sono fissi, congelati.
2. **System and software design:** parte di progettazione
3. **Implementation and unit testing:** fase di testing
4. **Integration system testing:** quando tutto è stato integrato correttamento, si passa alla fase di manutenzione
5. **Operation and maintenance.**

Poi si riparte allo stesso modo con lo stesso ciclo.

Vantaggi: ogni documentazione che forniamo è ben precisa, consistente, ben fatta. Anche se mi accorgo che ci sono dei requisiti sbagliati, non potrò modificarli, quella documentazione è congelata. Il prodotto software e tutti i documenti a corredo sono tutti consistenti l'uno con l'altro. Quindi i vantaggi sono: consistenza, documentazione e prodotto di alta qualità. Consistenza tra artefatti (**artefatti:** singoli prodotti tra le varie fasi).

Svantaggi: poca flessibilità. Tempi molto lunghi, dato che ogni fase tende a durare tanto. Dato che questo processo di sviluppo richiede di produrre tanta documentazione.

Quando è utile? Quando abbiamo tanto tempo a disposizione. Sufficiente tempo a disposizione. Questo processo dura tanto, circa un anno (o 10 mesi, circa). Se i requisiti sono stabili, è utile. Utile quando dobbiamo assicurarci che il sistema software funzioni sempre. Quindi quando ci sono scenari ad alto rischio, critici: es.

sistema software per un aereo, per una centrale nucleare etc. Quindi tempi di sviluppo lunghi ma c'è la necessità che tutti funzioni perfettamente.

Processi Evolutivi

Se ne delineano soprattutto due:

1. **Per esplorazione:** situazioni in cui, parlando col cliente ci si accorge ceh sono troppi i requisiti e non sono così semplici da comprendere. Bisogna procedere per passi. Si estrae dai requisiti una parte. Si cerca di comprenderli bene, e di farne una progettazione, e poi una codifica. Si fa vedere quella parte al cliente, per avere dei feedback. Se tutto va bene, si può passare a un'altra parte. Quindi uno sviluppo con piccole parti di requisiti e piccole parti del sistema software. E poi si integrano le parti se vanno bene. Per esplorazione perché si ha una visione orientativa su tutto il sistema. Ci si specializza su singole parti. I requisiti diventano chiari a poco a poco, pezzo per pezzo.
2. **Build and fix:** si costruisce un software sapendo già che quello che stiamo scrivendo è sbagliato e dovrà essere aggiustato. Questo a causa del fatto che i requisiti non sono chiari. Questo quando il cliente non riesce a spiegarci cosa vuole esattamente. Solitamente è meglio abbandonare il cliente, ma se proprio dobbiamo farlo, si procede per tentativi. Si prova a interpretarli in un modo, si mostra al cliente la bozza del sistema software sperando che sia accettato dal cliente, per verificare se ci si è capiti. Approccio molto rischioso: si va per le lunghe e non ci si mette mai veramente d'accordo. Meglio usarlo solo per piccole parti del sistema software in cui non si è chiaro quali sono i requisiti. Per l'intero sistema software non conviene come approccio. Si procede per mock.

Problemi: tempi lunghi (anche per l'esplorazione, perché ci vuole tempo per esplorare a poco a poco i requisiti). Progettazione carente, perché si aggiungono pezzi e non c'è un'architettura chiara, non c'è una buona documentazione.

Altri Processi

Processo di sviluppo incrementale: hai uno sviluppo via via sempre più concreto, aggiunti pezzi pian piano. Viene integrato in altri macro-processi.

Processo CBSE (Component-Based Software Engineering): basato su **COTS** (Components Off The Shelf, componenti esistenti). In un'azienda grande con uno storico di sviluppo software, ci sono degli sviluppi in parte simili ad altri, e quindi si può cercare di riutilizzare componenti esistenti. Ovviamente se il cliente vuole qualcosa di più personalizzato, allora non si può riusare così semplicemente il software. Ovviamente per l'azienda è un vantaggio in termini di tempo, in quanto continua a guadagnare su sistemi software che aveva già sviluppato in precedenza. **Costo:** quanto costa il software se lo abbiamo già realizzato? Dipende, a volte anche in una frazione del costo iniziale per lo sviluppo di quel sistema.

Processo a Spirale (Boehm)

Si compiono vari cicli, attraversando le sezioni del piano e si compiono vari cicli. Seguendo la linea circolare, stiamo attraversando più volte, allontanandoci via via dal centro, Ogni volta che attraversiamo un quadrante/settore, facciamo una certa attività. Il processo di sviluppo ci dice di sviluppare in maniera incrementale. Fissiamo quali sono gli obiettivi di questo attraversameto. Si decidono quali sono gli obiettivi: realizzare un prototipo, l'interfaccia verso l'utente, etc.

Altro settore che ci suggerisce di fare una valutazione dei rischi. Quali sono i rischi che possono esserci e che non ti permettono di raggiungere quegli obiettivi.

Esempi: la tecnologia che è in grado di usare per fare quel sistema sw la sapeva usare solo una persona all'interno dell'azienda. Che si fa?
Oppure: libreria sw che stavo usando, ma scopro che ci sono degli errori. Si possono individuare diversi tipi di rischio. Una persona vuole prendere le ferie ma siamo in fase di produzione, etc.

Ultima fase: **revisione.** Ho raggiunto gli obiettivi che mi ero posto?

Per quanto riguarda gli obiettivi non ci dà dei dettagli. Ci si potrebbe dare, come obiettivi, di raccogliere dei requisiti per una parte, etc. Processo detto **metaprocesso,** perché possiamo, nella fase degli obiettivi e di produzione,

Processo Agile: perché possiamo adattarci alle esigenze che vengono fuori man mano. Possiamo eseguire più cicli della spirale. Per completare il sistema sw secondo il processo a spirale, dobbiamo attraversarli più volte. Se lo facciamo una volta sola, è a cascata. Quindi minimo 2.

La fase della valutazione dei rischi, ha senso farla ogni settimana? No. Ogni mese? Nemmeno. I rischi da un mese all'altro (es. persona che va in ferie, o vuole fare corso di aggiornamento etc), non dovrebbero cambiare eccessivamente. Non ha senso pianificare in modo super preciso una settimana o un mese. Ha più senso, invece, fare pianificazioni più a lungo termine. Un ciclo di spirale dura circa 6 mesi. Quindi l'intera fase di sviluppo dura minimo un anno (minimo due cicli). Per lo sviluppo di alcuni SO viene usato questo tipo di processo.

Diagrammi UMLDiagrammi UML dei Casi d'Uso

- Un diagramma dei casi d'uso descrive il comportamento del sistema per come appare dall'esterno, tramite una sequenza di messaggi fra sistema e attori.
- L'**omino** rappresentato si chiama **attore**, ovvero un utente o sistema esterno che interagisce col prodotto da sviluppare con un certo ruolo. Si disegna come una persona col nome scritto sotto.
- Le frecce che partono dall'omino verso le varie funzionalità indicano che può agire su tali funzionalità. Bisogna però **limitarsi a un certo livello di descrizione**, e non bisogna espandere troppo le funzionalità nel diagramma dei casi d'uso. Si devono individuare le **funzionalità significative** del sistema per gli attori.
- Un **caso d'uso** è disegnato come un'**ellissi** con il nome scritto dentro (tipicamente 1/2 parole), e indica una funzionalità offerta dal sistema. Non riguarda il comportamento interno o la sua struttura intenra.
- In un caso d'uso un attore sta cercando di ottenere un certo obiettivo.
- Un caso d'uso può avere vari scenari, e ognuno mostra un percorso alternativo.
- Solitamente in questo diagramma è presente un **rettangolo**: si occupa di **delimitare il confine del sistema**, e infatti gli attori stanno fuori mentre i casi d'uso stanno dentro.
- Una **relazione** (detta associazione) fra attore e caso d'uso è indicata con una linea che congiunge attore e caso d'uso. Vi sono vari tipi di relazioni:

1. **Generalizzazione fra attori:** l'attore più specifico eredita la capacità di quello più generale e ne aggiunge altre.
2. **Generalizzazione fra casi d'uso:** il caso d'uso più specifico eredita da uno più generale e aggiunge caratteristiche.
3. **Estensione fra casi d'uso:** un caso d'uso di base usa un altro caso d'uso come comportamento incrementale opzionale.
4. **Inclusione fra casi d'uso:** un caso d'uso incorpora il comportamento di un altro.

- È importante fare la distinzione fra `<<include>>` e `<<extends>>`. Le frecce tratteggiate possono puntare in due versi opposti, e questo fa la differenza.
  1. **Include:** il caso base chiama un altro caso. La freccia parte dal caso includente e punta al caso incluso.
  2. **Extends:** un caso aggiunge comportamento opzionale a un altro. La freccia parte dal caso estendente e punta al caso base.
- Bisogna limitarsi a un **certo livello di descrizione**. Non bisogna espandere troppo le funzionalità nel diagramma dei casi d'uso. Esistono tre livelli:
  1. **Kite level:** acquilone. Stiamo guardando il sistema da una decina di metri. Quindi non troviamo dettagli, anzi ne troviamo pochi. Casi d'uso molto astratti. Obiettivi di business di alto livello.
  2. **Sea level:** iniziamo a dare più dettagli. Es. l'utente può fare l'ordine, ma per farlo deve selezionare i prodotti, e si devono anche fornire dei dettagli. Giusto compromesso. Siamo al livello del mare. Livello di dettagli medio, è il più consigliato. È il livello dell'utente, quello raccomandato per la maggior parte dei diagrammi.
  3. **Fish level:** siamo sotto al livello del mare, in mezzo ai pesci. Stiamo dando troppi dettagli. Diagramma enorme. Troppo granulare.
- Diagramma molto utile per facilitare la comunicazione col cliente.

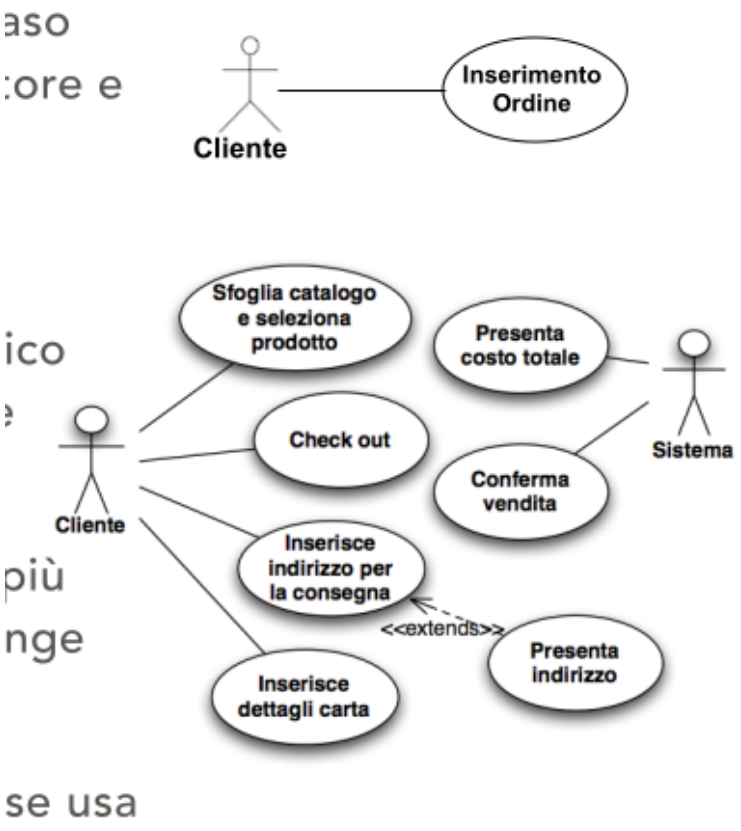


Diagramma UML delle attività

- I diagrammi UML delle attività sono il **punto iniziale della progettazione**.
- Sono usati per riflettere sulle attività che il sistema software dovrà realizzare.
- Possono essere usati come punto di partenza per ottenere diagrammi UML di collaborazione fra oggetti.
- Non mostrano quali oggetti svolgono le attività.
- In un sistema di gestione ordini, per esempio, i punti di avvio possono essere diversi. Es. alcuni per fornitore e altri per utente.
- **Rettangoli con lati piccoli arrotondati:** passi di elaborazione.
- **Frecce continue**: flussi.
- **Rombi:** ramificazioni condizionali o merge.

- **Cerchio pieno:** stato iniziale. Per indicare l'attività conclusiva si può mettere una freccia verso un cerchio che racchiude un cerchio annerito. In generale, non è necessario mettere il cerchio dell'attività finale, dato che si capisce, visto che si trova un rettangolo senza frecce uscenti.

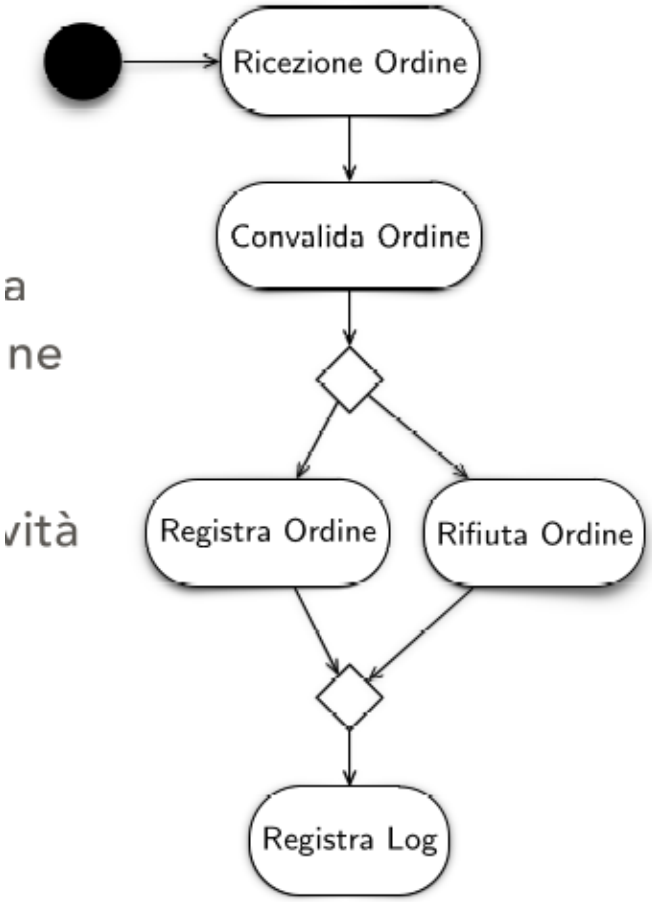


Diagramma con Flussi Paralleli

- Le **barre nere** indicano **punti di parallelizzazione** (una freccia entrante e due uscenti) o **di sincronizzazione** (due frecce entranti e una uscente, quindi join).
- Il cerchio pieno con circonferenza attorno indica lo stato finale.
- Ovviamente si possono avere più di due frecce uscenti (ovvero più di due flussi paralleli).

- Diagramma utile per facilitare la comunicazione con in cliente.

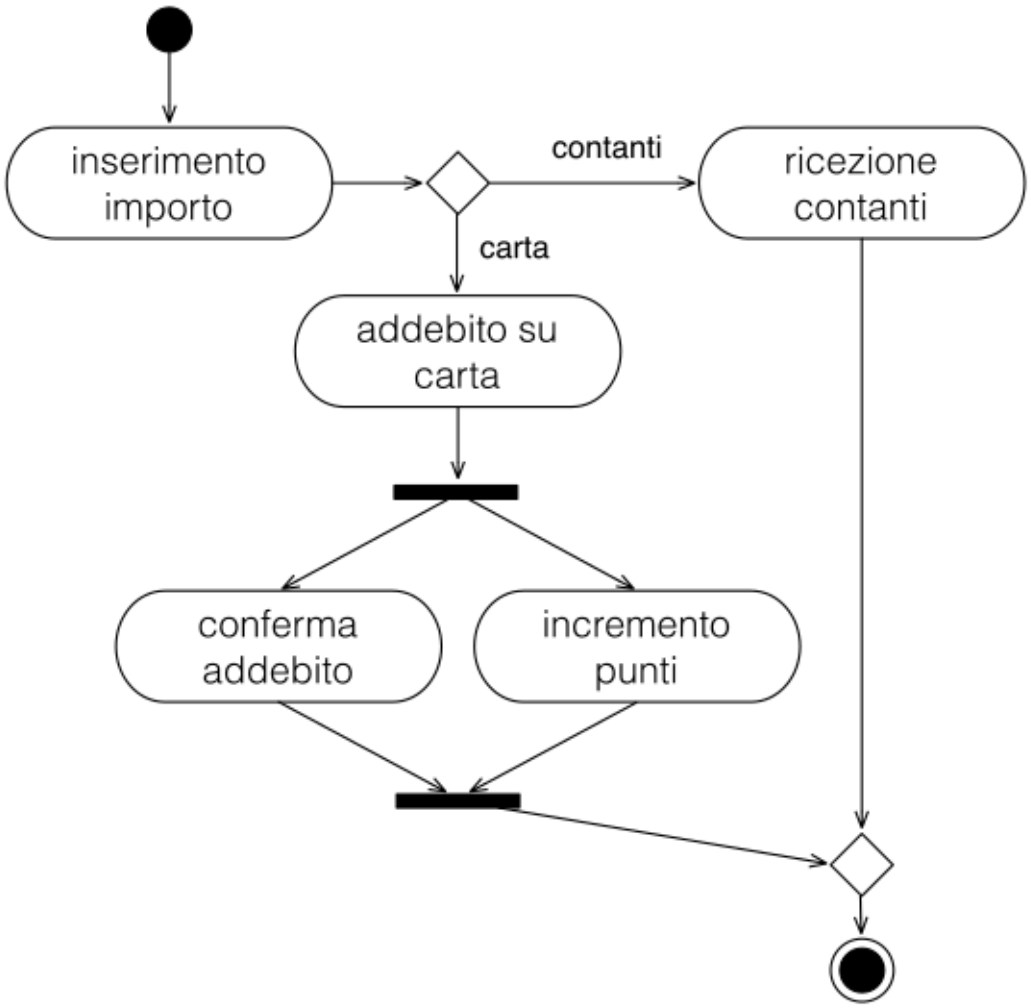


Diagramma con Corsie

- Le corsie sono **partizioni che indicano il responsabile** (attore o modulo) delle attività nella corsia.
- I **rettangoli** sono oggetti o dati.
- I **rettangoli arrotondati** indicano attività/operazioni ad alto livello del sistema.
- Le **frecce tratteggiate** indicano dati in input o output per un'attività.

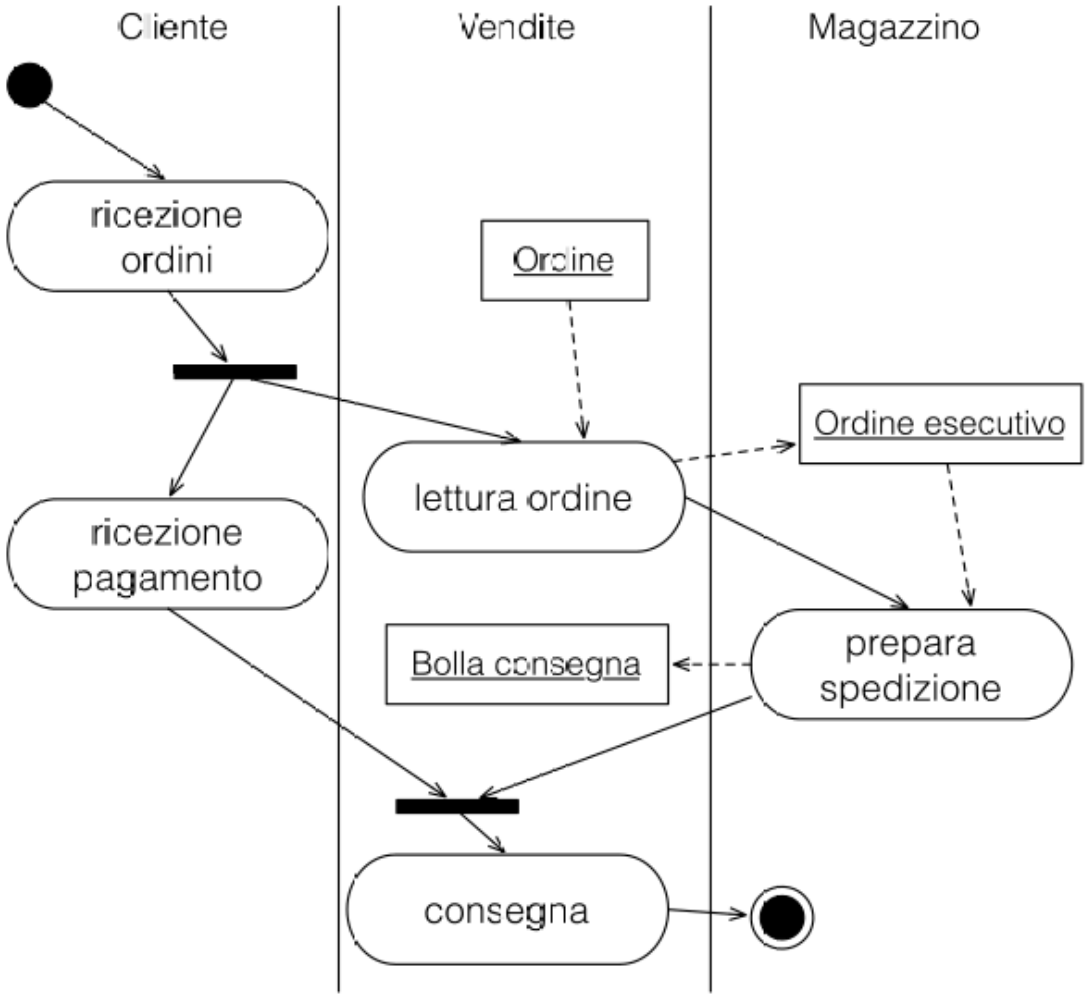
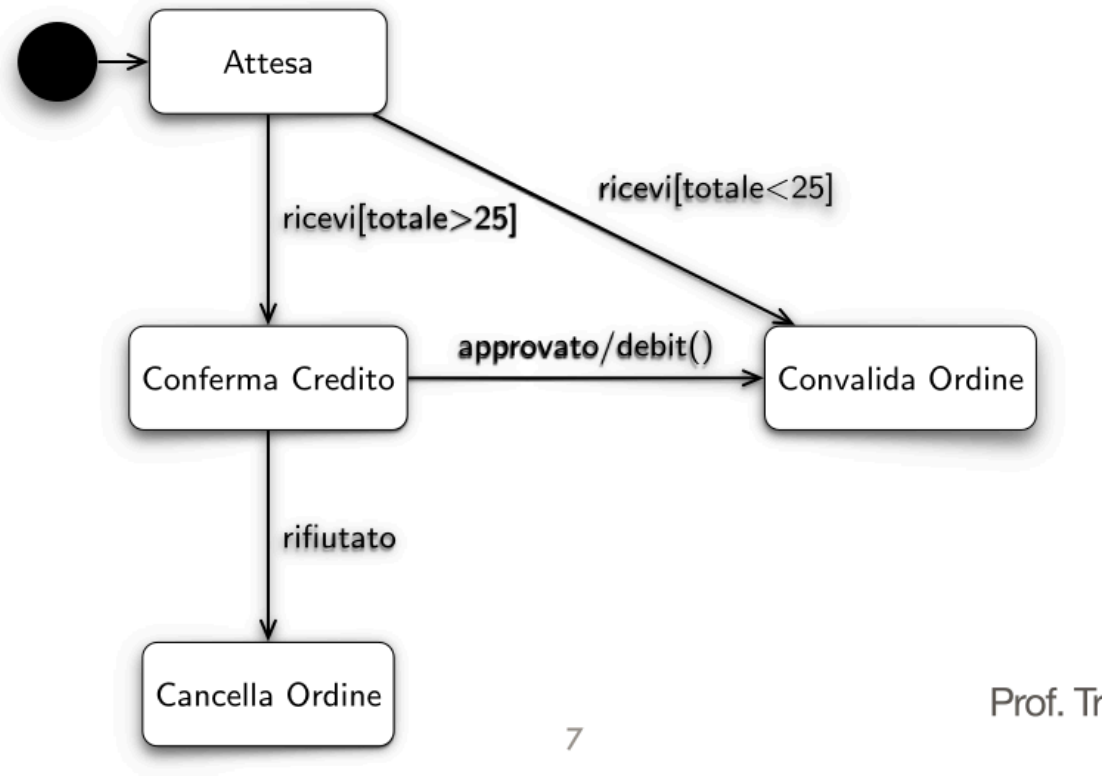


Diagramma degli Stati

- Lo stato descrive un **intervallo di tempo durante la vita di un oggetto**, ed è caratterizzato da valori o intervallo di tempo durante il quale si aspettano certi eventi o si fanno certe azioni.
- Una **transizione** permette di lasciare uno stato in risposta a un certo evento. Una transizione è caratterizzata da: un **evento**, una **possibile condizione**, un'**azione** e uno **stato di arrivo**.
- Gli stati vengono rappresentati come **rettangoli arrotondati**. Diverso dal rettangolo coi lati piccoli arrotondati (attività). La freccia, invece, rappresenta

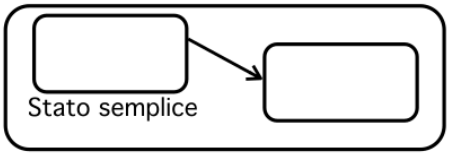
un evento.



- Il **flusso** è il seguente: se avviene evento e1 (scritto sopra la freccia con la punta né chiusa né piena), allora passo da uno stato all'altro. Dallo stato in cui mi trovo posso passare solo agli stati che sono collegati ad esso con delle frecce. Se avviene l'evento e2 ma questo evento è sopra una freccia che non riguarda lo stato in cui siamo ora, allora non accade niente.
  Se voglio esprimere meglio l'evento e1, allora posso scrivere tra parentesi quadre qual è la **condizione espressa**. Es. e1 [x > 10]. Se voglio specificare l'attività che voglio compiere, scrivo / metodo() o / descrizione evento. Quindi o un commento veloce o un metodo.
- Lo stato iniziale è rappresentato da un cerchio nero.
- Nel diagramma degli stati, vi possono essere degli stati composti, ovvero uno stato che consiste di vari sottostati sequenziali o concorrenti. Uno stato semplice non ha sottostati. Solo uno dei sottostati sequenziali può essere

attivo in un certo momento. Lo stato esterno rappresenta la condizione di essere in uno qualsiasi dei sottostati interni.

- **Modo 1:** all'interno del rettangolo degli strati composti vi è un cerchietto nero con freccia, che indica il primo degli stati che viene eseguito. Lo stato interno finale dei rettangoli si rappresenta tramite una freccia verso un cerchietto nero.
  Modo 2: altro modo per rappresentare stato iniziale e stato finale: far puntare la linea che fa iniziare lo stato composto direttamente allo stato iniziale, e la linea che esce dallo stato composto farla iniziare dallo stato finale. Così evitiamo i cerchietti e si capisce quali sono lo stato iniziale e finale.
- Una transizione verso o da un altro stato composto può invocare varie azioni di entry o di exit.
- La presenza di stati composti non è così ricorrente in un sistema software.
- **Stati interni paralleli:** linea tratteggiata orizzontale nel rettangolo, indica che gli stati interni sono paralleli. Ovvero gli stati sopra la freccia e quelli sotto sono paralleli. Se esco dallo stato composto concorrente, allora usciamo da tutti i suoi stati interni. Nemmeno gli altri paralleli continuano.
- Se divido lo stato con una linea orizzontale, in alto metto il nome e sotto metto una descrizione delle attività che avvengono. Si usano le parole chiave **entry**: op1(), ovvero l'operazone che compio quando entro nello stato, ed **exit**: op3(), ovvero l'operazione che compio quando esco dallo stato. E quando sono dentro lo stato, né all'inizio né alla fine, allora uso **doing**: op2(). Quando entro ed esco dal diagramma devo fare per forza op1 e op3. Le operazioni doing, però, potrei anche non farle. Magari esco prima.



Stato composto sequenziale

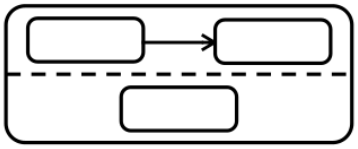
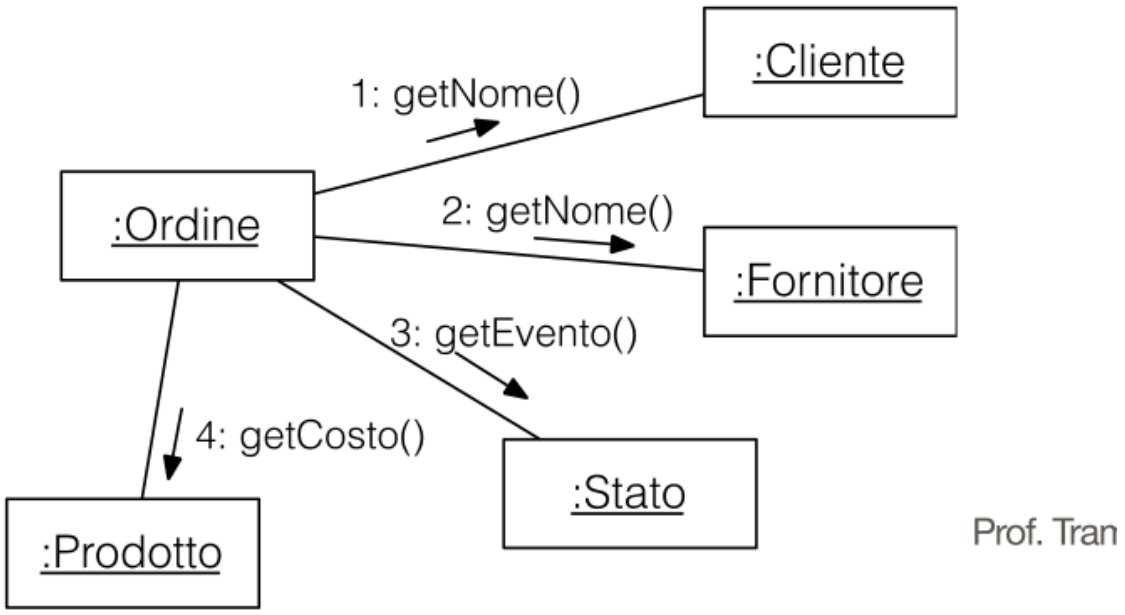

Stato composto concorrente

Diagramma di Collaborazione

- Mostra le **interazioni** fra oggetti.
- Il **flusso di messaggi** è indicato da frecce accanto alle associazioni fra le istanze che partecipano all'interazione.
- Gli oggetti sono rappresentati come **rettangoli**, con la notazione `nomeIstanza:nomeClasse` sottolineata. Se il nome dell'istanza non importa, si scrive solo `:nomeClasse`.
- L'**interazione** con un'istanza è espressa tramite una **freccia**.
- I **messaggi** sono indicati da etichette sulle frecce e hanno:
  1. Un **numero sequenziale** che indica l'ordine con cui avvengono. La numerazione può essere **gerarchica** per indicare chiamate annidate.
  2. Il metodo chiamato.
  3. Se occorre, un **valore di ritorno**.
- Il diagramma di collaborazione **mostra le stesse informazioni** del diagramma di sequenza, ma con una prospettiva diversa. Con il diagramma di sequenza l'**ordine temporale è immediato**, mentre con il diagramma di collaborazione vediamo **chi è collegato a chi**, quindi per capire quali oggetti comunicano fra loro. Si usa di più il diagramma di sequenza perché è **più leggibile**.



Sintesi degli scopi dei diagrammi UML

- **Casi d'uso:** si occupa di spiegare **Chi fa Cosa** con il sistema. Utile per parlare con il cliente e definire i requisiti.
- **Attività:** utile per capire **Come si svolge un processo passo per passo**. Visione del flusso interno. Sequenza di operazioni con ramificazioni e decisioni.
- **Flussi paralleli:** è una **variante dell'attività**, dove alcune cose accadono contemporaneamente.
- **Corsie:** è una **variante dell'attività**, dove si specifica anche **Chi fa ogni passo**. Quindi la differenza è che ci sono le colonne che separano le responsabilità.
- **Stati:** si occupa di far vedere **come cambia lo Stato di un oggetto nel tempo**. Non parla di processi, ma parla di un singolo oggetto, facendo vedere come evolve.
- **Collaborazione:** si occupa di rappresentare **Chi manda Quale messaggio a Chi, e in che ordine**. Quindi mostra le interazioni tra oggetti durante l'esecuzione di un caso d'uso.

Tecniche per Individuare le Classi

- Partire dal testo che descrive i requisiti ed effettuare l'**analisi grammaticale**.
- I **sostantivi** suggeriscono le classi o gli attributi.
- I **verbi** suggeriscono le operazioni.
- Fare attenzione al raggruppamento di attributi e operazioni nelle classi, ricordando che si vuole ottenere **coesione e singola responsabilità (KISS, SRP)**.
- Sul testo dei requisiti si possono individuare gli **oggetti fisici** menzionati per individuare le classi.
- Nella **Progettazione di classi**, potrebbe avere molto senso colorare le parole che compaiono nei requisiti con colori diversi per indicare a quale categoria appartengono. Per esempio, alle Classi potremmo associare il verde, agli attributi il marrone, e ai metodi l'arancione.

Lazy initialization vs eager initialization

- **Lazy (come `Logs`):** l'istanza viene creata solo quando viene richiesta per la prima volta (`if (null == obj) obj = new Logs()`). Non è un difetto — può essere una buona scelta quando la creazione è costosa e l'istanza potrebbe non servire sempre.
- **Eager (come `Fib`):** l'istanza viene creata subito alla dichiarazione dell'attributo statico (`private static Fib obj = new Fib()`).

Nota sulla concorrenza: la lazy initialization va bene solo in contesti **single-thread**. Con più thread in esecuzione contemporanea, possono crearsi condizioni di gara che portano alla creazione di più istanze. In quel caso bisogna usare `synchronized` sul metodo `getInstance()` per implementare un semaforo. I DP presentati nel corso sono pensati per sistemi a singolo thread centralizzato.

Variante: Multiton

Se all'interno del metodo statico si mette la condizione "se non hai raggiunto un certo numero di istanze, puoi continuare a crearne", allora si parla di **Multiton**. Il

codice delle classi chiamanti rimarrebbe invariato; si modifica solo la classe `Pagamenti`. Una variante Multiton potrebbe associare un'istanza a ciascuna cartella. **Principio delle Conseguenze Locali:** un cambiamento in qualche punto del codice non dovrebbe causare problemi in altri punti.

Conseguenze del Singleton

- Ha pieno controllo di come e quando i client accedono all'istanza.
- Evita variabili globali che tengono l'istanza condivisa.
- Permette di controllare il numero di istanze facilmente e in un solo punto.
- È più flessibile di usare `static` per tutto: con `static` si dovrebbero modificare tutte le invocazioni se si vuole cambiare il numero di istanze consentite; con Singleton si modifica solo la classe Singleton.

Non si può simulare questo comportamento senza metodi/variabili static e senza costruttore privato.

Conseguenze

- Il codice delle classi dell'applicazione conosce solo l'interfaccia Product e può lavorare con qualsiasi ConcreteProduct: i ConcreteProduct sono facilmente **intercambiabili**.
- Se si implementa una sottoclasse di Creator per ciascun ConcreteProduct, si ha una **proliferazione di classi**.
- Possono esserci più classi Concrete se i ConcreteProduct sono di tipo molto diverso.

Object Pool

Una variante del Factory Method. Un **object pool** è un deposito di istanze già create da cui un client estrae un'istanza quando ne ha bisogno, e la restituisce al pool quando ha finito.

- Il pool può crescere o avere **dimensioni fisse**. Con dimensioni fisse: se non ci sono oggetti disponibili al momento della richiesta, non se ne creano di nuovi.

- Tutte le creazioni devono passare dal factory method, con un controllo sulla dimensione massima.
- **Dove inizializzare gli oggetti del pool?** Dipende: se è necessario creare tutte le istanze all'inizio dell'applicazione, si fa nel costruttore (eager); se non è necessario, si crea on-demand (lazy, come la lazy initialization del Singleton).
- Il client restituisce al pool l'istanza usata quando non è più utile (`releaseShape(s)`).
- L'object pool dovrebbe essere unico → si può combinare con il Singleton.

java

```
public class CreatorPool {
    private List<Shape> pool = new ArrayList<>();
    public Shape getShape() {           // factory method
        if (pool.size() > 0) return pool.remove(0);
        return new Circle();            // crea on-demand
    }
    public void releaseShape(Shape s) { pool.add(s); } // restituzione
}
```

Dependency Injection

Il Factory Method può essere usato per implementare la **Dependency Injection**: un oggetto (client) riceve altri oggetti da cui dipende (le "dipendenze"), invece di crearli da solo.

- Il client **non crea** l'istanza di cui ha bisogno.
- Le dipendenze vengono **iniettate** al client tramite il costruttore (o parametri). Questo evita metodi setter e controlli per verificare che le dipendenze non siano null, rendendo il codice più semplice.
- L'oggetto che fa Dependency Injection si occupa di **fare wiring** (collegare) le varie istanze: in un unico posto si vedono tutte le connessioni tra gli oggetti.

Esempio: se `Studente` ha bisogno di `PianoStudi`, invece di creare `PianoStudi` al suo interno, lo riceve come parametro nel costruttore.

ConcreteFactory si può usare anche per la Dependency Injection — ma solo se si tratta di Product. Se ci si riferisce ad altre parti del sistema non correlate, non è appropriato.

Conseguenze

- Permette di usare classi **consistentemente** (per famiglie): se si usa `CreatorEng`, si ottengono sempre componenti in stile inglese.
- Le famiglie di classi sono facilmente **intercambiabili**: basta passare un `Creator` diverso.
- **Non è immediato** supportare nuove classi Product: bisogna aggiungere un metodo su AbstractFactory e su ogni ConcreteFactory.

Nota: i nomi potevano essere scelti meglio (detto dagli stessi autori). Quello presentato è un caso particolare. Non bisogna usare il casting all'interno di questo DP: se si ha bisogno di tipi derivati specifici, va usato un pattern diverso o si fa affidamento sul polimorfismo.

Note sulle correzioni rispetto alle versioni precedenti

JUnit 6: le slide del corso indicano JUnit 6 come versione più recente (con versione 6.0.3 nel pom.xml). Aggiornato di conseguenza.

`@BeforeEach` **vs** `@BeforeAll`**:** `@BeforeEach` viene eseguita prima di **ogni** singolo test (garantisce uno stato fresco); `@BeforeAll` viene eseguita **una sola volta** per tutta la classe di test (per inizializzazioni costose). Distinzione importante.

Lazy initialization nel Singleton: non è un errore, può essere una buona scelta — dipende dal contesto. Il problema emerge con la concorrenza.

Abstract Factory vs Factory Method: Abstract Factory gestisce famiglie di prodotti correlati (più gerarchie). Factory Method gestisce la creazione di un singolo tipo di prodotto.

LoD: il problema è quando **una** classe dipende da troppe altre, non quando **tante** classi dipendono da una classe centrale.

Polimorfismo e dispatch: il meccanismo di dispatch parte sempre dall'istanza a runtime, non dal tipo dichiarato della variabile.

Lezione 24/04

Prototype: design pattern creazionale.

Mentre F.M. cip orta a scrivere piu codice per distinguere Creator, ConcreteCreator, etc. Il prototype si comporta in modo diverso. Nel prototype possiamo evitare di scrivere il codice Creator e ConcreteCreator.

Serve per creare una copia dell'istanza. Invece di creare nuovi oggetti da zero (con l'operatore new), si creano nuove istanze **copiando** un oggetto esistente, che chiamermeo prototipo. Quando creiamo una copia dell'istanza, tutti i valori della vecchia istanza li mettiamo nella nuova istanza.

Lo scopo del prototype è risolvere situazioni in cui la creazione diretta di un oggetto è costosa, complessa o vincolata.

- **Prototype**: definisce l'interfaccia per le operazioni comuni e dichiara un metodo per clonare sè stessa.
- **ConcretePrototype**: è una classe che implementa l'operazione di clonazione. Ci saranno varie classi con questo nome (1,2,etc), tutte che implementano le operazioni nell'interfaccia.

- **Client**: classe che crea nuovi oggetti tramite clonazione dell'istanza prototipo, quindi senza indicare la classe esplicitamente. Client conosce solo l'interfaccia Prototype.

Internamente, usiamo new.

Due possibili tipi di copie: **shallow copy** e **deep copy.** La shallow copy copia l'oggetto solo superficialmente, ma se tra gli attributi dell'oggetto vi sono dei riferimenti ad altre variabili/oggetti, allora questi verranno copiati in modo superficiali, e saranno condivisi fra i due oggetti. Con la deep copy, invece, copiamo tutto, non solo i valori di tipo primitivo. Creiamo infatti nuove istanze anche dei riferimenti. Quindi se c'era un riferimento a un oggetto questo non verrà copiato in modo superficiale, ma verrà creato un nuovo oggetto dove tutti i riferimenti sono diversi, quindi gli oggetti punteranno a due oggetti uguali ma che però non sono lo stesso oggetto perché non condividono la locazione di memoria. Questi due tipi di copie devono essere implementati da noi all'interno di clone.

addAll: crea una copia per ogni elemento passato. Da vedere per slide Prototype Esempio.

Ovviamente, per come è strutturato il problema, bisogna implementare una clone per ogni ConcretePrototype, dato che per ogni ConcretePrototype ognuna ha i suoi attributi, probabilmente diversi dagli altri.

Prototype: Conseguenze

Il vantaggio del D.P. è che non abbiamo bisogno di creare una struttura apposita Creator/ConcreteCreator.

Se all'inizio non ho nessun istanza, come chiamo il metodo clone? Dove prendo la prima istanza? Qui nasce l'esigenza di un manager di Prototype. Ci serve per farci dare la prima istanza.

Ci serve F.M. o Prototype? Dipende dalle circostanze. Dipende se mi serve una copia oppure creare una nuova istanza con nuovi valori.

Metodo main: dependency injection, inietta le istanze con cui lavorare (in questo caso Calendario e Notebook).

Design Pattern Adapter

Adapter permette ad alcune classi di interagire, eliminando il problema di interfacce incompatibili.

Attenzione: spesso non intendiamo interfaccia come keyword di Java. Con interfaccia intendiamo semplicemente il punto di aggancio per interagire con altre classi.

Problema: supponiamo di avere delle classi che sono state costruite per interagire con altre classi, supponiamo scritte da un altro team. Supponiamo che inizialmente pensavamo che quelle classi avrebbe avuto metodi con nome X, Y, Z. Quei metodi si scopre che ci sono ma hanno dei nomi diversi. Tuttavia, le classi che abbiamo creato interagiscono con metodi che però non si chiamano così. Oppure magari il tipo del parametro di un metodo non corrisponde: ci aspettavamo un int ma in realtà è uno string. Che si fa in questi casi? Quando si è avanti nell'implementazione, è bene non modificare il codice. Cambiare il codice quando ormai è già denso e siamo avanti nel progetto, non conviene fare questa scelta. Pensiamo a una cosa: l'altro team che ha cambiato il nome previsto per il metodo oppure ha cambiato il tipo per un parametro di un metodo, probabilmente avrà dei buoni motivi per averlo fatto. Quindi è possibile adattare il nostro codice al loro. Loro non hanno scritto dei metodi "sbagliati", ma si sono accorti che forse era meglio chiamare alcuni metodi in altri modi e cambiare alcuni tipi.

I metodi che voleva usare il client, sono definiti all'interno di Target. Probabilmente il client per testare la sua libreria si era creato già un'interfaccia taget anche per testarla.

Adaptee: oggetto di libreria che necessita l'adattamento.

Adapter: classe implementata per eliminare l'ambiguità, tiene al suo interno un'istanza di Adaptee. Rimuove l'incompatibilità includendo una chiamata col nome giusto. Semplicemente una chiamata di metodo, nulla di complesso.

E se dopo aver curato l'adapter viene rilasciaa una nuova versione della libreria con problemi di incompatibilità? Si crea un nuovo Adapter per quella versione. L'adapter precedente resta.
Se poi ci sono problemi con la libreria nuova, magari poi potrebbe essere rimossa. E quindi eliminare l'adapter precedente o modificarlo potrebbe portare problemi perché se non ce lo siamo salvati poi dovremmo riscriverlo. Meglio preservarlo.

Meglio legarsi alle interfacce che alle implementazioni concrete, rende più flessibile il codice.

Adapter a due vie: mette a disposizione sia i metodi dell'interfaccia target sia i metodi dell'interfaccia di libreria. Cioè, dà una scelta: vuoi invocare i metodi di libreria direttamente o vuoi invocare i metodi definiti dall'interfaccia target? Dà due modalità di interazione.

Posso realizzare un adapter a due vie con soluzione chiamata ObjectAdapter? Sì, basta che all'interno dell'implementazione di Adapter aggiungo i metodi della classe di libreria. Adapter a due vie NON è il classAdapter. Ma il classAdapter è sempre a due vie.

Test di precondizioni e postcondizioni. Quale variante utilizzo? L'objectAdapter, se ho la situazione della slide iniziale del diagramma, se istanzio Adapter, al suo interno vi è un'istanza di Adaptee. Posso distinguere il momento della creazione di Adapter dal client dal momento in cui istanzio Adaptee. Questa istanziazione può venire dopo, quando poi avrò bisogno di chiamare un metodo della libreria. Con l'objectAdapter posso rimandare l'istanziazione di Adaptee. Se la rimando, questa tecnica è nota come **lazy initialisation**

L'inizializzazione va fatta piano. Se ne faccio troppe all'inizio, ho dei tempi di attesa lunghi, che si sommano. Se le inizializzazioni le faccio a poco a poco, potrei non doverle fare mai. Con l'object adapter possiamo gestire questa tecnica, che ci può servire in molti casi.

Con la class adapter non possiamo avere la lazy initialisation, perché l'istanza è tutt'una: vengono creati insieme Adapter e Adaptee.

Con class adapter abbiamo l'adapter a due vie. Siamo sicuri di volerlo?
Il Design Pattern Adapter porta un rallentamento dell'esecuzione, ma è trascurabile, non ce ne preoccupiamo.

Design Pattern Facade

Come una sorta di facciata di un edificio/chiesa. L'intento è quello di fornire un'interfaccia unificata che vada, per l'appunto, ad unire varie interfacce separate verso un unico sottosistema. Quindi si vuole definire un'interfaccia di alto livello che renda il sottosistema più facile da usare.

Problema: spesso si hanno tante classi che svolgono funzioni correlate e l'insieme delle interfacce. Queste interfacce hanno interazioni, per esempio, con le classi A,B,C.

Tuttavia, se cambio una delle classi A,B,C, poi sarò costretto a modificare anche le interfacce che si relazionano con la classe modificata. Ci conviene per questo mettere un'interfaccia in mezzo, semplificata, e all'occorrenza avere solo una classe da semplificare.

Facade: classe che mettiamo in mezzo, offre interfaccia semplificata e di alto livello al client.

Supponiamo che A,B,C abbiano rispettivamente 3,5,6 metodi. L'interfaccia Facade non avrà 14 metodi, perché sennò non sarebbe un'interfaccia "semplificata". Fornisce solo quelli più importanti.

Con il D.P. l'unico a interagire con i Client è il Facade. La propagazione delle modifiche si ferma al Facade, e la compilazione delle modifiche si ferma pure al Facade. In generale, questa è una cosa che fa la differenza con progetti molto grandi.

Se ho solo Client1 e Client2, e ho le classi A,B,C, e Client1 comunica solo con A, come gestisco bene le separazione tra le due parti (client e sottosistema)? Se tutte le classi stanno all'interno della stessa cartella, all'interno del codice posso utilizzare una qualunque classe che sta sulla stessa cartella. In Java è possibile che una classe contenga al suo interno l'implementazione di un'altra classe. Potrei dividere sotto cartelle diverse le varie classi, oppure metterne una dentro l'altra (?).

Vd. esempio lib == null, socio == null, etc sono **precondizioni** (come nell'adapter).

Se metto gli allineamenti di condizione, il codice diventa più indentato e meno leggibile. Quindi uscire quando nel codice una condizione non è verificata conviene. Invece di fare if else annidati e mettere un return solo alla fine.

Java

Ciclo for avanzato: togliamo dei dettagli senza però comunicare meno informazioni al programmatore. Meno dettaglio non significa meno informazione.

In generale, questo ragionamento si può applicare in modo più esteso: Java infatti nasce come linguaggio imperativo, ma da Java 8 (2014) sta includendo caratteristiche della programmazione funzionale. Ottica: dare più core, e non frequenza maggiore (si sovraccarica troppo sennò il singolo core). Quindi questo porta al parallelismo.

Funzioni anonime: espressione lambda

Come le arrow function di Javascript. Frecca: a sx si mettono i parametri (0 o più). Con 1 solo parametro non mettiamo parentesi tonde (se le mettiamo non è un errore a livello di sintassi). Con 0 (o 2+) parametri, le tonde devono essere messe. A destra poi c'è la funzione effettiva.

In Java per implementare interfacce si possono usare classi anonime. Si può scrivere il codice della classe all'interno di un'altra classe, ma senza darle un nome.

Quello evidenziato in giallo è ridondante: è qualcosa che già il compilatore sa, perché è quello che vuole il costruttore di Sera. Quindi può essere eliminato per rendere il codice più compatto e leggibile.

Espressione Lambda: utile per implementare un'interfaccia con un solo metodo.

E se avessi definito un'interfaccia con più di un metodo, questa cosa funziona ancora o no? Non si può fare. Il compilatore non ha la possibilità di scegliere: quello che gli passo è l'unica cosa che può utilizzare per l'interfaccia. Non farebbe distinzione fra i due parametri. Dovremmo dirgli a cosa si riferisce ogni espressione lambda, ma questo non è previsto per il linguaggio.

Interfacce che definiscono un solo metodo: **interfacce funzionali.**

Parti di libreria Java riprogettate per prendere in ingresso un'espressione lambda.

Stile dichiarativo vs imperativo: il primo è più "conciso", e si legge più velocemente.

Novità con lo stile di programmazione funzionale: possiamo passare come parametro a una funziona un'altra funzione. E in quel caso la funzione a cui viene passata l'altra funzione è nota come funzione di alto livello.

Tutti i tipi della gerarchia string, etc, hanno bisogno di stream. Per questo motivo, stream viene ereditato dall'interfaccia Collection. "Metodo stream implementato

nell’interfaccia Collection”: è un’eccezione, solitamente le interfacce devono solo scrivere la firma dei metodi, non implementarli. Esiste la possibilità di avere metodi di default, e questi sono implementati dentro l’interfaccia.

Metodo filter: prende in ingresso l’espressione lambda. Operazione intermedia, può essere eseguita in un momento successivo rispetto a dove è inserita. Viene eseguita quando è necessario.

Fondamentali: filter, map, reduce. Compito: hai una lista, sai qual è il tipo contenuto all’interno della lista, e devo contare, per esempio, quanti sono gli elementi maggiore di 5. Oppure devo filtrare la lista tenendo solo quegli elementi.

Lezione 04/05Metodo Filter

Questo metodo è utilizzato per rispondere a domande del tipo: “Si abbia una lista di String, contare quante volte è presente un certo valore”.

Programmazione fluent: perché non ci fermiamo. Invochiamo in maniera concatenata, continuando a esprimere quello che desideriamo venga eseguito a runtime. Quindi niente metodi separati da punti e virgola, ma concatenazioni.

Filter ha uno stream in ingresso e uno in uscita, quello in uscita è composto dai valori selezionati dall’espressione lambda.

Metodo count: restituisce il numero di elementi presenti nello stream. Restituisce un tipo long.

Filter: operazione detta **lazy** o intermedia. Questo perché può essere eseguita in un momento successivo rispetto a dove è inserita. Cioè non è eseguita dove noi la scriviamo, ma un po’ dopo. Perché viene rimandata? Per fare sì che vengano inserite delle ottimizzazioni, per evitare overhead. Esempio: se avessi messo due filter, uno sopra l’altro: sintatticamente si può fare.

Riflessioni sull’implementazione di filter: con un ciclo for, che scorre i vari elementi della lista. Quindi se mettiamo due filter una dopo l’altra, saremmo costretti a scorrere due volte la lista, e quindi deve essere ottimizzata [da rivedere, non mi torna]. Per questo la filter non deve essere eseguita subito.

Più filter, dal punto di vista della leggibilità, è migliore. Se iniziamo a mettere degli AND diventa meno leggibile.

Approfondimento sull'ottimizzazione di filter dal compilatore: il vantaggio della lazy evaluation è dato dal fatto che i due filter vengano uniti in un unico filter. Non tanto perché questo porta a un numero di iterazioni minore (il che è falso, a livello computazionale sono pressoché identiche). Sono identiche in quanto l'unico filter che si ottiene ha un AND fra le due condizioni, e ovviamente l'AND è cortocircuitato. Questo porta a fare esattamente lo stesso numero di operazioni dei due filter. Il vero vantaggio è dato dal fatto che così non si perde tempo ad allocare liste/strutture dati intermedie. Con i due filter, viene creata prima una struttura dati con gli m elementi finali, e poi un'altra con i k elementi finali. Più lavoro per il garbage collector. Con l'unico filter unito, invece, si fa direttamente una lista con k elementi finali. Risparmio notevole. Stesso risultato. Consiste in questo l'ottimizzazione del compilatore (o meglio, dell'architettura della libreria stessa a runtime).

L'esecuzione del metodo filter viene rimandata quindi il più possibile. L'operazione terminale count genera un valore e forza l'esecuzione delle precedenti operazioni.

Ricordiamo che il valore di ritorno di filter è uno stream, e di count è un long.

Le operazioni che restituiscono uno stream sono operazioni **intermedie o lazy.** Questo perché possiamo concatenarle fra di loro. Possiamo distinguere se un'operazione è intermedia o terminale in base al tipo del valore di ritorno.

Esempio: quanti elementi della lista hanno lunghezza 5? Filtriamo tutte le stringhe con lunghezza 5 con filter, e poi le contiamo con count.

Predicate

Il tipo Predicate è parametrizzato. Rappresenta un'interfaccia funzionale, ovvero un'interfaccia che definisce un solo metodo. È una funzione che restituisce un boolean.

Esempio di predicate/funzione anonima: positive. Restituisce True solo per elementi ≥ 0.

@FunctionalInterface: fa parte della libreria Java. Informa il compilatore che l'interfaccia ha un solo metodo. Serve per proteggere il codice. Ci fa ricordare che in quel codice abbiamo usato solo espressioni lambda. Ci protegge dal non scrivere altri metodi senza pensare che in realtà quell'interfaccia l'avevamo creata per avere un solo metodo.

Metodo definito all'interno dell'interfaccia Predicate si chiama test. Non stiamo mettendo il nome di un metodo. Questa è semplicemente una variabile di tipo Predicate.

Map

Metodo che opera sugli stream, e prende in input un'espressione lambda. Quando chiamiamo Map, noi vogliamo, per ogni istanza che abbiamo nella stream, fare una trasformazione. In Map si applica uno stream di ingresso, e tramite l'espressione lambda che gli passiamo, ci restituisce uno stream di uscita. Corrispondenza 1-a-1. Dimensione dello stream di ingresso uguale alla dimensione dello stream di uscita.

Filter serve solo per fare selezioni, Map serve solo per fare trasformazioni.

Breve excursus su equals(): se dobbiamo confrontare dei numeri in Java, == va bene. Per le stringhe, a volte == ci dà il risultato che ci aspettavamo, altre no. Quindi dipende. Per questo conviene usare il metodo equals. All'interno del runtime di Java ci sono delle "ottimizzazioni". Se metti una stringa con la lettera a singola, prendo la lettera. Quando prendo stringhe molto lunghe, invece, fa il confronto con la lunghezza della stringa. E per questo motivo potrebbe dare risultati sbagliati.

Reduce

Riduce, dall'intero stream, applicando un'operazione a tutti gli elementi dello stream, e vuole ottenere un unico risultato.

Primo parametro: di tipo T, detto identità, dello stesso tipo degli elementi della stream. Passiamo come secondo parametro un'espressione lambda, e che ritorna un elemento di tipo T.

Si usa quando si vuole passare da un insieme di valori ad un singolo valore, per es. per ottenere la somma. Il primo parametro, in questo caso, sarebbe 0. Questo perché rappresenta l'identità della somma. Perché X + 0 fa X. Rappresenta anche il punto di partenza della somma.

(a, v) → indica che vengono passati due parametri, a e v. a è il totale che si è accumulato fino a quel momento (all'inizio 0), e v gli elementi dello stream che

man mano vengono sommati. L'accumulatore viene inizializzato col valore del primo parametro, quindi 0. Non fa alcuna trasformazione di tipo.

Reduce è anche presente nella versione senza il primo parametro. In alcuni casi non è banale capire che primo parametro passare. E quindi usiamo reduce con un solo parametro. Se usiamo questo reduce, nel caso in cui lo stream abbia valori numerici il risultato non cambia. Se non gli passiamo 0, il primo passaggio che facciamo è: prendiamo i primi due elementi e li sommiamo. Senza identity, facciamo un'iterazione in meno. Però c'è un **avvertenza:** quando usiamo reduce con i due parametri, reduce ci restituisce un unico elemento di tipo T. Cosa può succedere in questa versione? Supponiamo che la filter abbia passato a reduce uno stream vuoto. Reduce ci restituisce 0. E se usiamo reduce con un solo parametro? Non ci restituisce zero. Ci restituisce un tipo Optional. Tipo definito all'interno dele librerie Java che fa da contenitore. All'interno ci può essere il tipo T. Se lo stream era vuoto, all'interno del tipo Optional che fa da contenitore, non conterrà niente. Valore restituito da reduce sempre diverso da Null.

Reduce è un'**operazione terminale** perché non ritorna uno stream, ma un valore. Le ottimizzazioni sono possibili se concateno operaazioni che dopo restituiscono stream, e quindi la reduce non porta alcuna ottimizzazione.

Lezione 06/05

Lo stream può essere utilizzato una volta sola, se lo scorriamo e ci operiamo sopra una volta, non potremo rifarlo.

Se usiamo la seguente sintassi:

```
nomeClasse::nome_metodo_classe
// È un "puntatore" al metodo. Non stiamo chiamando il metodo
in quel momento,
// Ma stiamo creando un riferimento al metodo.
// SENZA parentesi tonde.
// Il metodo deve ovviamente avere gli stessi parametri di in
gresso
```

```
// e tipo di ritorno dell'espressione lambda che invece avrem
mo potuto scrivere.
```

Quindi le sintassi possibili sono:

1. Scrivere noi la nostra funzione lambda.
2. Utilizzare una funzione già definita all'interno di una funzione, come visto poco sopra.

In questo caso stiamo scrivendo nomeClasse per indicare la classe, ma se volessimo passare l'istanza? Se il metodo lavora sull'istanza, ovvero sui suoi attributi, allora viene passato invece del nome della classe il nome dell'istanza.

Come funziona questo riferimento? Tramite una sorta di chiamata a funzione.

Classi di libreria Java mai modificata, restano sempre tali e quali. La JVM controlla se sono quelli originali. Se qualcuno ha tentato di modificarle, blocco l'esecuzione, per garantire la sicruezza.

Reduce, map e filter in che classe sono implementati? NON in Stream, perché Stream è un'interfaccia. La classe in cui vengono implementati questi metodi non ci viene fatta sapere.

Il tipo presente nello stream dobbiamo sasperlo. Il tipo dobbiamo saperlo, conoscerlo, in entrambe le sintassi possibili.

Tipi primitivi: int, float, double, long. Definiti nel linguaggio Java.

Tipi definiti come classi: iniziale maiuscola. Stesso nome, tranne int che fa eccezione e si chiama Integer. float è Float, e così via. Presentano quindi dei metodi con codice. I tipi primitivi sono semplicemente tipi, non hanno codice. È questa la differenza.

Importante: la lista è parametrizzata con i tipi che sono classi, **MAI** con tipi primitivi.

```
List<int> l // è SBAGLIATO
List<Integer> l // Corretto
// Poi dentro la lista posso metterci il valore numerico solt
anto
// e basta, senza dover fare new di qualcosa. Non cambia null
```

```
a, ma
// quando lo definiamo deve essere un Integer.
```

Passaggio automatico tra int e Integer e viceversa. **Unboxing** e **autoboxing.** Cioè, se io ho un tipo definito come classe richiesto dalla Lista, ma gli passo un int, non è un problema, il compilatore lo permette.

Differenza tra stile imperativo e funzionale

Lo stile imperativo è quello a cui siamo abituati, quello funzionale invece è con metodi come map, reduce e filter.

Programmazione funzionale: manifesto di più l'intenzione di quello che voglio fare rispetto a quello che faccio. Capire un codice in stile imeprativo richiede di doverlo leggere tutto fino alla fine, e i passaggi riga per riga non chiariscono "gradualmente" ciò che vuole fare il codice. Se vedo che inizializzo una variabile a 0 e poi scrivo un for, senza leggere la riga del ciclo for non ho capito nulla di quello che vuole fare il codice, mentre lo stile funzionale mi fa capire gradualmente ciò che vuole fare il codice. Leggere il codice in stile funzionale mostra più intenzione per quello che si vuole fare, ed è più facile capire cosa vuole fare quella riga di codice, e anche il codice che devo scrivere è più piccolo.

Dichiarativo: metodo di libreria che mi fa dei controlli, es. contains. Mi verifichi se è contenuto o meno? Poi come lo fai non importa.

Funzionale è diverso: se tu mi dici che quell'elemento è presente, allora poi devi eseguire quel codice che io ti passo. Funzionale è di più rispetto a dichiarativo.

Max

MAX: contenuto nel tipo stream della libreria standard Java.

Max prende come valore di ingresso un'operazione che serve a confrontare due elementi.

Ma massimo estratto in base a cosa? Se fosse su tipi numerici non dovremmo dirglielo, già è conosciuto. Ma come si calcola il valore massimo tra due istanze di Persona etc? Questo perché la lista contiene dei tipi che abbiamo definito noi. Dobbiamo dirgli come estrarlo questo massimo. Quando utilizziamo il metodo max

dobbiamo fornirgli un pezzo di codice che gli permette di fare il controllo tra le istanze della lista.

Dobbiamo dirgli l'attributo su cui confrontare, es. Persona::getEta. Dopo di che, questo attributo lo diamo in pasto a un altro metodo comparing, di tipo Comparator, che costruisce una implementazione di un Comparator che sa fare il confronto in base a quell'attributo passato.

Max possiamo utilizzarla passandogli due attributi e poi usare l'espressione lambda. Possiamo utilizzarla in entrambi i modi, sia sfruttando il valore positivo o negativo della sottrazione sia usando Comparator.

Se vogliamo mettere il risultato di uno stream in un contenitore stabile e permanente, dobbiamo chiamare `.toList()` invece di chiamare collect e Collector. Questa versione potrei usarla quando devo aggiungere delle variazioni. Collector ha anche altri metodi che potrei imparare. Invece di `.collect(Collector.toList())` scriviamo direttamente toList.

Programmazione Parallela

Quando chiamiamo questi metodi sugli stream, l'esecuzione può essere sequenziale. Es. prendo un elemento dalla lista, lo seleziono/trasformo e così via, e poi passo al prossimo.

Es. se prima ho messo filter e poi map e poi toList, il primo elemento lo prendo, faccio filter, supponiamo di tenerlo, lo trasformo con la map, e poi lo metto in lista. E così via.

Tuttavia, posso anche utilizzare gli stream in **modo parallelo.** L'esecutore è la JVM. Nella mia applicazione, quello che ho scritto lo do in pasto all'esecutore del JVM. Se la JVM e l'hardware hanno deciso, mi supportano, che l'esecuzione può avvenire in parallelo, allora avviene. Come programmatore devo stare attento: il risultato che produrrei in modo sequenziale, è uguale all'esecuzione parallela? Quello che ho messo alla fine della lista potrebbe essere controllato prima di uno che è in mezzo alla lista. Da programmatore devo verificare se l'ordine di esecuzione è importante: se non lo è, posso eseguire in parallelo. Se preso l'elemento, l'unica cosa importante è l'elemento in sè, e non lo stato esterno, allora posso eseguire in parallelo. Se i filter e la map modificano lo stato esterno, allora non posso eseguire in parallelo.

Per eseguire in parallelo, devo chiamare il metodo `parallelStream` e basta, non devo preoccuparmi di nient'altro. L'esecuzione diventa più veloce.

Non sono io programmatore quanti thread lanciare e se lanciare più thread, ma la JVM. Dentro le librerie Java, vengono prese delle decisioni. Es. se hai pochi elementi nella lista, far partire più thread non è giustificato per quel piccolo risparmio. Se ho un numero sufficiente di elementi, allora quel tempo in più speso di overhead per far partire i thread è giustificato dal guadagno ottenuto nell'esecuzione.

Quando utilizzo ArrayList e faccio andare in parallelo, ho un guadagno più alto rispetto ad altre strutture come LinkedList. LinkedList costringe l'esecutore a dire: devo eseguire sull'i-esimo elemento ma prima devo trovarlo. Anche gli altri tipi, come vector e treesetm sono un po' più lenti rispetto ad ArrayList.

In generale, se scriviamo il ciclo for e non utilizziamo gli stream, con un singolo thread, siamo un po' più veloci. Lo facciamo per leggibilità e chiarezza del codice. Inoltre, scrivere gli stream è utile perché poi per passare a programmazione parallela basta scrivere parallelStream. Col ciclo for dovrei invece mettermi a scrivere tanti istruzioni che lanciano i thread e così via, e quindi ci vuole più tempo. In generale conviene usare gli stream perché sono più chiari e leggibili, anche se sacrifichiamo un po' di performance (poco). Quando passiamo a parallelStream, miglioriamo tantissimo l'esecuzione.

Passando da una versione di Java a una successiva posso avere guadagni di prestazioni senza cambiare il codice, perché internamente introducono delle ottimizzazioni delle prestazioni. Es. anche nel passaggio da Java 25 a Java 26.

Metodo sort

Come da nome, produce un risultato ordinato. Gli elementi che abbiamo nello stream sul quale sort viene invocato, possono essere dei tipi qualsiasi, non sono tipi per cui il possibile ordinamento è già noto a priori (es. int, float, etc). Bisogna anche fornire al metodo la caratteristica sulla quale fare il confronto per capire se unoì oggetto è maggiore o minore a un altro.

Analogo al max. Quando vogliamo stabilire se un valore è più grande di un altro, o gli passiamo un'espressione come visto prima, oppure si passano i due attributi etc, come prima.

Metodi findAnye e findFirst

Metodi disponibili sul tipo stream, come filter, map, reduce, max, count, toList, collect, forEach, sort, distinct.

Es. trovare persona che si chiama Mario con altezza almeno 170. Se un filter ha individuato un elemento, ti fermi. Ritorna Optional perché potrebbe anche non trovarne nessuno.

Perché uso findFirst e findAny? Se li uso senza parallelismo, il risultato è praticamente uguale. Con parallelismo, ovviamente è diverso. Per findAny il primo che trova l'elemento blocca l'esecuzione di tutti. Con findFirst, se eseguo in parallelo mi viene assicurato che l'elemento trovato sarà proprio il primo presente nella lista. Operazioni cortocircuitate, perché mi evitano della computazione, cioè evitano di dover scorrere necessariamente tutta la lista.

Operazioni stateless e stateful

Operazioni come map e filter vengono dette stateless, ovvero non hanno bisogno di conoscere tutta la lista, possono funzionare sul singolo elemento. Es. la filter opera sulla lista guardando ogni volta il singolo elemento.

Operazioni sorted e distinct sono dette stateful, perché per poter fare l'ordinamento o eliminare i duplicati devo conoscere tutta la lista. Non posso agire sul singolo elemento e basta, devo confrontare l'elemento considerato con altri, necessariamente.

Caso particolare: reduce e max. Né stateles né stateful. Accumulano il risultato. È vero che prendono un valore alla volta, ma poi questo viene confrontato con un altro elemento della lista.

Una cosa che possiamo fare con gli stream è generarli.

Due operazioni: iterate e generate. Iterate restituisce uno stream infinito e ordinato dato dall'esecuzione iterativa di una funzione f applicata inizialmente ad un elemento seme, quindi produce uno stream di seme, f(seme), f(f(seme)), e così via.

Generate: altro modo per generare dei valori senza prendere in ingresso nessun parametro.

Metodo limit: molto importante con iterate e generate. Questi due metodi non si fermerebbero mai se non mettiamo la chiamata a limit. Indica quanti valori devono essere presenti per finire l'invocazione.

Se scriviamo limit(1), es. dopo la filter, è esattamente il comportamento di findAny. Per limit il valore di ritorno è il tipo degli elementi dello stream, per findAny è Optional.

Se siamo interessati a trovare i primi tre elementi che soddisfano una condizione, mettiamo limit a 3.

E se ci sono meno di tre elementi? Ce ne restituisce meno, scorre tutta la lista e poi si ferma. La limit ci assicura che al massimo avremo tre elementi.

Debug

A volte, quando si scrivono delle operazioni sugli stream, es. map, poi filter, poi limit e poi forEach, è un problema. Fare debug sugli stream è difficile, perché solo alla fine ottengo il risultato. E i risultati intermedi? Come controllo che siano quelli che mi aspetto?

Usando il **metodo peek:** non modifica lo stream in input, e mi ci fa fare delle operazioni sopra, come una print. Utile per fare debug. Raccomandazione: usarla solo in questo modo, per la stampa. Non è buon uso modificare lo stato di persistenza del sistema dentro la peek. Bisogna solo stampare.

Lezione 11/05

[da recuperare]

IntStream: non è una sottoclasse/superclasse di Stream. In IntStream sono presenti alcuni operazioni non presenti in Stream, e viceversa.

IntStream: rappresenta uno stream di valori int.

Metodo rangeClosed: restituisce una sequenza di int nell'intervallo specificato (estremi inclusi).

Metodo sum: somma tutti i valori presenti nello stream fra loro. Questo metodo sum, però, non esiste per Stream, ma solo per IntStream.

Metodo mapToObj() restituisce uno stream di oggetti a partire da un IntStream.

Per passare da Stream a IntStream: devo invocare sullo stream il metodo mapToInt().

Per passare da IntStream a Stream: devo usare il metodo boxed(), che restituisce uno Stream di Integer.

Esempio: passare da Stream a IntStream e sommare tutte le lunghezze.

Float stream: non c'è bisogno di impararli. Una volta che abbiamo capito IntStream, sappiamo usare anche DoubleStream e FloatStream.

Qual è il vantaggio di scrivere un metodo per intero o scriverlo mettendolo dentro una variabile? In generale, è più breve. Fa le stesse cose, ma sintatticmanete è più semplice.

E tra la variabile e l'espressione lambda?

Quando l'espressione lamdba non prende niente in ingresso, dobbiamo comunque mettere le parentesi tonde e la freccetta.

La variabile sup, che è Supplier, non contiene una stringa, ma un'espressione lambda [vedi slide].

sup mi riesce a dare una stringa, mentre s è una stringa. La stringa s si prende col get da sup.

Utile perché questo pezzo di codice può calcolare una parte di stringa, es. stringa + contatore. Infatti utile per definire il tipo a runtime, e per prendere la fotografia di una stringa nel momento esatto in cui lo vogliamo.

Nota: forEach consuma lo stream, peek si usa per debug.

Composite

Non vogliamo fare distinzione fra oggetti semplici e composti. Vogliamo organizzarli in una struttura ad albero.

Còmposit

Quando questo D.P. è ben utilizzato? Quando abbiamo degli oggetti

Se facendo una visita trovo un elemento composto, allora questo elemento non è foglia ed è radice di un sottoalbero.

Non devo distinguere fra semplice e composto, voglio trattare le due tipologie di oggetti allo stesso modo.

Es. gestione di file. Metodo apri su una cartella deve essere lo stesso di quello usato per aprire un file specifico.

Altro esempio: rappresentazioni di elementi grafici, come figure semplici (linea, box, etc) e figure composte (raggruppamenti di figure semplici e composte).

Component: interfaccia che rappresenta elementi semplici e non.

Per la classe composite, l'oggetto composto contiene all'interno tanti elementi. Se pensiamo alla struttura ad albero, l'oggetto composto è un nodo con figli. Un nodo che ci va arrivare ad altri oggetti semplici o composti, che possiamo contenere all'interno dell'oggetto composto. Oggetto composto contiene all'interno oggetti di tipo component, ovvero in quella lista possono andare sia istanze di tipo leaf (compatibili con component) e di tipo composite (compatibile con component).

Se dentro composite dichiarassi una lista di istanze di tipo leaf, potrei mettere leaf, ma non istanze di composite.

Quello che facciamo dentro composite per invocare il metodo, è quello di invocare il metodo con lo stesso nome sugli oggetti contenuti nella lista.

Simbolo simile a un rombo: **aggregazione (o diamond)**. "Questa classe è definita come un aggregamento delle altre classi". Non deve fare aggiunte particolari, deve solo invocare i metodi sui suoi oggetti. Significa che non deve aggiungere altro codice, ma solo eseguire i metodi sui singoli elementi.

Aggregazione: chiamiamo il metodo e abbiamo finito.

Senza aggregazione: chiamiamo il metodo e poi facciamo altre operazioni, es. controlli etc. Quindi aggiungiamo del codice oltre a chiamare il metodo.

Chiamo su ogni elemento, e ogni elemento o fa l'operazione del metodo, o chiama il metodo su ognuno degli oggetti contenuti.

Se tolgo dall'aggregato l'oggetto semplice, l'oggetto semplice continua a esistere? Quando ho un'aggregazione, sto supponendo che le istanze raggruppate all'interno dell'aggregazione, debbano continuare a esistere anche se le rimuovo dall'aggregazione. Altro concetto che può essere utilizzato è quello della **composizione.** Penso a un raggruppamento di istanze in cui non ha senso se le istanze le tenessi separate. Cioè, queste istanze hanno senso solo se composte,

singolarmente no. Es. rappresentazione di figure, se ho rettangolo in mezzo ad altre figure, e lo separo, anche singolarmente ha senso di esistere. Esempio: se ho un elemento ordine con determinati dettagli, e una classe stato_ordine. L'ordine può essere inviato, confermato, etc. Quindi classe di informazione che si riferisce allo stato dell'ordine. Stato_ordine ha senso che esista senza che ci sia l'ordine di riferimento? No.

Ordine [rombo] freccia stato_ordine.

Il simbolo usato è il **rombo annerito** che indica la **composizione.** L'istanza di stato ordine deve essere sempre associata a un'istanza di ordine.

Oggetti semplici e composti possono essere aggiunti a runtime a un oggetto composto. Il client può invocare un'operazione di aggiunta di oggetti semplici a un oggetto composto.

Aggiunta o rimozione di qualunque elemento da una aggregazione può essere fatta in qualunque momento a runtime.

Sull'oggetto aggregato, dovrò usare add e remove.

Due implementazioni leggermente diverse, bisogna fare una scelta:

1. Implemento metodo add dentro Leaf e lo lascio col corpo vuoto. Il compilatore non può dirmi che manca quel metodo, e la classe non è astratta. Se viene chiamata add sull'istanza di leaf: non fare niente. Così ho garantito trasparenza, il chiamante rispetto all'operazione add o remove è di tipo leaf o composite. Se però chiamo add su leaf, a livello logico è un errore perché non succede niente.
2. Altra scelta: poiché non ha senso che add e remove vengano invocate dentro leaf, non le definisco a livello di interfaccia, e le definisco solo dentro Composite. Per invocare questi elementi devo legarmi al tipo Composite. Ho sicurezza, ma non ho trasparenza.

Quindi: o li definisco solo dentro composite, e ho sicurezza ma non trasparenza, oppure li definisco nell'interfaccia e ho trasparenza ma non sicurezza, e a runtime dovrò fare dei controlli.

La trasparenza ce l'ho finché non mi serve chiamare add o remove.

```
Component c
c.sposta();
c.add(qualcosa); // Il compilatore, nel caso in cui abbiamo s
icurezza
// ma non trasparenza, mi blocca, perché add non è un metodo
di Component.

// Potrei fare un casting e forzare il compilatore a fargli f
are la add
Composite c1 = (Composite) c // "Ti assicuro che c a runtime
sarà una variabile
// di tipo Composite
c1.add(); // Così mi sono dovuto legare al tipo Composite. Pe
r questo non ho
// trasparenza, perché devo legarmi subito a Composite.
```

A volte, se ho quest'albero di istanze a run-time, e sono a un certo punto dell'albero nell'esecuzione di un certo nodo, potrei chiedermi: qual è il nodo dentro al quale sono aggregato? Mi serve capire, quando sono all'interno di un certo nodo, qual è il contenitore di un certo nodo. Nel momento in cui faccio il raggruppamento, lo devo impostare a quel valore.

Lezione 18/05

Esercizi come: riconoscere il design pattern dalle relazioni fra le classi/interfacce in teoria adesso non è più presente come esercizio. Tuttavia, può essere utile esercitarvisi, vd. esempio su Composite.

Classe ProdSemplice, per essere completa e fare sì che soddisfi l'interfaccia prodotto, deve implementare il metodo descrizione. Non c'è corrispondenza fra il nome del parametro che inserisco in record e il metodo descrizione di ProdSemplice.

Nota: `Prodotto::print` non è un'invocazione di metodo, infatti mancano le parentesi tonde. Inoltre, a sinistra dei due punti va il nome della classe, non dell'istanza.

Esercizio: si fornisce interfaccia, nome del metodo, tipo di ritorno. Viene fornita classe Leaf tramite record. Es. fornisce metodo che si chiama auto, realizza il ruolo di Leaf per interfaccia Composite, e chiede di completare l'implementazione del D.P.

Decorator

Parole chiave: dinamicamente e aggiunta di responsabilità. Con responsabilità intendiamo "vogliamo un frammento di codice, un comportamento, in più". Dinamicamente: aggiunta a runtime. Quindi vogliamo un'aggiunta di responsabilità a runtime. Stranezza: come facciamo ad aggiungere codice a runtime? O ce lo abbiamo o non ce lo abbiamo. Aspetto da chiarire.
Importante: aggiungere responsabilità **non a una classe, ma a un oggetto.** Responsabilità intesa anche come una nuova funzionalità (scorrere una lista, fare una determinata operazione, etc). I decorator forniscono un'alternativa felssibile all'implementazione di sottoclassi per estendere funzionalità. Se si parlasse di aggiungere responsabilità a un'intera classe, potremmo aggiungerla creando una sottoclasse.

Approccio flessibile: avvolgere (wrap)) il componente in un altro oggetto che aggiunge una responsabilità.
Rappresentazione: un oggetto posso immaginarlo come un cerchio, con uno strato più esterno e uno più interno. Da pensare come a una cellula. Quando usiamo un oggetto wrapper, questo è un cerchio che avvolge l'oggetto già esistente. Avvolgiamo e nascondiamo l'oggetto. Così l'oggetto ha una sua interfaccia, e da lì decidiamo cosa far propagare all'interno e cosa no. I livelli (ovvero il numero di cerchi che si avvolgono) non devono essere un numero fisso. Dobbiamo avere la possibilità di mettere quanti livelli vogliamo in base alle nostre esigenze, senza porci limiti.

Il problema dell'ereditarietà è che ci richiede di scrivere troppe classi. Es. se ho tre classi c1, c2, c3 con determinate funzionalità. Se poi voglio ereditare sia da c1 che da c2, allora devo creare la classe c1c2. Idem per c1c3 e per c2c3. Capiamo che è un approccio scomodo.

Decorator: è una classe. Contiene attributo di tipo Component. Come lo inizializziamo questo attributo e cosa ci facciamo poi?

ConcreteComponent, essendo un’implementazione dell’interfaccia, deve necessariamente avere operation. A meno che non sia astratta e quindi non vada implementata da altre sue sottoclassi. Nel nostro caso, è una classe che deve implementare l’interfaccia, non è astratta. Il comportamento base è dentro ConcreteComponent, non dentro Decorator.

Il ConcreteDecorator ha una linea diretta verso ConcreteComponent? No. Quando al ConcreteDecorator voglio invocare il comportamento base, mi rivolgo alla superclasse. Il comportamento aggiuntivo è definito dentro operation. Non gli devo cambiare nome. Se cambiassi nome, starei stravolgendo il D.P. e quindi il client deve accorgersi del nome nuovo, e quindi non c’è più compatibilità con l’interfaccia Component. Il nome non devo cambiarlo, altrimenti non sto più implementando il D.P. Decorator, perché starei costringendo chi invoca questo metodo a capire se è un metodo che aggiunge funzionalità oppure no. Non è che tutti quelli che utilizzano l’istanza devono sapere se sta venendo usata una funzionalità aggiuntiva oppure no.

Il nucleo centrale di tutti questi “avvolgimenti” è il ConcreteComponent. Non conviene aggiungere varianti dentro ConcreteComponent come metodi pubblici. Si snatura il D.P. e sicuramente si sbaglia qualcosa.

Usare Decorator come abstract non conviene perché sporca il codice e si fa fare più lavoro al programmatore perché poi servirà una classe non astratta che deriva da Decorator. Anche se non mi serve istanziare Decorator, non va fatto abstract.

Lezione 22/05

Singole responsabilità aggiuntive che si aggiungono al singolo oggetto, e non a tutta la classe, es. posso configurare a runtime: mi serve questa cosa? e quest’altra? Allora possiamo usare il D.P. Decorator.

Ci sono volte in cui non posso implementare un ConcreteDecorator che sia “sconnesso” da un ConcreteComponent, ovvero non fatto ad hoc per il ConcreteComponent.

È anche possibile rimuovere le possibilità aggiuntive, non solo metterle. Per esempio con wrapper.

Perché il Decorator non è una classe astratta: ripropongo nuovamente il motivo. Nel decorator ci mettiamo un attributo e un metodo che poi vengono usati anche nelle sottoclassi. Questi metodi li vogliamo implementare così, vogliamo che facciano qualcosa. Non vogliamo scrivere queste codice nelle sottoclassi, concreteDecorator. Se fosse astratto, questo codice dovremmo implementarlo sotto.

Nota: in realtà possiamo mettere la keyword abstract e non dà errore. Si può usare anche senza usare metodi astratti.

L'ordine di estrazione delle responsabilità con wrapped deve seguire l'ordine iniziale. È come spacchettare qualcosa. Si parte dallo strato più esterno.

Supponiamo di avere un utente che possiede il biglietto. Cioè possiede un riferimento al biglietto. Quando poi il biglietto ottiene un comportamento aggiuntivo, es. uno sconto, l'utente che ha il riferimento al biglietto avrà il biglietto segnato scontato o no? La risposta è sì.

Il client però non conosce e non deve conoscere gli strati esterni. Il client deve avere come interfaccia dell'oggetto sempre la stessa, al di là del fatto che abbia comportamenti aggiuntivi o meno. Se ci va bene questo comportamento, ovvero che il client non può "vedere" gli strati più esterni (anche se ovviamente queste aggiunte si notano dalle chiamate di metodo), allora ha senso usare il D.P.

Il ConcreteDecorator può essere implementato bene proprio perché questo è indipendente dal comportamento base. Il modo in cui implemento ConcreteDecorator non dipende dall'implementazione del comportamento base. L'unica cosa che posso conoscere del comportamento base è il valore di ritorno della chiamata al metodo. E questo non è, ovviamente, guardare nello specifico l'implementazione del comportamento base. Sono cose diverse. Quindi sono indipendenti. Se i due algoritmi dovessero lavorare insieme, in modo più stretto rispetto all'invocazione di quel metodo e prendere il valore di ritorno, allora non sarebbero indipendenti.

Individuazione delle classi

Sostantivi: tipicamente suggeriscono classi/attributi.

Verbi: suggeriscono operazioni (metodi).

Design Pattern Bridge

Design Pattern strutturale.

Intento: obiettivo di disaccoppiare le astrazioni dalle implementazioni, in modo che possano cambiare in modo indipendente. Cambiamento più comodo rispetto a doverle modificare rispetto a tenerne conto momentaneamente, o guardarle insieme.

Con astrazione potremmo pensare che ho una classe astratta o un'interfaccia, e con implementazione pensare all'effettiva implementazione dell'interfaccia o dei metodi. Queste sono effettivamente indipendenti (astrazione e implementazione) nel codice. L'interdipendenza comunque vale: se modifico l'interfaccia poi dovrò fare modifiche anche nell'implementazione. Uno potrebbe pensare questo, ma **non è quella corretta.**

Quello che voglio: nella mia applicazione sto costruendo tante astrazioni. Da vedere come una classe astratta aggiunta per avvicinarmi al dominio.

Con questo D.P. io organizzo tutte le astrazioni con le loro implementazioni. Implementazione da intenedersi come: a un certo punto, la mia classe es. Studente, dovrà legarsi con una rappresentazione a livello di interfaccia verso l'utente, ovvero cosa di Studente devo visualizzare? E come?

Potrebbe capitare che quando modifico/aggiungo una astrazione, dovrei poter aggiungere tante classi. Se devo disegnare le informazioni dello Studente, e ho le informazion di Studente e un'interfaccia verso l'utente, Studente deve potersi interfacciare con gli elementi grafici. Mi lego alla piattaforma 1 o alla piattaforma 2? Output che devo dare all'utente: lo do attraverso l'una o l'altra libreria?

Classe Studente con StudenteAWT e StudenteWEB. Se vogliamo legarci a solo due piattaforme, va bene. Se però aggiungiamo un'altra piattaforma saremo costretti a creare uno Studente legandoci alla piattaforma. Problema: oltre all'astrazione Studente, mi serve, per esempio, Registro, mi serve Professore. Se aggiungo astrazioni, che succede? Supponiamo di avere solo due piattaforme, AWT e WEB. Quando mi devo legare alla piattaforma, poi ho bisogno di più classi. Quindi avrò RegistroAWT e RegistroWEB, idem per Professore. Lo stesso per tutte le classi che dovranno usare quella libreria. Così ho tre astrazioni (Studente, Registro,

Professore), e due librerie. Quindi sei implementazioni. Se da 2 passo a 3 librerie, allora ho nove implementazioni.

Voglio quindi disaccoppiare implementazioni da astrazioni. Se le separo bene, allora ho più possibilità di fare variare l'una e l'altra in modo più flessibile. Così ogni astrazione nuova non ha bisogno di n implementazioni. Se mi serve una piattaforma o un'astrazione in più, con il bridge, mi servirà sempre e solo una classe aggiuntiva. L'aggiunta scala linearmente.

Altro esempio tipico per il Bridge: figure geometriche. Linee, rettangoli , cerchi.

Soluzione

Abstraction: definisce dei metodi generali per tutte le astrazioni che ci servono. È una classe che sarà inizializzato con un attributo di tipo Implementor. Linea verso Implementor: Abstraction tiene un attributo di tipo Implementor, che verrà poi implementato dai ConcreteImplementor.

RefinedAbstraction: sottoclassi di Abstraction che usano i metodi della Abstraction. Ogni RefinedAbstraction al suo interno implementa le operazioni della Abstraction messe a disposizione. Dentro la sottoclasse chiamiamo i metodi della superclasse Abstraction.

Interfaccia Implementor

Concrete Implementor: implementano l'interfaccia. Si collega a una delle piattaforme. Avremo un ConcreteImplementor che sa disegnare sulla piattaforma P1, un ConcreteImplementor che sa disegnare sulla piattaforma P2, e così via. Dire che V2Drawing dovrà chiamare line sulla piattaforma significa che verrà chiamata su una classe che conosce Line.

Si chiama **Bridge** perché si forma una gerarchia di Forma e una gerarchia di Drawing (Abstraction e Implementor). E si può vedere come una sorta di ponte con i suoi supporti.

Es. se devo disegnare un rombo, posso farlo facendogli chiamare le funzioni della superclasse (il metodo drawLine quattro volte). Tuttavia, a volte ci saranno delle nuove classi che devono disegnare figure che non possiamo disegnare con le funzioni della superclasse, e in quel caso bisognerà fare un aggiunta, ma dovrebbe essere un caso raro.

Questo D.P. ci permette anche di modificare la piattaforma utilizzata a runtime.

Se ho Client che chiama draw, questo potrà essere chiamato sia da Forma che Rettangolo etc. Metodo che facciamo definire alle sottoclassi. Forma come classe astratta.

Factory Method, Prototype, etc hanno obiettivo comune di far legare il client a un'interfaccia. Per Composite c'era Component, per Factory c'era Product, e così via.

Le RefinedAbstraction non devono per forza fornire gli stessi metodi. Posso implementare anche metodi propri fra RefinedAbstraction. Le implementazioni quindi possono aggiungere dei metodi.

Fine design pattern strutturali.

Extreme Programming

Tipo di sviluppo **agile.** Manifesto per lo sviluppo agile, fatto nel 2001. Molto usato in Nord-Europa e a livello mondiale. Dà dei principi da seguire per l'organizzazione del lavoro.

La documentazione può passare in secondo piano rispetto all'interazione col cliente. Il cliente deve poterci dire quando desidera cosa vuole. Si deve poter comunicare sempre. In netto contrasto con il processo a cascata. Qui, invece, persone privilegiate rispetto a tutto il resto.

Non ci sono piani fissi. Non c'è una sequenza di azioni per lo sviluppo. Si ha un'idea di quello che si deve fare. Anche in fasi avanzate dello sviluppo, si deve poter modificare il sistema software. È importante che tutti i programmatori abbiano molta esperienza, e che tutti abbiano la motivazione per creare un buon sistema sw. Persone motivate che abbiano l'obiettivo di sviluppare qualcosa a cui tengono. Non deve essere solo lavoro per paga, ma l'avere motivazione per ottenere un ottimo risultato.

Processo di sviluppo **XP** documentato da Beck

Cunningham: creatore della tecnologia wiki wiki, alla base di wikipedia. Idea: ho un HTML, posso modificarlo? wiki wiki permette di fare questo.

Processo che dà tanti consigli. Es. avere un incremento continuo del sistema sw. Altro consiglio: migliorare continuamente il codice e avere piccoli team di sviluppo.

Piccoli: da 2 a 12. Novità, perché nei processi a cascata le persone che ci lavoravano erano diverse centinaia. Avere rilascio del codice ogni 2 settimane e avere un piccolo team di lavoro è un grosso cambiamento.

Si basa su **12 pratiche.** Devono essere messe in pratica tutte, nessuna esclusa.

1. **Gioco di pianificazione:** ha come obiettivo una raccolta di requisiti. Al cliente viene chiesto di scrivere delle story card. Pezzettini di carta, con dimensioni 12*7. Formato cartoline. Si dà un pacchetto di queste carte, storycard, vuote. Di piccole dimensioni, e sono separate le une dalle altre. Gli si chiede di scrivere cosa deve essere realizzato, un requisito, sulla storycard. Poiché quello che bisogna scrivere è poco, non deve essere troppo complesso. Se lo spazio non basta, bisogna ricominciare. Si strappa e si ricomincia. Ovviamente si danno al cliente delle indicazioni, e si cerca di capire cosa ha scritto. Se non si capisce, si ricomincia. Successivamente, gli sviluppatori fanno una stima del requisito. Si fa una stima del tempo necessario per realizzare il requisito. Se il tempo stimato per la realizzazione del requisito è grande, si strappa e si divide. Es. 2 giorni è troppo. Non è una persona che prende le decisioni. Si prende una decisione collettiva. Quello che si suggerisce è mettere più priorità alle cose più rischiose, ovvero fare prima. Le funzionalità più rischiose, quelle che interessano di più al cliente, vanno fatte prima. Per ogni cosa che il cliente ha in mente, gli si fa scrivere una storycard. Si avranno alla fine tante storycard, come le distribuiamo? Prima si fanno quelle a priorità più alta, poi quelle a priorità più bassa. Questo gioco di pianificazione viene fatto ogni due settimane. Vengono predisposte le schede per la terza settimana. Ogni due settimane, il cliente può dirci quali sono le sue priorità. Non bisogna pianificare troppo in avanti. È importante che le storycard le scriva il cliente anche perché in questo modo si prende la responsabilità di quello che sta chiedendo. Quello che viene richiesto dal cliente deve essere testabile. Se non lo è, si strappano. Le storycard devono avere una data e un numero progressivo. Il posto in cui vengono inserite le storycard è una lavagna, una storyboard. Divisa in zone. Una colonna per la prima settimana, una seconda colonna per la seconda settimana, e una terza colonna per tutto quello che eccede le prime due settimane.
2. **Pair Programming**: ci si organizza, come sviluppo, a coppie. Si prende la storycard e si va nella postazione. Un unico computer. Due sviluppatori. Uno

dei due sviluppatori usa mouse e tastiera, l'altro legge e dà consigli. Uno sviluppatore è detto driver (tastiera e mouse) e l'altro è detto navigator. L'idea è che ci sia un confronto continuo sul software. Si confrontano sulla scrittura del codice. Entrambi i programmatori sono esperti ed entrambi motivati. Ciascuno dà consigli all'altro senza dare consigli troppo elementari. Bisogna essere bene o male allo stesso livello. Se tra i due c'è un grande divario di competenze, non funzionerà. Se quello con più esperienza ha la tastiera, non avrà bisogno dei suggerimenti di quello che poca esperienza, mentre se quello che ha poca esperienza ha la tastiera, non riuscirà a comprendere i consigli dati dall'esperto. Il navigator non deve avere un altro pc, ma può avere della carta, dove scrive i test. Quando vengono scritti i test durante l'implementazione XP? Durante l'implementazione, in quanto vengono scritti su carta. La documentazione del processo XP è molto molto povera, va in secondo piano. Non essendoci documentazione, come si capisce che sistema software si stanno realizzando? Dato che si fa pair programming, le coppie sono sempre diverse, e in questo modo viene sparsa la conoscenza sul codice.

3. **Piccole Release**: fornire una nuova versione del sistema sw ogni due settimane. Non significa che è completo, ma vogliamo avere qualcosa di molto piccolo, ma funzionante. Nel gioco di pianificazione, il cliente che scrive le storycard, dovrà venire tutti i giorni in sede. Questa pratica è nota come cliente in sede. Se l'azienda che ci chiede il sw, dovrà fornirci una persona della sede cliente, darla alla sede degli sviluppatori, a disposizione di tutti. Avendo a disposizione il cliente, durante il pair programming se ci sono dei dubbi su una storycard, o ci si accorge che non si è capito veramente a fondo il requisito, si chiedono al cliente dei chiarimenti. Il cliente in sede scriverà dei test funzionali. E così si capirà che tipo di output vuole dato un certo input.
4. **Metafora:** se stiamo di fronte a un problema, e non sappiamo che scelta fare, questa ci aiuta a fare la scelta. Potrebbe essere relativa a come voglio realizzare il sistema software. Una frase da ricordare. Es. come visualizzo questo risultato/avviso? La frase potrebbe essere “La UI è un desktop”. Non è la pratica più importante di tutte, dato che comunque tutte queste scelte devono essere discusse più in dettaglio. Probabilmente è la meno importante.

5. **Design Semplice:** quando devi realizzare una certa funzionalità, pensa alle classi che servono, interzione fra le classi, e pensa alla progettazione più semplice possibile. Ovviamo con Design Semplice si vuole anche un codice di qualità. Design semplice che cambia man mano che aggiungo i requisiti. Quindi non faccio una progettazione ampia scrivendo tutti i requisiti in una volta, ma la costruisco piano piano. Build Design Upfront: non è un approccio da seguire in questo caso, perché porta a progettare tutto all'inizio. Pay per use: pagare solo per quello che serve. Se sto usando una lista, non penso subito al composite, non è necessario. Non bisogna aggiungere parti non necessarie.
   Documentazione: qual è il momento per fare documentazione per il processo XP? Si fa tramite le **CRC (Class Responsibility Collaboration)** per documentare il design. Carta grande 7×12. In alto c'è scritto il nome della classe, piccolo rettangolino. Sotto, in due rettangoli separati di circa le stesse dimensioni, si mette **responsibility** e **collaboration.** In responsibility indichiamo le responsabilità della classe in questione, e in collaboration mettiamo le classi con cui interagisce. Entrambe le sezioni vengono scritte in **linguaggio naturale.** Quindi non scriviamo il nome dei metodi. La documentazione nel processo XP consiste solo di queste due parti: storycard per progettazione, e CRC per la documentazione. Si punta a **codice leggibiile,** che è quasi **auto-documentato.** La presenza di codice di qualità mitiga la necessità di documentazione. Altre cose che mitigano la necessità della documentazione: presenza del cliente in sede e Pair Programming, non si ha una grande documentazione sul codice, ma il codice è stato realizzato da due programmatori, quindi ci sono due programmatori che conoscono il codice di quella parte e quindi si può chiedere a loro. Quindi c'è una collaborazione. L'idea dell'XP è che tutti possono interagire facilmente fra di loro. Quindi non c'è il momento in cui diciamo "questo è il processo dell'inizio della documentazione". Questa fase non esiste.
6. **Testing:** molto importanti nel processo di sviluppo XP, utilissimo per sopperire alla mancanza di documentazione. Quando vengono scritti i test? Durante l'implementazione del codice (durante la Pair Programming, su carta). I test possono essere scritti sia prima che dopo la scrittura di codice funzionale. Possiamo anche adottare la TDD. Quando eseguo i test? Spessissimo, anche ogni pochi minuti. I programmatori eseguono i test sulla singola funzionalità.

Se pensano che funzioni, integrano la parte funzionante nel sistema software. Supponiamo siano 12 sviluppatori. Quindi 6 team. Ognuno prende una storycard. Alcune possono richiedere una giornata di lavoro, altre due ore, altre mezza giornata, etc. Supponiamo di metterci 4 ore per storycard. Non ci interessa che finiscano tutti sincronizzati. Anzi, non finiranno sincronizzati. Ma tutti avranno, alla fine del loro lavoro, un componente aggiuntivo. Ma quanti test eseguo? Per un sistema software di medie dimensioni, ci sono **milioni di test,** con circa **un migliaio di classi.** Quanto tempo ci vuole a eseguire questi test in tempi utili? Supponendo che il sistema software realizzato non sia banale, ci vuole tempo. Non possiamo separare il sistema sw in parti separate, sennò diventa solo test di componenti (che abbiamo fatto man mano). Dobbiamo verificare che tutte le parti interagiscano correttamente. Non bisogna testarlo solo localmente, ma globalmente. In generale, questo numero di test è comune a tutti i processi di sviluppo.

7. **Possesso del codice collettivo:** se ho scritto del codice buono, ma qualcun altro ha scritto del codice cattivo, non bisogna assolutamente pensare “non è un problema mio/non è colpa mia”. È impensabile all’interno del processo di sviluppo XP. Il codice è collettivo, la responsabilità è di tutti. Se si hanno altri modi per scrivere meglio del codice esistente, bisogna proporlo. Ricordiamo che alla base dell’XP ci stanno dei programmatori motivati.
8. **Integrazione continua:** il sistema sw va sempre integrato, le varie parti vanno unite.
9. **40 ore a settimana:** ogni programmatore deve lavorare circa 8 ore al giorno. Lavorare un numero di ore maggiore è sbagliato, porta il programmatore a ragionare meno bene e con più fatica.
10. **Cliente in sede:** già visto prima. Se non si riesce ad avere la persona sul posto, si può optare per farlo lavorare da remoto. Approccio strategico per avere un prodotto completo e funzionante per come lo vuole il cliente. Senza questo, come sviluppatori non riusciremmo a creare il prodotto in tempi ridotti ed esattamente secondo le aspettative del cliente.
11. **Standard di codifica:** non bisogna fare costruzioni complicate per il design, ma mantenere le cose semplici. Il codice deve apparire uniforme, tutti devono seguire le stesse convenzioni.
    Es. l’utilizzo di Tab in determinati casi anziché dello spazio etc. Convenzione

sull’uso dei commenti, sul come usarli, sul nome dei metodi delle classi etc. Bisogna essere pronti a modifciare il proprio modo di fare, perché potrebbero esserci delle convenzioni migliori suggerite da altri sviluppatori del team. Se non si conosce bene il team, bisogna prima mettersi d’accordo sulle convenzioni da usare, e confrontarsi per capire qual è l’approccio migliore.

12. **Refactoring:** miglioriamo la struttura del codice senza influenzarne il comportamento. Naturalmente serve fare refactoring controllato tramite testing.

Considerazioni conclusive: XP avvantaggia il codice rispetto alla documentazione. I principi XP non erano nuovi nemmeno quando sono stati scritt nel libro per l’XP. Già i programmatori esperti lo mettevano già in atto. Ci sono state delle evoluzioni dopo il 1999.

- Nel 2008 è stato introdotto stackoverflow, utile per documentazione domanda-risposta.
- Dal 2014 è stato introdotto il **Mob Programming:** da 2 programmatori si passa a 5, con in aggiunta il cliente e l’architetto del software (chi conosce tutto il software).
- Dal 2020 circa sono nate delle alternative al Pair Programming in sede, facendo sessioni di lavoro da remoto, tramite, per esempio, Live Share su VSCode. Anche il cliente può essere da remoto, e non per forza in presenza.
- IA Generativa.

Nota: non seguire una pratica del XP, comporterà dei disagi futuri.

Tecnica di sviluppo Scrum

Toyota Production System: copiato nel mondo dello sviluppo, sia in XP che in Scrum. Backlog: lavoro arretrato. È lavoro da fare.

Ci sono andare anche i requisiti non funzionali. Item ancora più piccoli di quelli che scriviamo nella storycard. Il **Product Backlog** è una lista di cose da fare, una lista universale. Lo Scrum Team trasforma i requisiti in codice. **Sprint:** intervallo di lavoro, ogni sprint dura un mese di tempo. Quali sono gli obiettivi della singola sprint viene deciso dal team che lavora, ma anche da chi deve prendere decisioni strategiche su tutto il prodotto. In generale dalle persone interessate

(bisogno/strategico) e decidono cosa devono fare. Ogni sprint può aggiungere elementi nel backlog.

Empirismo: conoscenza deriva dall'esperienza, e le decisioni sono prese in accordo a quel che è osservato (Locke, Berkeley, Hume).

Lean: essere snelli, evitare lo spreco, e basarsi su quello che serve in un determinato momento. Design semplice.

Toyota: fare rifornimenti sulla base della necessità dei pezzi. Altri, invece, prendevano molte componenti in grandi quantità e le mettevano in magazzino.

Pllastri Scrum

Trasparenza: il processo e il lavoro dentro devono essere visibili a chi svolge il lavoro e a chi riceve i risultati.

Lezione 3 GiugnoEventi di Scrum

Sprint: sono iterazioni che hanno lunghezza fissa, tipicamente un mese.

Sprint Planning: produce la pianificazione del lavoro da svolgere nello Sprint. Dura al massimo 8 ore per uno Sprint al mese.

Bisogna capire cosa si intende con "ho finito": si intende quando abbiamo terminato di scrivere il sistema software? O diciamo che abbiamo finito quando abbiamo fatto anche i test? Oppure quando abbiamo fatto il deploy? A volte questi step non sono così distanti fra loro, a volte ci vuole poco tempo per passare al successivo. Bisogna decidere cosa intendiamo con "ho finito questo obiettivo". Per questo è necessaria una **Definition of Done.** Può usarsi anche per piccoli obiettivi interni allo Sprint, non per forza per tutto lo Sprint. "Abbiamo realizzato questa componente dello Sprint", ma esattamente cosa si intende? Quindi questa Definition of Done vale sia per i piccoli step che per tutto lo Sprint.

Scrum produce vari **artefatti:** ovvero risultati di varie attività che sono state fatte, "sottoprodotti".

Product Backlog: si estraggono le varie cose da fare, da cui si ricava la lista delle cose da fare.

Product Goal: si hanno degli obiettivi noti come **Sprint Goal,** ovvero l'obiettivo della Sprint. Il Product Goal descrive lo stato futuro del prodotto. Si trova nel Product Backlog.

Sprint Backlog: è composta da uno Sprint Goal e da un set di elementi Product Backlog (ovvero alcune delle cose da fare, definite nel Product Backlog).

Increment: qualcosa che porta verso il Product Goal.

Nota importante: Scrum non sostituisce XP. Con Scrum le cose rimangono più generali. Es. non descrive esattamene il processo di raccolta dei requisiti. Non parla di come programmano i programmatori (es. Pair Programming). Quindi XP e Scrum possono coesistere, anche perché Scrum è molto più "generico" come processo di sviluppo. Noi non abbiamo fatto tutti i processi di sviluppo, ma i più importanti sono XP e Scrum.

Review: dove siamo arrivati con gli obiettivi che ci siamo posti. Il prodotto a che punto è?

Con la **Retrospective** invece ci si chiede se il team sta andando bene, se ci sono cose che potrebbero essere fatte in altri modi etc.

C'è una nuova versione dello sviluppo Scrum da parte degli autori iniziali dello Scrum, dove verrà integrata l'IA.

Oltre al TDD, esistono il DDD (Domain Driven Development) e di recente è nato il SDD (Specification Driven Development). Sta emergendo da poco. L'idea è: se i singoli algoritmi me li dà l'IA, devo stare attento a cosa chiedo all'IA, ovvero alle specifiche, e metto in secondo piano l'organizzazione del codice. Funziona per alcuni sistemi software, ma non sempre. Ancora non è un processo di sviluppo affermato. Per progetti grandi, il rischio sono le allucinazioni degli LLM.

DDD: quando noi conosciamo bene il dominio su cui deve esistere il sistema software, riusciremo a fare software adatti a quel tipo di dominio. Bisogna comprendere bene il dominio per individuare correttamente la progettazione ad oggetti che ci serve. Esistono alcuni design pattern basati su DDD ma non sono troppo precisi come gli altri che abbiamo fatto.

Lezione 5 GiugnoDesign Pattern State

Intento: permettere a un oggetto di alterare il suo comportamento quando il suo stato interno cambia. Far sembrare che l'oggetto abbia cambiato la sua classe.

Capiamo bene da questa descrizione che lo State è un Design Pattern **Comportamentale.**

Problema:

- Il comportamento di un oggetto dipende dal suo stato e il comportamento deve cambiare a run-time in base al suo stato. Chiaramente non ci conviene risolvere il problema usando un grande blocco di istruzioni if-else all'interno dei metodi che necessitano di conoscere lo stato.
- Le operazioni da svolgere hanno alcuni grandi rami condizionali che dipendono dallo stato.
- Lo stato è spesso rappresentato dal valore di una o più variabili enumerative costanti.
- Spesso varie operazioni contengono la stessa strutture condizionale. Capiamo bene che quindi da tutti questi blocchi condizionali si può cercare di **astrarre** un unico blocco di if-else da inserire all'interno di un singolo metodo, che chiameremo ogni volta che vogliamo conoscere lo stato interno.

Soluzione:

- Inserire ogni ramo condizionale in una classe separata.
- **Context:** definisce l'interfaccia che interessa al client. Mantiene un'istanza di una classe ConcreteState che definisce lo stato corrente.
- **State:** definisce un'interfaccia che incapsula il comportamento associato ad un particolare stato del Context.
- **ConcreteState:** sono le sottoclassi che implementano ciascuna il comportamento associato ad uno stato del Context.

Collaborazioni:

- Il Context passa le richieste dipendenti da un certo stato all'oggetto ConcreteState corrente.
- Un Context può passare sè stesso come argomento all'oggetto ConcreteState per farlo accedere al contesto se necessario.

Conseguenze:

- Il comportamento associato a uno stato è localizzato in una sola classe (ConcreteState), e si parizione il comportamento di stati differenti. Per tale motivo, si possono aggiungere nuovi stati e transizioni facilmente, creando nuove sottoclassi. Incapsulare azioni di uno stato in una classe impone una struttura e rende più chiaro lo scopo del codice.
- La logica che gestisce il cambiamento di stato è separata dai vari comportamenti ed è in una sola classe (**Conext**) anziché (con istruzioni if o switch) sulla classe che implementa i comportamenti. Tale separazione aiuta a evitare stati inconsistenti, poiché i cambiamenti di stato vengono decisi da una sola classe e non da tante.
- Il numero di classe totale è maggiore, le classi sono più semplici. Non è uno svantaggio, anzì, è un grande vantaggio.

Considerazioni: notiamo che quando abbiamo fatto le varie versioni del codice, abbiamo sempre parlato di voler scrivere metodi piccoli. Di non preoccuparci di scrivere correttamente il for, perché comunque quello ce lo forniranno le librerie. Quindi non possiamo scrivere cicli for, o comunque meglio non scriverli (raramente). Quindi: scrivere poco codice, pochi cicli for, e non scrivere condizioni.

Design Pattern State: ogni blocco di istruzione condizionale mettilo in un ConcreteState a sè stante.

Tecnica di Refactoring usata: **Replace Conditional with Polymorphism**.

Il cambiamento di stato si può gestire in Context? Sì. Però il Context dovrebbe dire: da che stato parto, e a che stato posso arrivare? Quindi rimuovere i vari return degli stati nei metodi di ConcreteState.

A questo punto, il valore che potrebbe venire restituito per ogni State, potrebbe essere un booleano. Per indicare se l'operazione è andata a buon fine oppure no.

Solo il Context deve sapere che esiste l'interfaccia, esistono più classi, etc.

Questa considerazione riguarda il Facade, sono simili sotto questo punto di vista. D.P. che offriva al client un'interfaccia semplificata.

Il client che si rivolge a Context nota un vero e proprio cambiamento della classe, come se cambiasse veramente. Poi internamente cambia, perché effettivamente ci

sono cambiamenti tra ConcreteState.

```
return new Bloccato().inTesta(s)
// equivalente a

var b = new Bloccato();

var b1 = b.intesta(s)

return b1


// sul nuovo riferimento, io chiamo il metodo intesta. Quello
che
// mi ritorna in testa, lo restituisco.
//  il tipo di scrittura del codice dove tutte le operazioni
sono concatenate
// è detta "fluent".
```

Per quanto riguarda var, da Java 19 possiamo dichiarare variabili locali (non attributi) con var. Posso dichiarare solo variabili locali che inizializzo subito con var. L'inizializzazione della variabile dopo l'uguale mi permette di capire il tipo della variabile. Il tipo è presente, lo faccio individuare al compilatore, in quanto il lato destro restituisce un determinato tipo. Estremamente leggibile, anche il programmatore, e non solo il compilatore, capisce con che tipo ha a che fare. Non ci sono incertezze sui tipi e non stiamo utilizzando tipi dinamici.

Es. il valore di b1 sarà il valore di ritorno indicato dal tipo di ritorno del metodo inTesta().

```
// altro esempio:
return new Bloccato().intesta(s).paga();
```

Questo tipo di codice facilita anche la scrittura di test, dato che i metodi sono piccoli con singole responsabilità.

Design Pattern Chain of Responsibility

Se la richiesta non si sa a chi arriva, e può arrivare a un qualsiasi ricevente, noi dobbiamo farla arrivare al ricevente giusto per far eseguire il comportamento che vogliamo. Si crea una separazione fra ricevente e richiedente.

Intento: evitare di accoppiare il mandante di una richiesta col suo ricevente, dando così a più di un oggetto la possibilità di gestire la richiesta. Concatena gli oggetti riceventi e passa la richiesta lungo la catena fino a che un oggetto la gestisce.

Motivazioni:

- Maggiore flessibilità nell'eseguire la richiesta dato che possono eseguirla in molti.
- Per esempio, vogliamo fornire messaggio di aiuto in una finestra di interazione verso l'utente. L'utente potrebbe richiedere l'aiuto quando ha inserito un certo dato. Se non c'è un'informazione così specifica, cioè campo con "nome" e capisce che deve mettere nome. Gli scrivo "codice" ma non sa che codice darmi. La richiesta di aiuto che l'utente può fare in più punti del modulo potrebbe provenire da client diversi, e propagarsi verso un possibile ricevente di questa richiesta.

Soluzione:

- **Handler:** definisce l'interfaccia per gestire le richieste.
- **ConcreteHeadler:** implementa l'interfaccia Headler. Ognuna di queste implementazioni, può rispondere a una determinata richiesta. Se sa rispondere, risponde, sennò propaga verso gli altri ConcreteHandler della catena. Rischio: ci potrebbe essere una catena che non sa rispondere.
- **Client:** inizia effettuando la richiesta a un ConcreteHandler. Chi risponde non è strettamente collegato a chi ha fatto la richiesta.

Design Pattern che potrebbe essere implementato insieme al Composite. Gestisce già un insieme di elementi. Dentro Component potrebbe esserci un'operazione del tipo "richiesta" e "propagazione della richiesta". Mi hai passato un parametro? È il contesto giusto per entrare in azione? Se sì, propago.

SetNext ha un parametro di tipo [...].

Dinamicamente possiamo cambiare la catena di propagazione.

Questo D.P., in genere, non lo implementiamo come una cosa che la classe fa da ConcreteHandler, gli facciamo fare altre cose. Il Client è molto semplice, e si lega a uno dei ConcreteHandler.

Lezione 08/06Design Pattern Command

Intento: incapsulare una richiesta in un oggetto, . Quando parliamo di richieste non pensiamo semplicemente a un parametro che vogliamo inviare. Non è una semplice stringa. È una serie di **comandi**, ovvero una serie di invocazioni. Il parametro richiesto è un **oggetto che sa fare invocazioni**. Quindi si parla di una serie di comandi che si vogliono eseguire, e non semplicemente di una serie di parametri. Quindi incapsuliamo dentro un oggetto delle operazioni da far compiere a un ricevente.

Come lo facciamo? Il Client conosce una interfaccia nota come Command, e usa un'implementazione di questa interfaccia che incapsula queste operazioni da eseguire. È il ricevente dei comandi da eseguire.

Soluzione:

- **Command:** definisce un'intrfaccia per eseguire un'operazione. Ha un metodo `execute()`.
- **ConcreteCommand**: classe che implementa l'interfaccia. Cominciamo a notare che possiamo realizzare più ConcreteCommand. Se passiamo ConcreteCommandA o ConcreteCommandB, non c'è differenza. Il client si lega all'interfaccia Command. Ci basta solo passargli l'istanza giusta.
- **Invoker**: chiede al comando di effettuare la richiesta. Sa che a un certo punto deve mettere in atto una serie di attività, descritte all'interno di ConcreteCommand. L'invoker pensa: ora posso chiamare in metodo execute sull'istanza ConcreteCommand che mi è stata passata. Se cambiamo l'istanza di ConcreteCommand, l'Invoker sarà sempre quello che "avvia" le operazioni, ma ne eseguirà di diverse, dato che gli abbiamo passato un'altra istanza di ConcreteCommand.
- **Receiver:** il ConcreteCommand conosce l'istanza della classe Receiver, e invoca su questa le operazioni. Quindi chi riceve queste operazioni è la classe

Receiver.

Quindi: l'Invoker deve eseguire delle operazioni, descritte dal ConcreteCommand, e vengono eseguite sul Receiver, e quindi il ConcreteCommand conosce il Receiver. E quindi il ConcreteCommand su chi deve agire? Il Client istanzia il Receiver e il ConcreteCommand e li associa fra di loro.

Esempio di applicazione: abbiamo un menù di voci. Quando abbiamo un bottone cliccabile, abbiamo dietro il bottone un pezzetino di codice che segnala che deve eseguire determinate operazioni. Quello che può succedere una volta premuto il bottone, può essere una di varie cose. Se nella finestra di dialogo cambio il contenuto, magari il bottone è uguale, ma le operazioni poi possono essere diverse. Es. comando salva su file. Ma magari ancora non editato niente, non ho modificato il documento, etc. Il tipo di operazioni che devo fare è diverso a seconda del fatto che ci sia stata una modifica oppure no. Questo click dà vita a: esegui tutte le operazioni per il salvataggio oppure no.

Reverse dell'esecuzione

Design Pattern utile a tornare indietro rispetto alle operazioni fatte dall'Invoker. Cioè si può tornare indietro. Si può ripristinare l'ambiente rispetto all'ultima/alle ultime operazioni fatte.

L'operazione execute è per "fare qualcosa", per svolgere operazioni. Serve un metodo per tornare indietro, e quindi un metodo simile va implementato dentro ConcreteCommand. Bisogna quindi avere i dettagli sia per eseguire operazioni che per tornare indietro. Quindi il ConcreteCommand sa come conservare, e così come sa come "aggiungere", sa come rimuovere. Quindi sa fare sia la execute che l'opposto e riprendersi il dato, per poi poter nuovamente eseguire, nel caso, la execute. Oppure, un altro modo è che prima di aggiornare si fa una **copia del contenuto di Receiver**. L'importante è implementare l'operazione di reverse, poi come lo fai è un dettaglio algoritmico interno al ConcreteCommand.

Conseguenze

- Il pattern Command disaccoppia l'oggetto che invoca l'operazione da quello che conosce come effettuarla.

- I comandi diventano oggetti e possono essere manipolati ed estesi come altri oggetti.
- Si può pensare a un altro oggetto come una Macro di un singolo Command. Abbiamo pensato a Command come un'operazione "salvaFile", "apriFile", etc. Ma in situazioni più complesse, magari supponiamo che il SO ci dica che la memoria non è più sufficiente. E quindi dovremmo rimuovere nella memoria alcuni documenti aperti, e magari implementare operazioni più complesse. I singoli comandi dell'elenco di un Menù, possono essere eseguiti tutti insieme e in un colpo solo tramite un'operazione più complessa. Potremmo inserirli dentro un ConcreteCommand, che ha come Receiver degli altri ConcreteCommand. E così abbiamo creato una Macro per tanti Command, inseriti però in un solo ConcreteCommand. Un po' come il Composite che implementa l'interfaccia Component ma poi si rivolge ai vari comportamenti semplici per dire di eseguire una determinata operazione. Un MacroComando si potrebbe anche implementare tramite D.P. Composite.

Esempio: Calendario con menù

- In execute conservo il parametro che mi è stato passato, e lo conservo dentro uno stack di nome timestamps.
- Se unexecute viene invocato, mi basterà rimuovere gli elementi dallo stack timestamps. Posso implementare l'operazione di undo perché so come si comporta: aggiungere una coppia di informazioni e so come rimuovere un elemento dallo stack. L'operazione di undo è "personale" per ogni metodo di execute.
- Utilizziamo putIfAbsent perché così evito la situazione in cui metto una coppia di elementi con una chiave già presente e dove la vecchia coppai viene sovrascritta. Questo metodo fa un inserimento solo nel caso in cui quella chiave non era già presente.

D.P. Mediator

Intento: definire un oggetto che **incapsula come un gruppo di oggetti interagisce**. Il Mediator promuove il lasco accoppiamento fra oggetti poiché evita

che essi interagiscano direttamente, e permette di modificare le loro interazioni indipendentemente da essi.

Caratteristica principale: stiamo incapsulando tutte le interazioni fra gli oggetti, non un singolo comportamento o una singola relazione. Gli oggetti che collaborano smettono di conoscersi a vicenda e conoscono solo l'interfaccia del Mediator.

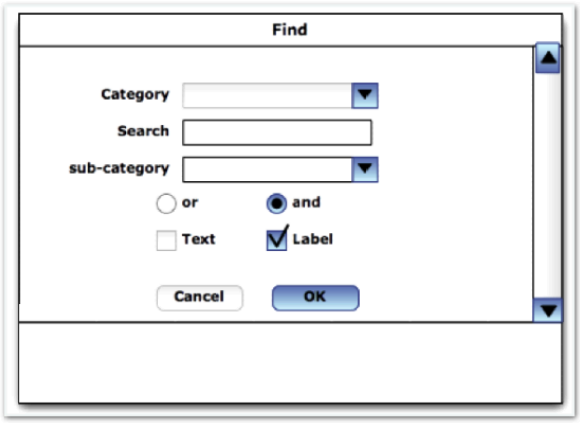
Motivazioni:

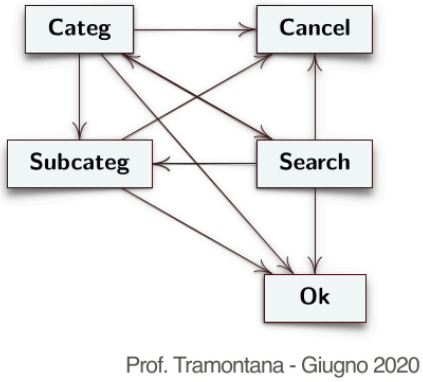
- La **distribuzione di responsabilità** fra vari oggetti può risultare in molte connessioni fra oggetti, e nel caso peggiore **un oggetto conosce tutti gli altri**.
- Molte connessioni rendono un oggetto dipendente da altri e l'intero sistema si comporta come se fosse **monolitico**.
  Inoltre potrebbe essere **difficile cambiare il comportamento del sistema** poiché il comportamento è distribuito fra oggetti. In questo modo riusare un singolo oggetto fuori contesto diventa quasi impossibile, perché per usarlo dovremmo anche portarci dietro tutta la rete di oggetti a cui è collegato.
- Si possono evitare questi problemi incapsulando il comportamento collettivo in un **oggetto mediatore separato**. Il mediatore serve da intermediario ed evita che gli oggetti dipendano fra loro. Il Mediator non elimina la dipendenza, la **ridireziona**. I Colleague dipendono ancora da qualcosa: da un'unica interfaccia stabile Mediator, e non da classi concrete degli altri Colleague.

Esempio:

- Supponiamo di sviluppare una finestra di ricerca. Ogni elemento visualizzato (testo, lista, bottone, etc) è controllato da una corrispondente classe.
- Ciascuna classe deve comunicare il suo stato alle altre per far aggiornare la visualizzazione.
- Ciascuna classe, senza un Mediator, chiamerà i metodi di tutte le altre classi, e quindi ciascuna classe è dipendente dalle altre. Per esempio: se cambiamo la Category, allora la lista delle Sub-Category disponibili deve aggiornarsi di conseguenze. Se il campo Search è vuoto, il bottone Ok probabilmente dovrà restare disabilitato perché non c'è nulla da cercare. Non è un caso che Ok sia il nodo con più freccette entranti: il suo stato enabled/disabled dipende, in pratica, dalla combinazione di tutti gli altri input.

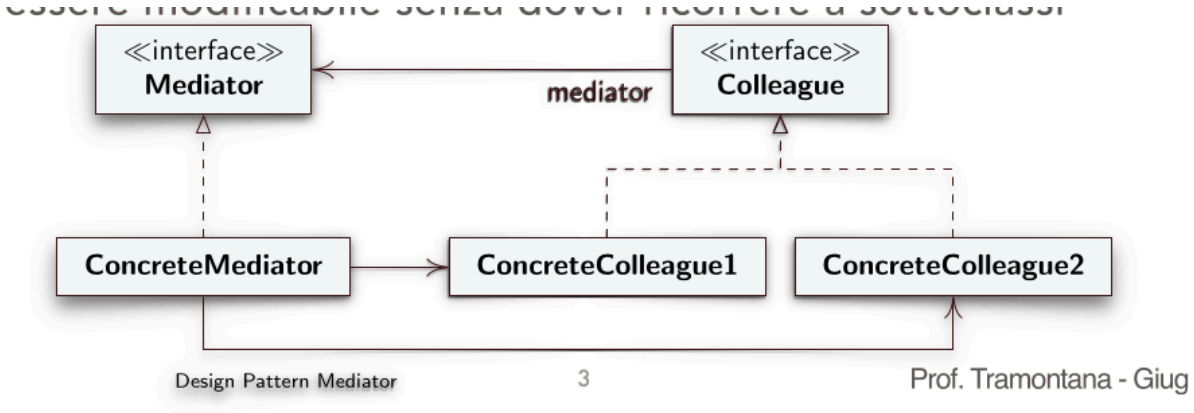
- Se volessimo riusare la classe Search da qualche altra parte, non possiamo staccarla facilmente, in quanto porta con sé chiamate dirette a Category, SubCategory, Cancel e Ok.





Applicabilità - Quando bisogna usare il Mediator?

1. Un insieme di oggetti **comunicano in modo ben definito ma complesso**. Le interdipendenze che ne risultano sono non strutturate e difficili da comprendere.
2. **Riusare un oggetto è difficile** poiché esso comunica con tanti altri oggetti.
3. Un comportamento che è distribuito fra tante classi dovrebbe essere modificabile anche senza dover ricorrere a sottoclassi. Ovvero **il comportamento collaborativo è sparso fra tutte le classi**, e per cambiarlo dovrei fare delle modifiche in molte classi. Col Mediator, per modificare il comportamento mi basta scrivere un nuovo ConcreteMediator o modificare quello esistente.



Soluzione:

- Isolare le comunicazioni (complesse) tra oggetti dipendenti creando una classe separata per esse.
- **Mediator:** definisce una interfaccia che fa punto di incontro per gli oggetti connessi Colleague.
- **ConcreteMediator:** implementa il comportamento cooperativo e coordina oggetti Colleague. Inoltre, questo mantiene i riferimenti ai suoi Colleague.
- Ogni **Colleague** conosce il Mediator, e comunica con il Mediator quando avrebbe comunicato con un altro Colleague.
- I **ConcreteColleague** mandano richieste (e ricevono richieste) a (e da) un oggetto Mediator. Il Mediator implementa il comportamento cooperativo inoltrando le richieste a opportuni ConcreteColleague.
- Perché Colleague (e quindi i ConcreteColleague) conoscono solo l'interfaccia Mediator generica, e il ConcreteMediator invece conosce i ConcreteColleague specifici? Perché, dal lato suo, il Mediator ha bisogno di chiamare metodi specifici di un ConcreteColleague in particolare, e quindi deve conoscerli. Qui notiamo ovviamente un **accoppiamento**, ma è il costo da pagare per avere il resto del sistema disaccoppiato. Il ConcreteMediator diventa il punto in cui ogni estensione del sistema si paga.
- È importante sapere che, in alcune applicazioni pratiche, **Mediator e ConcreteMediator possono coincidere**. Ciò viene fatto in quanto nella maggior parte dei casi reali non si ha bisogno di più implementazioni intercambiabili di Mediator per lo stesso gruppo di Colleague. Tuttavia, in

questo modo perdiamo la possibilità di sostituire il Mediator con un'implementazione diversa senza toccare i Colleague (es. per una **variante del comportamento di coordinamento**).

Conseguenze:

- **Centralizza il controllo**: la maggior parte della complessità che risulta nella gestione delle dipendenze è spostata dagli oggetti cooperanti al Mediator. Questo rende gli oggetti più facili da implementare e mantenere. Questo ovviamente ha anche un aspetto negativo da contemplare: il trade-off è che così il Mediator stesso sarà potenzialmente più complesso di ciascun Colleague preso singolarmente, al punto di poter diventare un monolite difficile da mantenere.
- Le classi Colleague sono **più riusabili** poiché la loro funzionalità fondamentale non è mischiata con il codice che gestisce le dipendenze. In questo modo i Colleague possono essere usati così come sono, senza bisogno di fare sottoclassi per cambiare il modo in cui operano.
- **Il codice del Mediator non è in genere riusabile** poiché la gestione delle dipendenze implementata è solo per una specifica applicazione.
- **Semplifica i protocolli a oggetti**: sostituisce relazioni **many-to-many** tra Colleague con relazioni **one-to-many** tra Mediator e Colleague.
- **Astrae come gli oggetti cooperano**: rende la mediazione un concetto indipendente, incapsulato in un oggetto a sé, permettendo di concentrarsi su come gli oggetti interagiscono separatamente rispetto a cosa fa ciascun oggetto individualmente. Così, leggendo il ConcreteMediator è possibile capire l'intera logica di cooperazione del sistema.

Considerazioni finali:

- **Quantificare "tante dipendenze":** ovviamente non esiste un numero magico che possa indicare quante dipendenze sono tante e quante sono poche. Tuttavia, la soglia indicativa è che più di 5 dipendenze sono sufficienti per poter definire una classe "troppo accoppiata". Questo viola infatti la Law Of Demeter: parla solo con i tuoi amici, non con gli sconosciuti.
- **Dipendenza comportamentale vs Dipendenza strutturale:** il Mediator non elimina il bisogno che gli oggetti dipendano l'uno dall'altro a livello di

comportamento. Se Category cambia, Sub-Category deve aggiornarsi, ovviamente questo requisito non scompare. **Quello che il Mediator elimina è la dipendenza strutturale**, ovvero il riferimento diretto.

- Prima del Mediator, la singola classe deve sapere chi avvisare, conoscere le interfacce delle altre classi, e capire cosa passare loro. Dopo l'introduzione del Mediator, la classe diventa più "generica", e tutta la specializzazione che aveva prima legata alla cooperazione con le altre classi si sposta sul Mediator.
- Se nella progettazione c'è una **relazione che forma un loop che torna su sé stessa**, è un segnale che qualcosa nel design è sbagliato. Invece, se riusciamo a disegnare le frecce di un diagramma UML senza farle sovrappore o incrociare, è un indizio che le dipendenze sono gestibili. Il contrario, invece, segnala che le dipendenze sono eccessive e quindi potrebbe avere senso utilizzare il Mediator.
- **Mediator vs Facade:** Facade semplifica un'interfaccia verso un sottosistema con comunicazione **unidirezionale**, e il sottosistema non sa che Facade esiste. Mediator invece centralizza comunicazione **bidirezionale** tra Colleague consapevoli della sua esistenza.
- **Mediator e Observer:** a volte questi D.P. collaborano, in quanto i Colleague potrebbero comunicare col Mediator usando l'Observer (dove i Colleague notificano il Mediator).

Progettare per Preservare

Principio generale: Programmare per le interfacce, non per le implementazioni.

- Creare un oggetto specificandone la classe esplicitamente **orienta a una particolare implementazione invece che a una interfaccia**, e questo può complicare i cambiamenti futuri.
  È meglio creare oggetti indirettamente.
  Pattern utili: Factory Method, Prototype.
- Le **dipendenze da operazioni specifiche vincolano a un modo di soddisfare una richiesta**, evitando che le richieste siano indicate nel codice permette di modificare a compile-time e a run-time il modo in cui le richieste saranno soddisfatte. Il problema è che il codice client chiama direttamente

un’operazione precisa per soddisfare la richiesta (es. con una catena di if/else).
Pattern utili: Chain Of Responsibility, Command.

- **Dipendenze da piattaforme hardware e software**, ovvero API esterne specifiche di una piattaforma, rendono più difficile fare il porting del sistema software verso altre piattaforme. È persino difficile tenerlo aggiornato per la stessa piattaforma. Bisogna progettare il sistema limitando le dipendenze dalla piattaforma.
  Pattern utili: Factory, Bridge. Possiamo separare l’astrazione dall’implementazione.
- **Dipendenze da rappresentazioni o implementazioni** (come un oggetto è rappresentato, conservato o implementato) rendono le classi client soggette a cambiamenti quando l’oggetto cambia. Bisogna nascondere le informazioni alle classi client, e in questo modo evitare cambiamenti a cascata.
  Pattern utili: Factory, Bridge.
- **Dipendenze di oggetti da algoritmi** che sono spesso estesi, ottimizzati e rimpiazzazti durante lo sviluppo e il riuso, provocando cambiamenti agli oggetti nel caso di cambiamento degli algoritmi. Si devono isolare gli algoritmi soggetti a cambiamenti. In questo modo, ogni volta che l’algoritmo cambia bisogna modificare l’oggetto, per questo conviene isolare l’algoritmo in una gerarchia di classe a parte, intercambiabile.
  Pattern utili: State/Strategy, Template Method.
- **Stretto accoppiamento fra varie classi** porta a sistemi monolitici. **Classi strettamente accoppiate** sono difficili da modificare, e riusare singolarmente, senza comprendere e cambiare le altre classi.
  Il lasco accoppiamento incrementa la probabilità che una classe sia riusata da sola, e che il sistema sia compreso, modificato ed esteso più facilmente. Relazionare bene le classi è difficile, le chiamate dirette creano dipendenze che si accumulano e complicano il sistema.
  Pattern utili: Abstract Factory, Bridge, Chains of Responsibility, Command, Facade, Mediator, Observer.
- **L’estensione di funzionalità tramite sottoclassi** è **riuso a scatola bianca**, e richiede profonda comprensione della superclasse e l’override di ciascuna operazione può richiedere l’override di un’altra. Può condurre a un grande

numero di classi (per estensioni).
È meglio **limitare il numero di livelli in una gerarchia** di classi e usare la composizione anziché l'ereditarietà: **black box reuse**.
Alcuni design pattern producono una progettazione in cui si possono personalizzare le funzionalità definendo solo una sottoclasse e componendo le sue istanze con le altre.
Pattern utili: Bridge, Chain Of Responsibility, Composite, Decorator, Observer, Strategy.

- Quando è **impossibile modificare classi**, per es. perché il codice sorgente non è disponibile, o quando una modifica richiede cambiamenti a tante altre classi.
  Pattern utili: Adapter, Decorator.